COMP2870 Theoretical Foundations: Calculus II

Thomas Ranner

21 April 2026

Table of contents

1 Introduction

1.1 Contents of this submodule

This part of the module introduces numerical algorithms involving derivatives and integrals. We will study these methods from both theoretical and practical perspectives, with an emphasis on real-world applications. To succeed in this part of the module, you will need to engage with both Python programming and pen-and-paper mathematical work.

1.1.1 Topics

We will have 6 double lectures, 2 tutorials, and 3 labs. We break up the topics as follows:

Lectures (*The following schedule is still to be confirmed*)

1. Introduction to ordinary differential equations and Euler's method (week 9, L1)
2. The midpoint method and systems of differential equations (week 9, L2)
3. Introduction to optimisation (week 10, L1)
4. Nonlinear solvers (week 10, L2)
5. Partial derivatives and Gradient flow (week 11, L1)
6. Using gradient flow to train a neural network (week 11, L2)

Labs

1. Ordinary differential equations (Week 9)
2. Nonlinear solvers (Week 10)
3. Using gradient flow to train a neural network (Week 11)

Tutorials

1. Ordinary differential equations (Week 11)
2. Optimisation (Week 12)

1.1.2 Learning outcomes

Learning outcomes are provided in each section of the notes.

1.2 Programming

This section of the notes links theoretical material with practical applications. In the weekly lab sessions, you will see how the methods introduced in the lectures behave in practice when implemented. Further guidance on the practical work will be provided in the lab sessions.

An important theoretical aspect of implementing these algorithms on a computer is the use of floating-point arithmetic. You will already have encountered floating-point numbers in your first-year studies, when you learned how computers store numbers and earlier this year when working on linear algebra problems. You will already know that computer cannot store every possible real number exactly. The inexactness of floating-point numbers has real consequences when performing computations.

Exploring the material in COMP2870: Linear Algebra is really helpful for understanding many of the choices that we make when forming methods here too.

We will make extensive use of `Python` and `numpy` during this section of the module. It may help you to revise some of your notes from last year on these topics.

1.3 Comments on these notes

There are two versions of these notes available: as an online website and as a PDF. I expect to make minor updates to both versions as the module progresses.

You can always find the latest versions at:

- html: https://comp2870-2526.github.io/calculus-ii-notes/
- PDF: https://comp2870-2526.github.io/calculus-ii-notes/COMP2870-Theoretical-Foundations--Calculus-II.pdf

Please either email me (T.Ranner@leeds.ac.uk) or use the GitHub repository to report any corrections.

```
This version of the notes is a5bb7ae+dirty.
```

2 Introduction to dynamic problems

Module learning objective

1. Identify some examples where differential equations arise throughout computer science and beyond,
2. Recognise parts of a differential equation in standard format,
3. Solve differential equations numerically using Euler's method.

There are many models and problems where time plays an important role.

1. Realistic behaviour within computer games requires an accurate and efficient *physics engine* which is used to predict the motion and impacts of e.g. projectiles, vehicles, people, etc. These are examples of dynamical models which must be solved and then rendered in "real time";
2. Planning and scheduling complicated robotic tasks requires solving dynamic models for how the robot will interact with its environment;
3. Most financial technologies require predicting the future price of some valuable object. This is typically done through solving dynamic models which need to be solved very quickly to achieve the best results;
4. Weather and climate predictions are made through solving very large dynamic models;
5. Dynamic problems are everywhere in scientific and engineering: e.g., chemical reactions, electrical circuits, biological populations and mechanical vibrations.

Often these models use the language of derivatives and calculus to express the underlying processes. We will spend the first two lectures of this part of the module covering how we can solve this type of dynamic problem using numerical methods.

As a computer scientist, this will help you in your future studies by introducing:

- Important methods for solving interesting problems often found in science and engineering, robotics and games design;
- The idea of splitting up a continuous problem into a finite number of steps which makes a tractable problem for a computer to solve;
- Measuring errors and efficiency of approaches with a view to balancing between them.

2.1 Rates of change

We will start with a recap of material that you learnt last year to define the objects we want to discuss. We will do this in a way that's driven by our applications and methods which will follow.

Suppose we know that a person is walking at a *constant* speed of 3 meters per second (m/s). What does that really mean? How far will they travel in:

- 0.1 seconds?
- 0.01 seconds?
- 0.001 seconds?
- 10^{-6} seconds?

What does this tell us about speed?

We have that

$$\text{distance travelled} = \text{speed} \times \text{time},$$

or equivalently:

$$\text{speed} = \frac{\text{distance travelled}}{\text{time}}.$$

What if the object's speed was not constant? For example, we could define the speed

$$S(t) = -(t-1)(t+0.5). \tag{2.1}$$

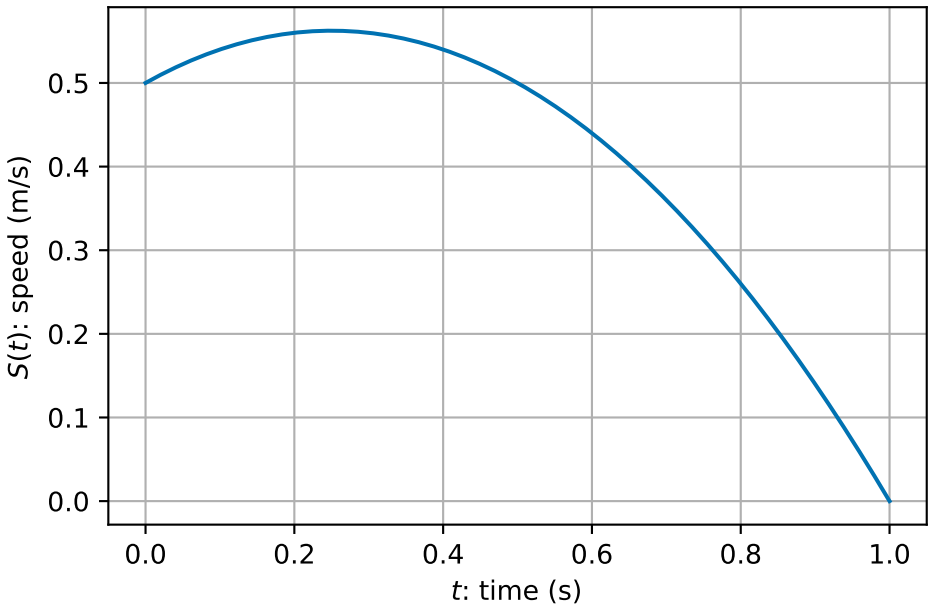


Figure 2.1: A sample object speed over time

How far would the object travel in one second now?

We could consider each tenth of a second separately and estimate the distance covered at each tenth of a second (assuming the $S(t)$ is approximately constant in each interval):

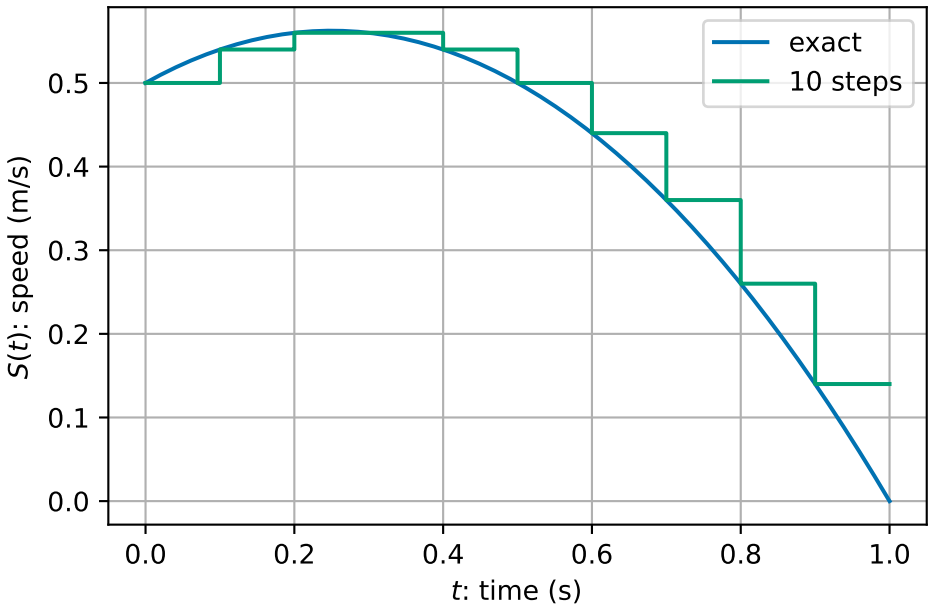


Figure 2.2: A sample object speed over time approximated over 10 steps

```
D, t = 0.0, 0.0

for _i in range(10):
    D = D + 0.1 * S(t)
    t = t + 0.1

print(D, t)
```

```
0.44000000000000006 0.9999999999999999
```

We could consider the same calculation of hundredths of seconds assuming that the speed is approximately constant in each interval:

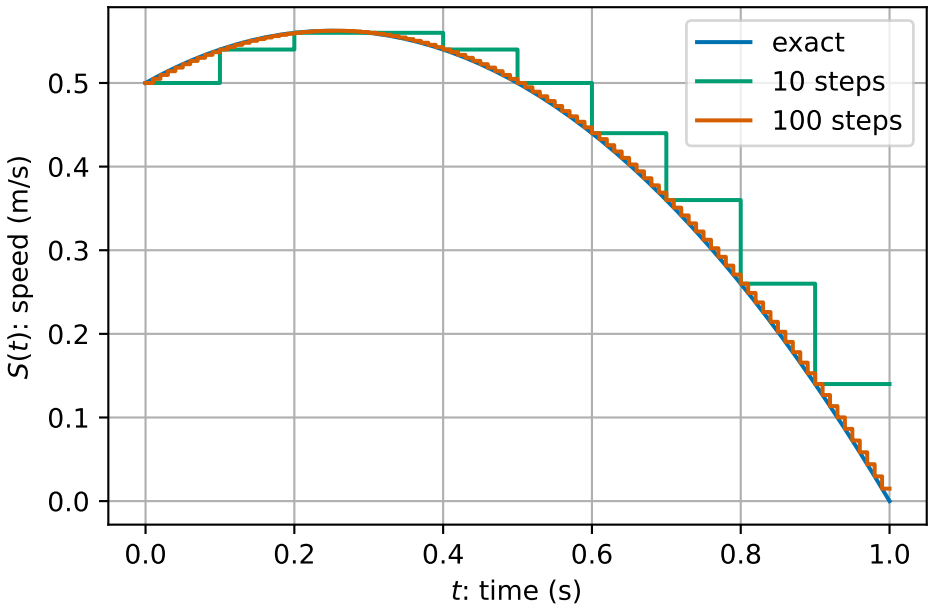


Figure 2.3: A sample object speed over time approximated over 100 steps

```
D, t = 0.0, 0.0

for _i in range(100):
    D = D + 0.01 * S(t)
    t = t + 0.01

print(D, t)
```

```
0.41914999999999963 1.0000000000000007
```

What about if we did a thousand steps

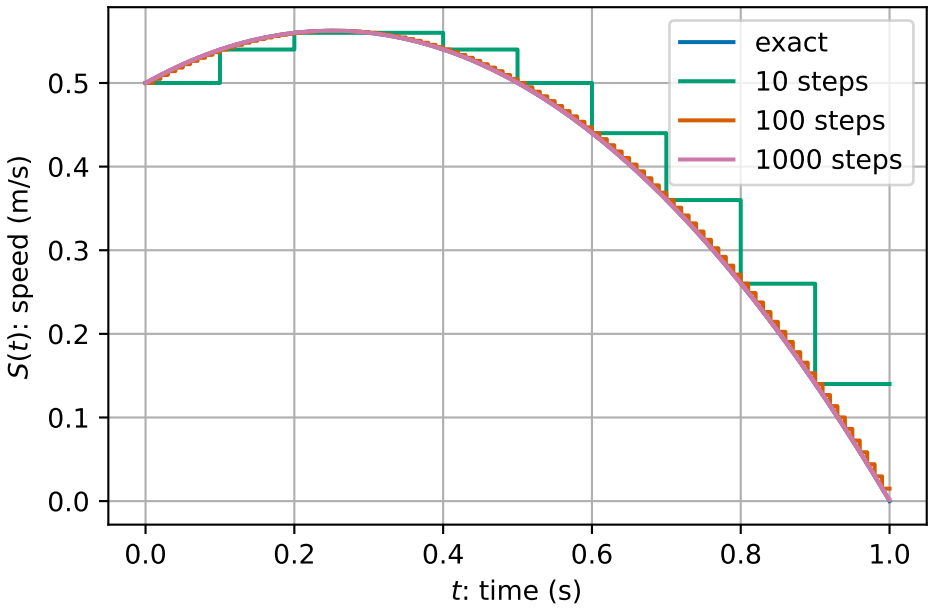


Figure 2.4: A sample object speed over time approximated over 1000 steps

```
D, t = 0.0, 0.0

for _i in range(1000):
    D = D + 0.001 * S(t)
    t = t + 0.001

print(D, t)
```

```
0.41691649999999947 1.0000000000000007
```

In summary, we get

Table 2.1: Computing the distance travelled using smaller steps.

intervals	increment size, h	total distance
1	1	0.5
10	0.1	0.44
100	0.01	0.41915
1000	0.001	0.416916
10000	0.0001	0.416692

intervals	increment size, h	total distance
100000	1e-05	0.416669

We appear to be converging to a limit as $h \to 0$...

We might express this mathematically as:

$$S(t) = \lim_{h \to 0} \frac{D(t+h) - D(t)}{h},$$

or that *instant* speed is the limit of average speed over smaller and smaller intervals.

Formally, we say that speed, $S(t)$, is the **derivative** of distance, $D(t)$, with respect to time and write $S(t) = D'(t)$.

2.2 Geometric picture

We can plot the expression for the speed, $S(t)$, from (2.1) and it's integral from 0 up to time t, which we call the distance. $D(t)$.

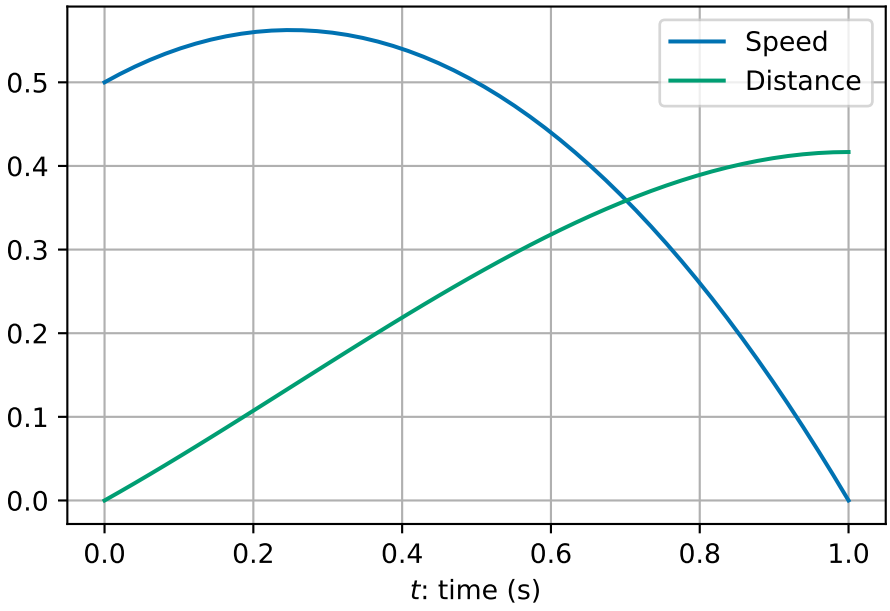


Figure 2.5: Distance and speed examples as a function of time

We see that the **gradient** (slope) of the green curve is the **value** of the blue curve.

- As the blue curve increases in value, the green curve increases in steepness.
- When the blue curve starts to decrease in value, the green curve decreases in steepness.
- By the time the blue curve has a value of zero the green curve is flat.

This gives us an alternative definition of derivative:

> The function $D'(t)$ is the function whose value is equal to the slope (or gradient) of $D(t)$ at every value of t.

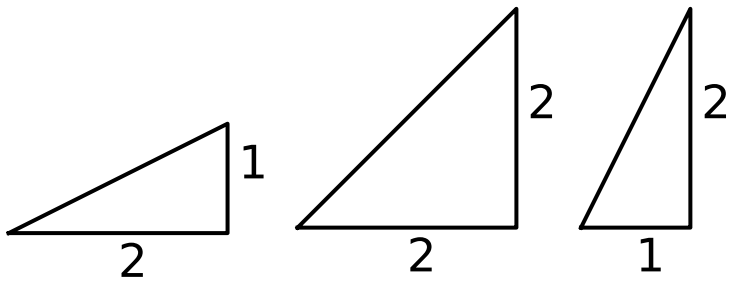


Figure 2.6: Examples of different sloped straight lines.

For a straight line, we can use the formula for a straight line to find its gradient. The equation of a straight line with slope m is given by

$$f(t) = mt + c.$$

For a curve (i.e. non-straight line), we can approximate the slope with a straight line approximation ("chord"):

$$\frac{f(t+h) - f(t)}{h}.$$

We can get a better approximation by taking a smaller value of h… This leads us back to the definition of derivative (Definition 2.1).

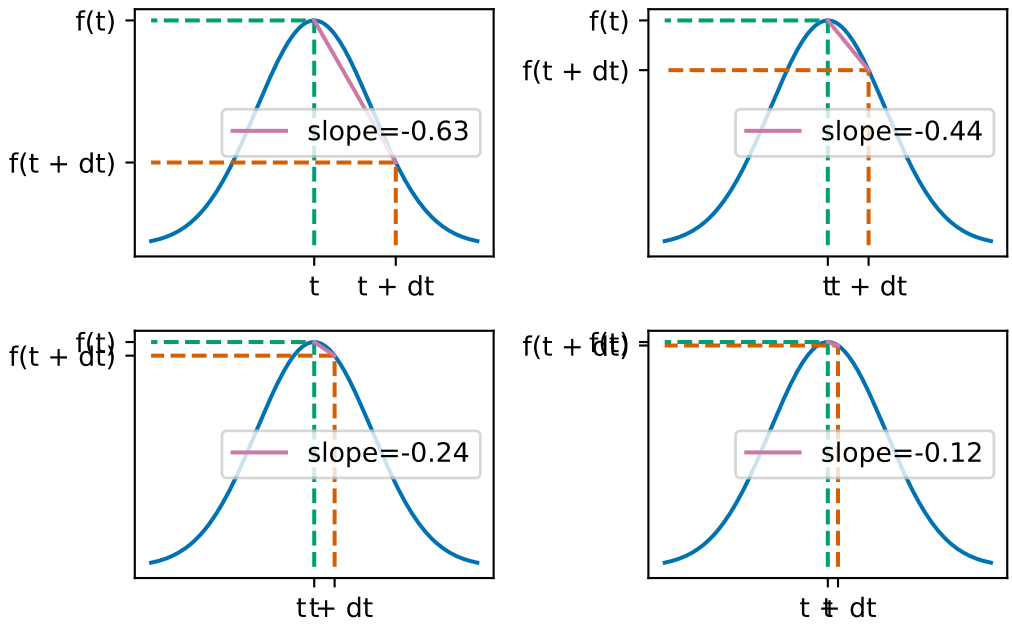


Figure 2.7: Sample straight-line approximations to the slope of a curve.

We can apply similar understanding no matter what the underlying function is. For example, we could do the same things with more complicated real world data. Here is an example using climate data (data source: Met Office).

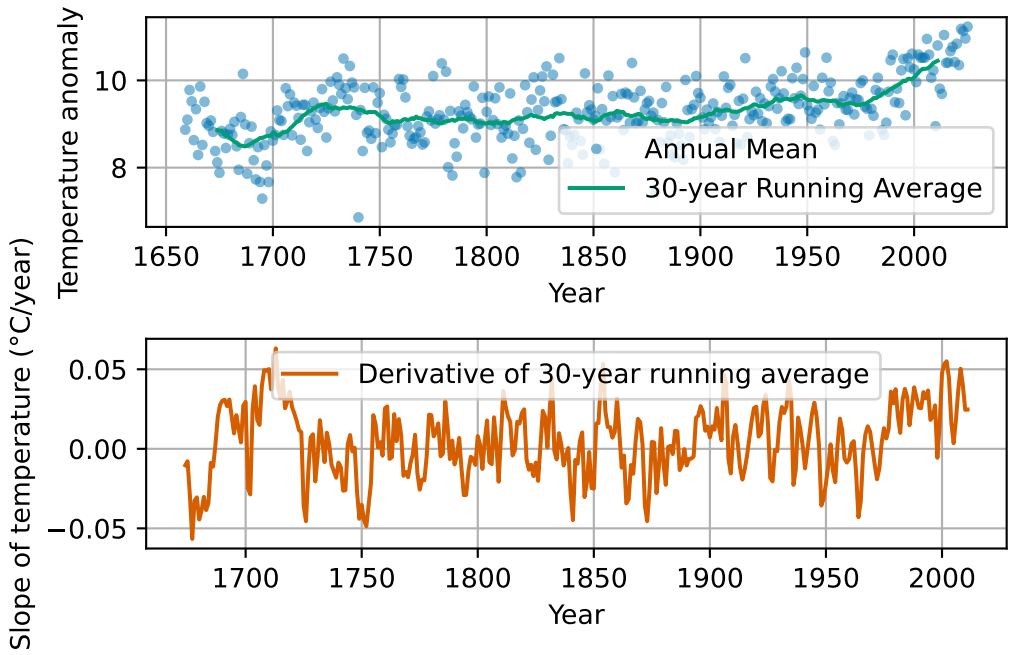


Figure 2.8: Central England Temperature anomaly. Annual Average 1659-2025.

2.3 Formulation in terms of formulae

Let's recall some of the more formal ideas that you learnt last year.

Definition 2.1. Let $f\colon (a,b) \to \mathbb{R}$ be a function on an interval (a,b) of the real line. We say that f is **differentiable** if for all $x \in (a,b)$ the limit:

$$\lim_{h\to 0} \frac{f(x+h)-f(x)}{h},$$

exists and is finite. If f is differentiable then we call the function $g\colon (a,b) \to \mathbb{R}$ the **derivative** of f defined by

$$g(x) = \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}.$$

We will write $f'(x) := g(x)$ or $\frac{\mathrm{d}}{\mathrm{d}x} f(x) = g(x)$.

Last year, you learnt some formula for derivatives. First that you can manipulate derivatives

in nice ways:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}x}(af_1(x)+bf_2(x)) &= a\frac{\mathrm{d}}{\mathrm{d}x}f_1(x)+b\frac{\mathrm{d}}{\mathrm{d}x}f_2(x) && \text{(sum rule)}\\
\frac{\mathrm{d}}{\mathrm{d}x}(f_1(x)f_2(x)) &= \left(\frac{\mathrm{d}}{\mathrm{d}x}f_1(x)\right)f_2(x)+f_1(x)\left(\frac{\mathrm{d}}{\mathrm{d}x}f_2(x)\right) && \text{(product rule)}\\
\frac{\mathrm{d}}{\mathrm{d}x}(f_1(f_2(x))) &= \frac{\mathrm{d}}{\mathrm{d}y}f_1(y)\Big|_{y=f_2(x)}\frac{\mathrm{d}}{\mathrm{d}x}f_2(x) && \text{(chain rule).}
\end{aligned}$$

You also learnt some derivatives of special functions:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}x}(x^p) &= px^{p-1}\\
\frac{\mathrm{d}}{\mathrm{d}x}\sin(x) &= \cos(x)\\
\frac{\mathrm{d}}{\mathrm{d}x}\cos(x) &= -\sin(x)\\
\frac{\mathrm{d}}{\mathrm{d}x}\exp(x) &= \exp(x).
\end{aligned}$$

Exercise 2.1. Compute the derivatives of the following functions:

1. $f_1(x) = 3x^2 - 5x + 1$
2. $f_2(x) = \sin(x^2)$
3. $f_3(x) = \exp(-x)\sin(2x)$
4. $f_4(x) = \frac{1}{\eta(x)}$ (for an arbitrary function η)
5. $f_5(x) = \tan(x)$. (*Hint:* use the fact that $\tan(x) = \sin(x)/\cos(x)$ then use the your answer for f_4 and the product and chain rules).

You also learnt about the opposite process to taking derivatives - integration - using Riemann sums:

Definition 2.2. Let $f\colon (a,b) \to \mathbb{R}$ be a function on the interval (a,b) of the real line.

Let $x_0 = a < x_1 < \ldots < x_n = b$ be a partition of the interval (a,b) into n smaller intervals (x_{i-1}, x_i) for $i = 1, \ldots, n$. We choose a point c_i in each interval (x_{i-1}, x_i) and let $h = \max_i x_i - x_{i-1}$. We define the **Riemann sum** of f by

$$s_n = \sum_{i=1}^{n} f(c_i)(x_i - x_{i-1}).$$

Then we define the **integral** of f over (a,b) is given by:

$$\int_a^b f(x)\,\mathrm{d}x = \lim_{h\to 0}\sum_{i=1}^{n} f(c_i)(x_i - x_{i-1}).$$

You learnt the following properties of integration:

$$\int_a^b c_1 f_1(x) + c_2 f_2(x)\,\mathrm{d}x = c_1 \int_a^b f_1(x)\,\mathrm{d}x + c_2 \int_a^b f_2(x)\,\mathrm{d}x$$
$$\int_a^c f(x)\,\mathrm{d}x + \int_c^b f(x)\,\mathrm{d}x = \int_a^b f(x)\,\mathrm{d}x.$$

You learnt that using the fundamental theorem of calculus you can compute integrals by *inverting* the derivative process:

$$\int_a^b x^p\,\mathrm{d}x = \left[\frac{1}{p+1}x^{p+1}\right]_{x=a}^b = \frac{1}{p+1}(b^{p+1} - a^{p+1}).$$

Exercise 2.2. Find the integrals of the following functions over $(0, 1)$.

1. $f_1(x) = 3x^2 - 5x + 1$.
2. $f_2(x) = x + \frac{1}{x^2}$.

Remark 2.1. The problem of finding distance over the time interval $(0, 1)$, given speed, $S(t)$, given in (2.1), can be solved using integration.

Indeed, using that $D'(t) = S(t)$, the fundamental theorem of calculus tells us that $D(t)$ is the integral of $S(t)$ with respect to time:

$$\begin{aligned}
D(1) &= \int_0^1 S(t)\,\mathrm{d}t = \int_0^1 -(t-1)(t+0.5)\,\mathrm{d}t \\
&= \int_0^1 -t^2 + 0.5t + 0.5\,\mathrm{d}t \\
&= \left[-\frac{1}{3}t^3 + \frac{0.5}{2}t^2 + 0.5t\right]_{t=0}^1 \\
&= (-\frac{1}{3} \times 1^3 + 0.25 \times 1^2 + 0.5 \times 1) - (-\frac{1}{3} \times 0^3 + 0.25 \times 0^2 + 0.5 \times 0) \\
&= \frac{5}{12} \approx 0.416667.
\end{aligned}$$

We compare this exact value to Table 2.1, where for 100 000 intervals, we estimated the total distance to be 0.416669.

The rest of our section work in this week's lectures is to solve similar equations in the case that our formula for $S(t)$ would depend on t and also D.

2.4 Differential equations

We have already seen and solved one differential equation - when we solved for the distance travel as a function of time: $D'(t) = S(t)$. That equation related the derivative of $D(t)$ to a function of time $S(t)$. When a model takes the form of an equation which involves one or more derivatives then it is called a **differential equation**.

Most models of dynamic processes take the form of differential equations including climate models, many models in engineering and science, financial models, physics engines in computer games and many more!

We are interested in equations of the form:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

The data (things given to us) are:

- the initial time t_0
- the initial value y_0 of y at the initial time t_0 - we call this the *initial condition*
- the right hand side function f which can be a function of both time and the solution y,

and we must find a function $y\colon [t_0, T) \to \mathbb{R}$ for some final time T.

We have seen one example above with $f(t, y) = S(t) = -(t-1)(t+0.5)$, $t_0 = 0$ and $y_0 = 0$. We will see more later.

Remark 2.2. One important consideration is that while many clever ways to solve differential equations exactly exist, most differential equations cannot be solved by hand. Thus numerical methods are not only convenient ways to find solutions of differential equations, in many cases they are essential.

The downside of a computational approach arises since we are now *approximating* the true solution. We will cover more on this in the next lecture.

Example 2.1 (Example differential equation - an object in free fall)**.** Consider the following simple model for an object falling from a large height, based on the two assumptions:

1. all objects are attracted downward with an acceleration due to gravity of $9.81\,\mathrm{m/s^2}$;
2. air resistance causes an object to decelerate in proportion to its speed (i.e. the faster it travels the greater the air resistance).

If we take downward as the positive direction, a simple model for the downward acceleration is

$$S'(t) = g - kS(t),$$

where g is the gravitational acceleration and $kS(t)$ models air resistance.

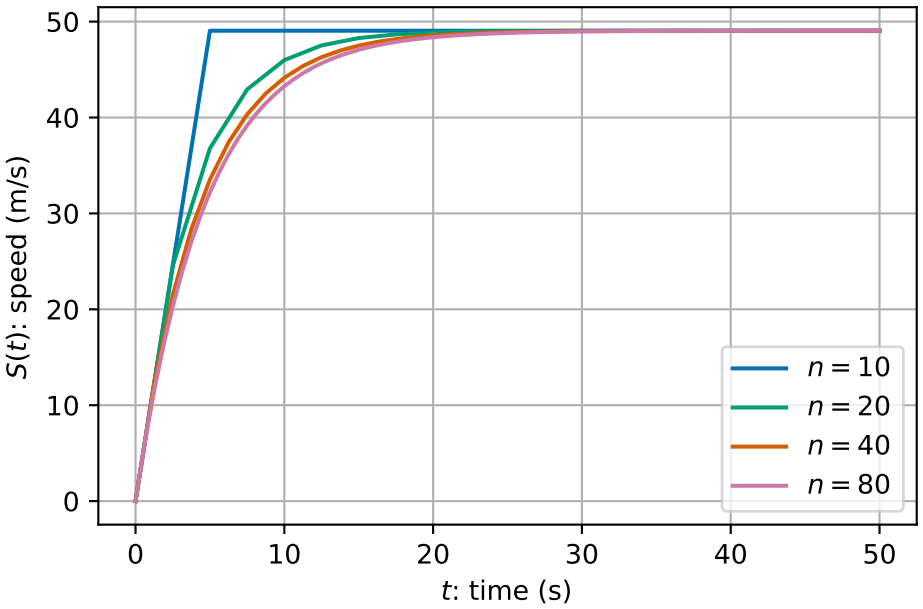
How can we solve this equation?

- This integral formula is not directly useful for computation, because $S(t)$ appears inside the integrand; to evaluate it, we would already need to know the solution S!

$$S(t) = S(0) + \int_0^T g - kS(t)\,\mathrm{d}t.$$

- There are techniques to solve this problem analytically (using so-called separation of variables) – but we are simple computer scientists looking for simple computer-based solutions instead!
- We can do similar to the above idea where we divide the time period into lots of small intervals and assume that everything is approximately constant on each time interval…

```
def freefall(n):
    """
    Compute the speed of an object falling freely.
    Input: n number of timesteps
    Output: t (time steps) and s (speed at each time step)
    """

    tfinal = 50.0  # Select the final time
    g = 9.81  # acceleration due to gravity (m/s)
    k = 0.2  # air resistance coefficient

    # initialise time and speed arrays array
    t = np.zeros(n + 1)
    s = np.zeros(n + 1)

    # set initial conditions
    s[0] = 0.0
    t[0] = 0.0

    dt = (tfinal - t[0]) / n  # calculate step size

    # take n time steps, in which it is assumed that the acceleration
    # is constant in each small time interval
    for i in range(n):
        t[i + 1] = t[i] + dt
        s[i + 1] = s[i] + dt * (g - k * s[i])

    return t, s
```



We see that as we take more time steps, we converge to a smooth solution.

2.5 Euler's method

The approach used in Example 2.1 can be applied to any differential equation in the standard format:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

For motivation, we assume we want to compute the solution at time $t_0 + h$. This can be given by the integral equation:

$$y(t_0 + h) = y(t_0) + \int_{t_0}^{t_0+h} f(t, y(t))\,\mathrm{d}t.$$

We can *approximate* the term inside the integral by it's value at t_0 (which we know since this uses the initial condition):

$$y(t_0 + h) \approx y(t_0) + \int_{t_0}^{t_0+h} f(t_0, y_0)\,\mathrm{d}t.$$

This gives us an integral, we can evaluate since the term inside the integral is constant:

$$\begin{aligned} y(t_0 + h) &\approx y(t_0) + (t_0 + h - t_0) f(t_0, y_0) \\ &= y(t_0) + h f(t_0, y_0). \end{aligned}$$

We can apply similar thinking to approximate the solution at time $t_0 + 2h$. This leads to Euler's method.

The general algorithm is very simple:

1. Set initial values $t^{(0)}$ and $y^{(0)}$ from the initial condition.
2. Loop over all time steps, until the final time T, updating using the formula:

$$\begin{aligned} y^{(i+1)} &= y^{(i)} + hf(t^{(i)}, y^{(i)}) \\ t^{(i+1)} &= t^{(i)} + h. \end{aligned}$$

The algorithm has one free parameter: the number of time steps, n. Choosing more time steps increases the cost of the algorithm helps us converge to the solution (more on this later). Once the number of time steps is fixed, we can calculate the time step $h = (T - t_0)/n$.

At each step, Euler's method uses the current slope to predict the next value.

Example 2.2. Take two steps of Euler's method to approximate the solution of

$$y'(t) = -(y(t))^2 + 3t^2 \quad \text{subject to the initial condition} \quad y(1) = 2,$$

for $1 \leq t \leq 2$.

In this example using Euler's method, we have:

- $n = 2$
- $t_0 = 1$
- $y_0 = 2$
- $T = 2$
- $h = (2 - 1)/2 = \frac{1}{2}$
- $f(t, y) = -y^2 + 3t^2$.

We start with $t^{(0)} = 1$ and $y^{(0)} = 2$.

- Step 1:

$$\begin{aligned} y^{(1)} &= y^{(0)} + hf(t^{(0)}, y^{(0)}) \\ &= 2 + \frac{1}{2} \times \left(-(2)^2 + 3 \times (1)^2 \right) \\ &= 2 + \frac{1}{2} \times -1 \\ &= \frac{3}{2} \\ t^{(1)} &= t^{(0)} + h \\ &= 1 + \frac{1}{2} = \frac{3}{2} \end{aligned}$$

- Step 2:

$$\begin{aligned} y^{(2)} &= y^{(1)} + hf(t^{(1)}, y^{(1)}) \\ &= \frac{3}{2} + \frac{1}{2} \times \left(-\left(\frac{3}{2}\right)^2 + 3 \times \left(\frac{3}{2}\right)^2 \right) \\ &= \frac{3}{2} + \frac{1}{2} \times \frac{9}{2} \\ &= \frac{15}{4} \\ t^{(2)} &= t^{(1)} + h \\ &= \frac{3}{2} + \frac{1}{2} = 2 \end{aligned}$$

Exercise 2.3. Complete the next two time steps from Example 2.2 to find an approximation of the solution at $T = 3$.

We see that Euler's method is a very general method that is very easy to apply. We can very quickly apply lots of very small time steps for (what we hope will be) a good approximation of the solution. This generality and simplicity is very powerful.

In the next lecture, we will measure the error of this approach and see how we can use our understanding of the errors to find an improve formulation.

2.6 Summary

This lecture introduces you to how we study dynamic problems, situations where things change over time, and why numerical methods are such powerful tools for computer scientists. You have explored familiar ideas like speed and distance, seeing how they connect through derivatives and integrals. From there, the lecture builds up your understanding of what a derivative really means (both as a limit and as a slope), how to compute them, and how integration reverses the process.

Once these foundations are in place, we can move into the world of differential equations, which are the backbone of models in physics, engineering, climate science, and even games. Because most real-world equations can't be solved exactly, we have seen how to approximate solutions using Euler's method: a simple, step-by-step numerical technique that computers can apply easily. By the end, we've seen how to turn a continuous, time-dependent problem into something you can compute, setting us up for the next steps on accuracy and error analysis in the next session.

3 Analysis of errors and systems of equations

Module learning objective

1. Use exact solutions to calculate errors in numerical methods for differential equations,
2. Calculate rates of convergence for differential equation solvers,
3. Solve differential equations numerically using the midpoint method,
4. Explain how to balance accuracy and efficiency when solving differential equations numerically.

3.1 Exact derivatives and exact solutions

In a small number of cases, it is possible to solve differential equations *exactly*. That is to write down a simple expression for the function which satisfies both the differential equation and its initial condition. This is rare and is the main reason why we use numerical methods. However, the special cases where we can find exact solutions are useful for *testing* our numerical methods.

Example 3.1. Consider the function $y(t) = t^3$.

We can find the derivative of $y(t)$:

$$\begin{aligned} y'(t) &= \lim_{h\to 0} \frac{y(t+h) - y(t)}{h} \\ &= \lim_{h\to 0} \frac{(t+h)^3 - t^3}{h} \\ &= \lim_{h\to 0} \frac{t^3 + 3t^2h + 3th^2 + h^3 - t^3}{h} \\ &= \lim_{h\to 0} \frac{3t^2h + 3th^2 + h^3}{h} \\ &= \lim_{h\to 0} 3t^2 + 3th + h^2 \\ &= 3t^2. \end{aligned}$$

By working backwards, we can make up our own differential equation that has $y(t)$ as a known solution. When $y(t) = t^3$, we can compute that

$$y'(t) = 3t^2 = \frac{3y(t)}{t}.$$

Hence, we know the solution to the following equation:

$$y'(t) = \frac{3y(t)}{t} \quad \text{subject to} \quad y(1) = 1. \tag{3.1}$$

If we solve this differential equation for t between 1 and 2, say, then we know that the exact answer when $t = 2$ is $y(2) == 2^3 = 8$.

Note here that we start from $t = 1$ to avoid dividing by zero $t = 0$.

We will use this differential equation where we know the exact solution to test Euler's method. Recall that for a differential equation given by:

$$y'(t) = f(t, y(t)) \quad \text{subject to the initial condition} \quad y(t_0) = y_0.$$

Euler's method is given by:

1. Set initial values $t^{(0)}$ and $y^{(0)}$ from the initial condition.
2. Loop over all time steps, until the final time T, updating using the formula:

$$y^{(i+1)} = y^{(i)} + hf(t^{(i)}, y^{(i)})$$
$$t^{(i+1)} = t^{(i)} + h.$$

Exercise 3.1. Compute two time steps of Euler's method to approximate the solution of (3.1) at time $T = 2$. What is the error between your computed solution and the exact solution?

We can also implement Euler's method in code:

```
data = {}

for n in [10, 20, 40, 80, 160]:
    # set up storage for all time steps
    h = (2 - 1) / n
    t, y = np.empty(n + 1), np.empty(n + 1)

    # perform time stepping
    t[0], y[0] = 1.0, 1.0
    for i in range(n):
```

```
        t[i + 1], y[i + 1] = t[i] + h, y[i] + h * (3 * y[i]) / t[i]

    # store solution
    data[n] = (t, y)
```

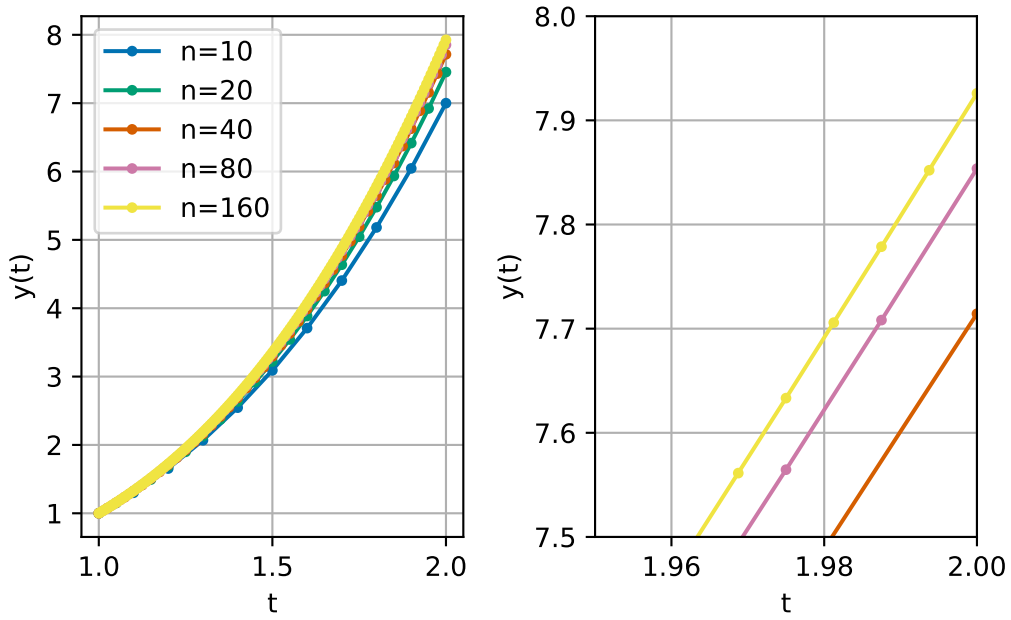


Figure 3.1: Plots of solution using Euler's method

We can see that as we increase the number of steps n (reduce the time step h), our solutions become better and better. We can also compute these values using the *absolute error*

$$\text{absolute error} = |y^{(n)} - y(2)|,$$

where $y^{(n)}$ is the solution at the final time step.

Table 3.1: A convergence study for Euler's method

n	h	Solution	Abs. error	Ratio
10	1.00000×10^{-1}	7.000000	1.00000×10^{0}	—
20	5.00000×10^{-2}	7.454545	5.45455×10^{-1}	0.545455
40	2.50000×10^{-2}	7.714286	2.85714×10^{-1}	0.523810
80	1.25000×10^{-2}	7.853659	1.46341×10^{-1}	0.512195
160	6.25000×10^{-3}	7.925926	7.40741×10^{-2}	0.506173

This table shows that indeed the absolute error is decreasing as $h \to 0$. We also compute the ratio between different errors. We see that each time h is halved, that the error is approximately halved. In other words, the error is approximately proportional to h.

Exercise 3.2. What might we expect the computed solution to be if we halved h one more time?

Something interesting happens when we plot h against the absolute error for Euler's method - we see that the errors all fall on a straight line when we use a logarithmic scale for both axes. Perhaps this is not too strange since we think that the error is approximately proportional to step size, so this line should be straight either on a linear or log scale.

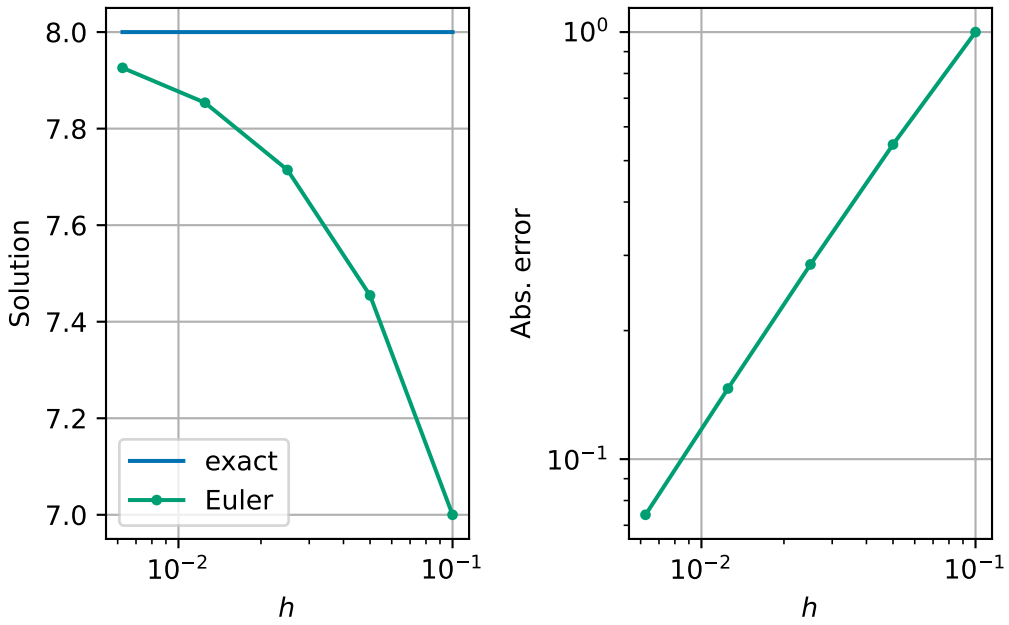


Figure 3.2: Plot of errors for Euler's method

Remark 3.1. Using log scales is really important when considering variables like h or error. Contrast the above nice plot, with nice evenly spread data points, with these linear scaled plots:

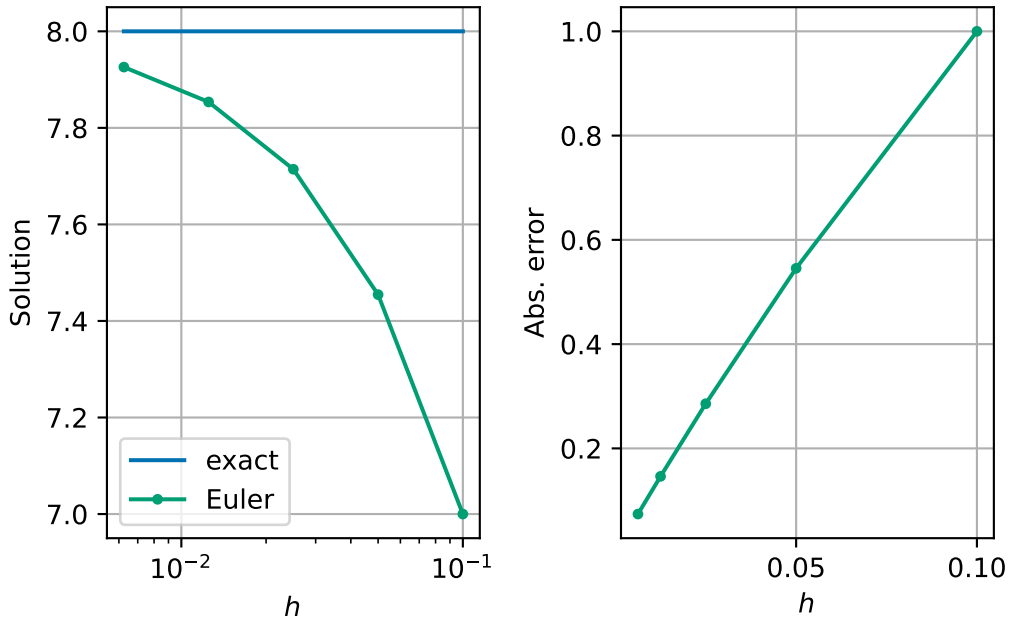


Figure 3.3: Plot of errors for Euler's method not using logarthmic scales.

Exercise 3.3. Why don't we use a log scale for solution too?

Remark 3.2. It is important to note that in this lecture, we are measuring the error at the final time point as a measure of how good our method is. This is a *global error* that measures the accumulation of *local errors* which happen at each time step. Other possible global errors include finding the maximum or average error over all time steps.

Sometimes we also care more about other, more physical based properties of our solution. For example, in long-time simulations scientists often want to ensure that the laws of thermodynamics are implemented exactly – i.e. that energy is conserved and that entropy increases.

3.2 Big O notation

When considering algorithm complexity, you have already seen big O notation. For example:

- Gaussian elimination requires $O(n^3)$ operations when n is large,
- Backward substitution requires $O(n^2)$ operations when n is large.

For large values of n the *highest* powers of n are the *most* significant. For smaller values of h, it is the *lowest* powers of h that are the most significant. When $h = 0.001$, h is much bigger than h^2, for example. We can make use of the big O notation in either case. Formally, we say

that $f(h) = O(h^p)$ as $h \to 0$ if there exists a constant $C > 0$ such that $|f(h)| < C|h|^p$ for all small enough h.

Example 3.2. Let $f(x) = 2x^2 + 4x^3 + x^5 + 2x^6$,

- then $f(x) = O(x^6)$ as $x \to \infty$,
- and $f(x) = O(x^2)$ as $x \to 0$.

In this notation, we can say that **the error in Euler's method is $O(h)$.**

3.3 The midpoint method – an improvement on Euler's method

We want to derive a new improved method for solving differential equations. We aim to produce a method with a smaller error for a similar amount of work per time step.

Let's assume that the error in Euler's method is exactly proportional to h. Under this assumption, if we halve h, we halve the error. In this derivation, we will start from the initial condition and want to approximate the solution after a time h. We will label the exact solution at $t = h$ by y_{exact}.

- Then, we continue by computing one time step from $y^{(0)}$ with time step h - we call this solution z_1. We label error between the exact solution and z_1 by E_1.
- We then take two time steps from $y^{(0)}$ now with time step $h/2$ - we call this solution z_2. We label the error between the exact solution and z_2 by E_2. Since in this derivation, the error is exactly proportional to the time step, we can see that $E_2 = \frac{E_1}{2}$.

In equations, we can express these relations as:

$$z_1 - y_{\text{exact}} = E_1$$
$$z_2 - y_{\text{exact}} = E_2 = \frac{E_1}{2}.$$

Subtracting twice the second equation from the first equation gives:

$$y_{\text{exact}} = 2z_2 - z_1.$$

If our assumption that the error in Euler's method is exactly proportional to h, combining z_1 and z_2 would give a solution with zero error! However, our assumption is in fact approximately true as we found in Table 3.1. So we might hope that we have found a method with smaller error than Euler's method with three times the cost of Euler's method (i.e. 3 Euler steps per one time step of this method - one to find z_1 and two to find z_2).

We can even formulate this method more compactly. Writing our new scheme in full, for a general time step i, we have for the one step approach:

$$z_1 = y^{(i)} + hf(t^{(i)}, y^{(i)}),$$

and for the two step approach:

$$\begin{aligned} k &= y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)}) \\ m &= t^{(i)} + \frac{h}{2} \\ z_2 &= k + \frac{h}{2} f(m, k). \end{aligned}$$

Combining z_1 and z_2 as suggested above we see:

$$\begin{aligned} y^{(i+1)} &= 2z_2 - z_1 \\ &= 2\left(k + \frac{h}{2} f(m,k)\right) - \left(y^{(i)} + hf(t^{(i)}, y^{(i)})\right) && \text{(substitute for } z_1 \text{ and } z_2\text{)} \\ &= 2k - y^{(i)} + h\left(f(m,k) - f(t^{(i)}, y^{(i)})\right) && \text{(rearrange)} \\ &= 2\left(y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)})\right) - y^{(i)} + h\left(f(m,k) - f(t^{(i)}, y^{(i)})\right) && \text{(substitute for } k\text{)} \\ &= 2y^{(i)} - y^{(i)} + h\left(f(t^{(i)}, y^{(i)}) + f(m,k) - f(t^{(i)}, y^{(i)})\right) \\ &= y^{(i)} + hf(m,k). \end{aligned}$$

The midpoint method for solving differential equations is then as follows:

1. Initialise starting values $t^{(0)}$ and $y^{(0)}$.
2. Loop over all time steps, until the final time, updating using the formulae:

$$\begin{aligned} k &= y^{(i)} + \frac{h}{2} f(t^{(i)}, y^{(i)}) \\ m &= t^{(i)} + \frac{h}{2} \\ y^{(i+1)} &= y^{(i)} + hf(m,k) \\ t^{(i+1)} &= t^{(i)} + h. \end{aligned}$$

Remark 3.3. In the end the cost of this formulation of the midpoint method is twice as large as Euler's method.

Example 3.3. Take two steps of the midpoint rule to approximate the solution of

$$y'(t) = -y(t)^2 \quad \text{subject to the initial condition} \quad y(0) = 2,$$

for $0 \leq t \leq 2$.

In this example we have:

- $n = 2$
- $t_0 = 0$
- $y_0 = 2$
- $T = 2$
- $h = (2 - 0)/2 = 1$
- $f(t, y) = -y^2$.

We start with $t^{(0)} = 0$ and $y^{(0)} = 2$.

- Step 1:

$$\begin{aligned} k &= y^{(0)} + \frac{h}{2} f(t^{(0)}, y^{(0)}) \\ &= 2 + \frac{1}{2} \times -(2)^2 \\ &= 2 + \frac{1}{2} \times -4 = 0 \\ m &= t^{(0)} + \frac{h}{2} \\ &= 0 + \frac{1}{2} = \frac{1}{2} \\ y^{(1)} &= y^{(0)} + hf(m, k) \\ &= 2 + 1 \times -(0)^2 \\ &= 2 + 1 \times 0 = 2 \\ t^{(1)} &= t^{(0)} + h \\ &= 0 + 1 = 1. \end{aligned}$$

- Step 2:

$$\begin{aligned} k &= y^{(1)} + \frac{h}{2} f(t^{(1)}, y^{(1)}) \\ &= 2 + \frac{1}{2} \times -(2)^2 \\ &= 2 + \frac{1}{2} \times -4 = 0 \\ m &= t^{(1)} + \frac{h}{2} \\ &= 1 + \frac{1}{2} = \frac{3}{2} \\ y^{(2)} &= y^{(1)} + hf(m, k) \\ &= 2 + 1 \times -(0)^2 \\ &= 2 + 1 \times 0 = 2 \\ t^{(2)} &= t^{(1)} + h \\ &= 1 + 1 = 2. \end{aligned}$$

3.3.1 Numerical results

We can also implement the method in code and explore how the error changes as we change the number of steps. For this test we use the same test as we did for Euler's method (3.1).

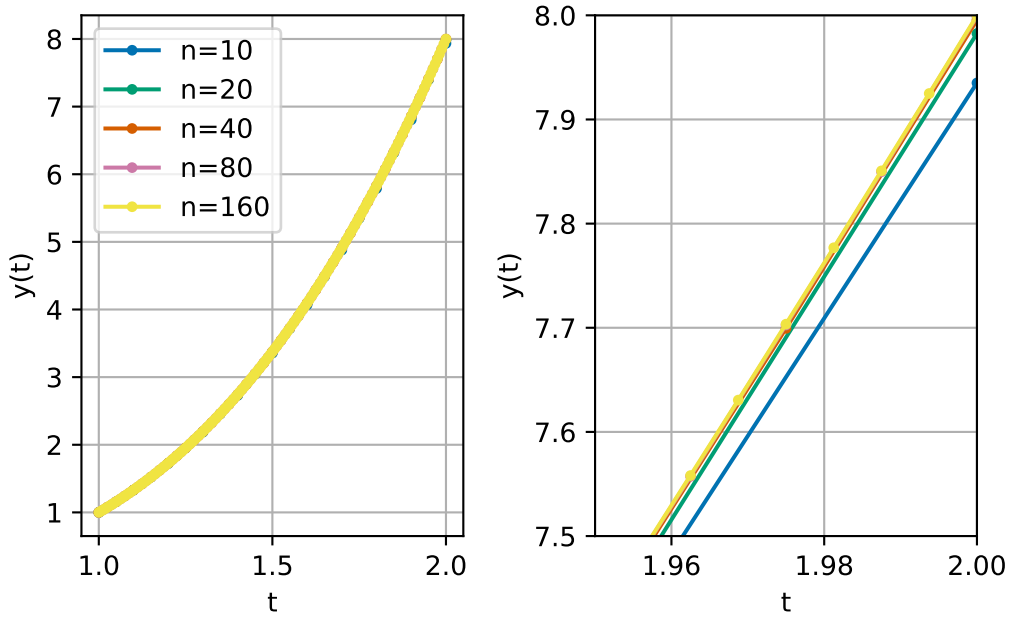


Figure 3.4: Plot of solutions using the midpoint method

We already see that we are much closer to the solution.

Let's compute the errors:

Table 3.2: A convergence study for the midpoint method

n	h	Solution	Abs. error	Ratio
10	1.00000×10^{-1}	7.935060	6.49403×10^{-2}	—
20	5.00000×10^{-2}	7.982538	1.74619×10^{-2}	0.268892
40	2.50000×10^{-2}	7.995475	4.52485×10^{-3}	0.259127
80	1.25000×10^{-2}	7.998849	1.15145×10^{-3}	0.254473
160	6.25000×10^{-3}	7.999710	2.90410×10^{-4}	0.252212

We now see that the error reduces by a *quarter* each time we reduce h by half. This means that the error is approximately proportional to h^2, or equivalently, we can say that the error is $O(h^2)$ as $h \to 0$.

This is a significant improvement on Euler's method. We say that the midpoint method is **second order**, whereas Euler's method is just **first order**.

We can see the significant reduction in error by plotting the errors for both methods:

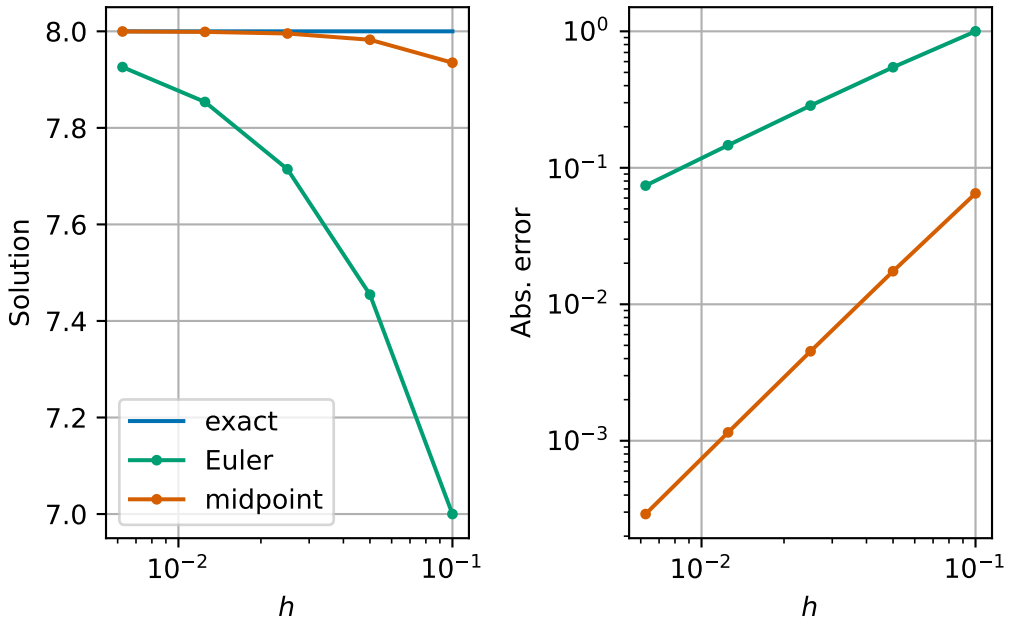


Figure 3.5: Errors for the midpoint method and Euler's method

Our interesting phenomenon of the straight line on the log-log plot of h and absolute error has happened again for the midpoint method. The difference now is that the line is much steeper. In a log-log plot, straight lines correspond to power laws (i.e., Ch^p) and the slope indicates the exponent in the power law (p). Indeed if $\text{error}(h) \approx Ch^p$ then

$$\log(\text{error}(h)) \approx \log C + p \log h.$$

We are seeing that our method behaves as if the error is proportional to h^2.

3.4 Balancing cost and accuracy

At first glance, it may seem that we now have a trade-off between reducing the error but at a higher cost by using the midpoint method. We can test this more explicitly. We repeat the experiments but use two additional measures of cost: the run time and the number of right-hand side (f) evaluations. When working with real-world systems, evaluating the right-hand side function often forms the main computational cost of both Euler's method and the midpoint method.

Table 3.3: A convergence study for Euler's method with running costs

n	h	Run time	Rhs evals	Run time ratio
10	1.00000×10^{-1}	8.23430×10^{-6}	10	—
20	5.00000×10^{-2}	1.46823×10^{-5}	20	1.783066
40	2.50000×10^{-2}	3.06469×10^{-5}	40	2.087336
80	1.25000×10^{-2}	5.70399×10^{-5}	80	1.861196
160	6.25000×10^{-3}	1.50255×10^{-4}	160	2.634209

Table 3.4: A convergence study for Midpoint's method with running costs

n	h	Run time	Rhs evals	Run time ratio
10	1.00000×10^{-1}	1.36143×10^{-5}	20	—
20	5.00000×10^{-2}	2.55374×10^{-5}	40	1.875778
40	2.50000×10^{-2}	5.32328×10^{-5}	80	2.084504
80	1.25000×10^{-2}	1.02337×10^{-4}	160	1.922443
160	6.25000×10^{-3}	2.24817×10^{-4}	320	2.196834

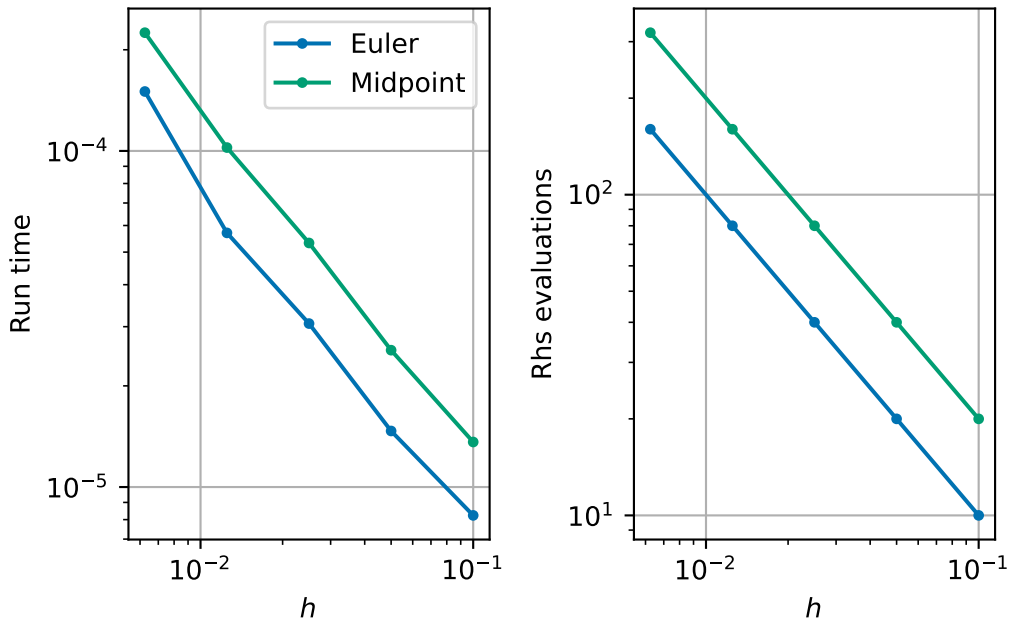


Figure 3.6: The computational cost of each method

However, we can make this plot another way by plotting the cost against the error.

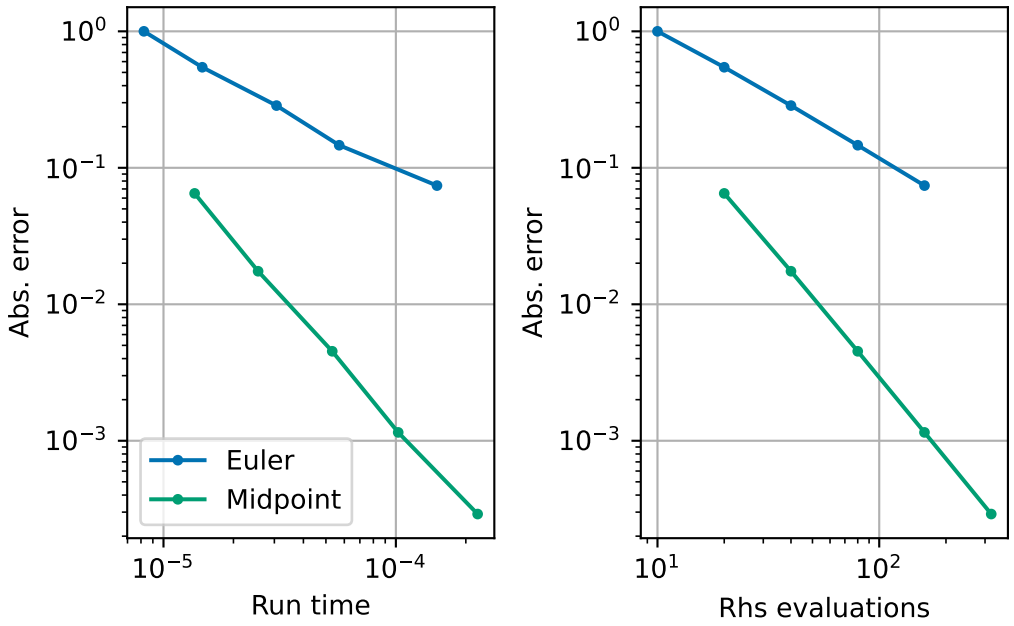


Figure 3.7: The computational cost of each method against error

We see now that whichever way you want to measure cost, you see that for a fixed cost the midpoint method gives a lower error. Although the midpoint method costs about twice as much per time step, its higher order accuracy means we need far fewer steps to reach a desired accuracy. So the best way to think is that if you have a budget of, say, right-hand side evaluations, N, you can afford for your computations, then in general, you should choose to use the midpoint method with $n = N/2$ time steps in order to minimise the error.

3.5 Summary

We have found different ways to computationally measure the quality of an algorithm to solve differential equations. We have used one way of measuring errors to derive an improvement to Euler's method called the midpoint method. We see that Euler's method is a first order accurate method and the midpoint method is second order accurate. The computational cost of the midpoint method is twice as high as Euler's method, but the higher order accuracy outweighs the higher computational cost in general.

4 A model for the trajectory of an object

Module learning objective

1. Give an example of how we can derive differential equations,
2. Apply numerical techniques to solve systems of differential equations.

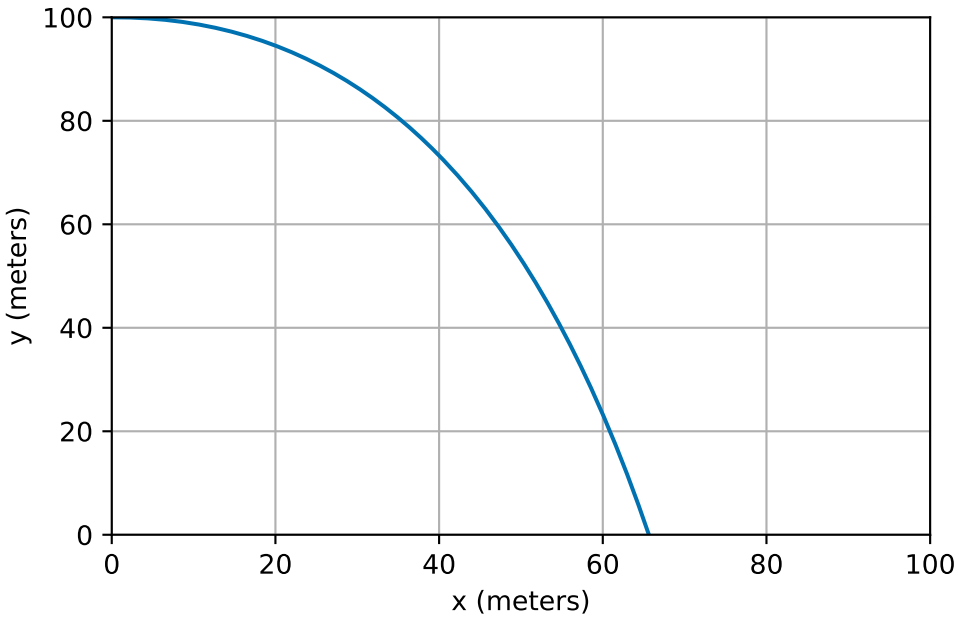
4.1 Newton's laws of motion

A large component of many computer games is the *physics engine*. Planning robotic manoeuvring requires physics-based models too. Both applications typically spend large numbers of CPU cycles simulating the motion of bodies using Newton's laws, based on the forces that are exerted upon them. In particular, Newton's second law of motion says that the net force on an object equals its mass multiplied by its acceleration:

$$\text{Force} = \text{Mass} \times \text{Acceleration}.$$

Once we know the acceleration of an object we can solve differential equations to calculate its subsequent velocity and position…

Consider an object being launched horiztonally from the top of a tall building. We know its initial position and speed in each direction and wish to predict its trajectory.



4.2 A mathematical model

In order to produce a model, we need to decide which are the most important forces acting on the object. One good choice would be to consider gravity and air resistance. Gravity only acts in the negative y direction (not the x direction), and always leads to a constant acceleration ($g = 9.81\,\mathrm{m/s^2}$). Air resistance *opposes* the motion in both x and y directions, and we will assume that the force is proportional to the speed in each direction (similarly to our free fall example, Example 2.1). We will denote the constant of proportionality by k.

We use the following notation:

- $X(t)$ the horizontal distance of the object at time t,
- $Y(t)$ the vertical distance of the object at time t (height above ground, increasing upwards),
- $U(t)$ the horizontal speed of the object at time t,
- $V(t)$ the vertical speed of the object at time t.

The above model leads to the following two differential equations (assuming the mass of the object is 1 kg):

$$U'(t) = -kU(t)$$
$$V'(t) = -kV(t) - g.$$

We also know that, from the definition of speed (rate of change of distance), that

$$X'(t) = U(t)$$
$$Y'(t) = V(t).$$

In addition to these 4 differential equations for the unknown speeds (U and V) and distances (X and Y), we also need to know the initial conditions for the system. We choose one scenario: at $t = 0$,

$$U(0) = 20\,\mathrm{m/s}$$
$$V(0) = 0\,\mathrm{m/s}$$
$$X(0) = 0\,\mathrm{m}$$
$$Y(0) = 100\,\mathrm{m}.$$

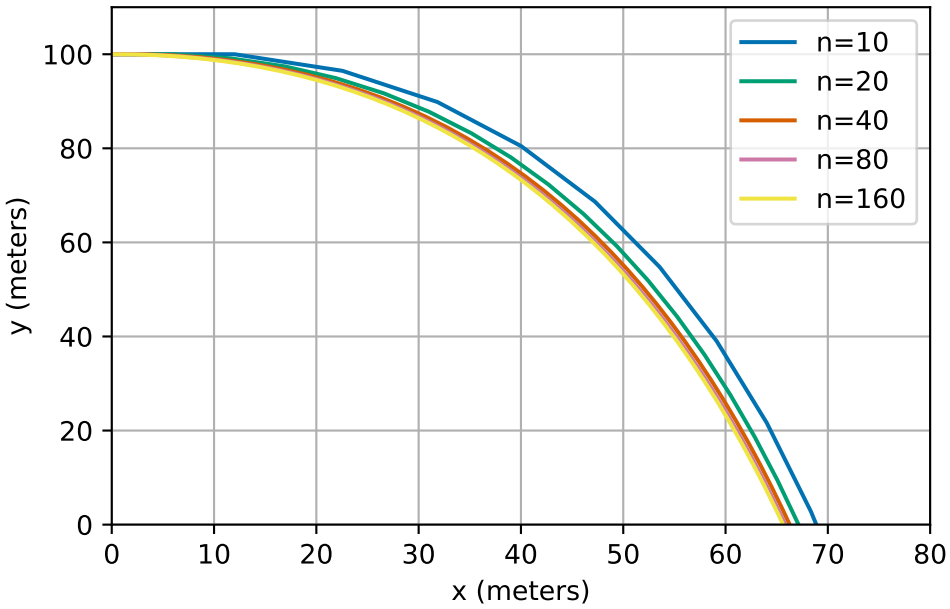
From now on we will omit the units in our calculations and assume they are implicit.

4.3 Python implementation

We can implement our model using Euler's method:

```
def projectile(n, T):
    """
    Simulate the 2D motion of a projectile using Euler's method.

    The model evolves position (X, Y) and velocity (U, V) from given initial
    conditions, under gravity and linear air resistance:
        U' = -k U
        V' = -k V - g
        X' = U
        Y' = V

    Parameters
    ----------
    n : int
        Number of Euler time-steps.
    T : float
        Final time of the simulation. The step size is `h = T / n`.

    Returns
    -------
    t : ndarray of shape (n+1,)
```

```
        Time points of the simulation.
    X : ndarray of shape (n+1,)
        Horizontal position of the projectile at each time step.
    Y : ndarray of shape (n+1,)
        Vertical position of the projectile at each time step.
    U : ndarray of shape (n+1,)
        Horizontal velocity at each time step.
    V : ndarray of shape (n+1,)
        Vertical velocity at each time step.
    """
    # initialise all arrays
    t = np.empty(n + 1)
    U = np.empty(n + 1)
    V = np.empty(n + 1)
    X = np.empty(n + 1)
    Y = np.empty(n + 1)

    # set initial conditions
    t[0] = 0.0
    U[0] = 20.0
    V[0] = 0.0
    X[0] = 0.0
    Y[0] = 100.0

    # define constants
    k = 0.2
    g = 9.81

    # calculate step size
    h = (T - 0) / n

    # perform euler steps
    for i in range(n):
        U[i + 1] = U[i] + h * (-k * U[i])
        V[i + 1] = V[i] + h * (-k * V[i] - g)
        X[i + 1] = X[i] + h * U[i]
        Y[i + 1] = Y[i] + h * V[i]
        t[i + 1] = t[i] + h

    return t, X, Y, U, V
```



4.4 System of equations

The above projectile example shows how to use Euler's method to solve a system of four differential equations. In general, a system of four differential equations will take the form:

$$\begin{aligned} y_1'(t) &= f_1(t, y_1(t), y_2(t), y_3(t), y_4(t)) \\ y_2'(t) &= f_2(t, y_1(t), y_2(t), y_3(t), y_4(t)) \\ y_3'(t) &= f_3(t, y_1(t), y_2(t), y_3(t), y_4(t)) \\ y_4'(t) &= f_4(t, y_1(t), y_2(t), y_3(t), y_4(t)). \end{aligned}$$

Exercise 4.1. How does this general form relate to the projectile example?

It is possible to apply Euler's method or the midpoint method to a general system such as that. These examples may look quite complicated at first, but they are simply applying the same techniques to systems of differential equations rather than a single differential equation.

Example 4.1. Consider the following system of differential equations:

$$\vec{y}\,'(t) = A\vec{y} \quad \text{subject to the initial condition} \quad \vec{y}(0) = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

where $\vec{y}(t) = (y_1(t), y_2(t))^T$ and $A = \begin{pmatrix} -1 & 1 \\ 1 & -2 \end{pmatrix}$. Approximate the solution at time $T = 1$ using 2 steps of Euler's method with $h = 0.5$.

- Step 1:

$$\begin{aligned}
\vec{y}^{(1)} &= \vec{y}^{(0)} + hA\vec{y}^{(0)} \\
&= \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \frac{1}{2} \times \begin{pmatrix} -1 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\
&= \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \frac{1}{2} \times \begin{pmatrix} 1 \\ -2 \end{pmatrix} \\
&= \begin{pmatrix} \frac{1}{2} \\ 0 \end{pmatrix} \\
t^{(1)} &= t^{(0)} + h \\
&= 0 + 1/2 = 1/2.
\end{aligned}$$

- Step 2:

$$\begin{aligned}
\vec{y}^{(2)} &= \vec{y}^{(1)} + hA\vec{y}^{(1)} \\
&= \begin{pmatrix} \frac{1}{2} \\ 0 \end{pmatrix} + \frac{1}{2} \times \begin{pmatrix} -1 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} \frac{1}{2} \\ 0 \end{pmatrix} \\
&= \begin{pmatrix} \frac{1}{2} \\ 0 \end{pmatrix} + \frac{1}{2} \times \begin{pmatrix} -\frac{1}{2} \\ \frac{1}{2} \end{pmatrix} \\
&= \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \end{pmatrix} \\
t^{(2)} &= t^{(1)} + h \\
&= 1/2 + 1/2 = 1.
\end{aligned}$$

4.5 Summary

We have seen that Euler's method can be applied to systems of differential equation. The same ideas as here can be used to apply the midpoint method to systems of differential equations. Systems of differential equations based on Newton's laws are extremely common in many applications including computer games and robotics.

The methods that we have learnt during this week are a small submit of the many methods available. There are a whole zoo of methods which have specialist properties such as even higher order convergence or preserve physical properties better. You will meet some on the lab sheet this week.

5 Introduction to optimisation

Module learning objective

1. Give some examples of continuous optimisation problems,
2. Give a definition of a minimum and minimiser of a function and understand some typical situations,
3. Distinguish between local and global minimisers,
4. State and apply conditions to check a functions $F\colon \mathbb{R} \to \mathbb{R}$ is convex.

So far we have studied dynamical problems, where derivatives describe change over time. We now turn to a second major use of derivatives: identifying optimal choices. Later we will see these themes reunite, when optimisation methods are interpreted dynamically through a technique called gradient descent.

Optimisation is one of the key problems of all numerical computation. It corresponds to many real world problems:

1. When making business decisions, we often want to find a solution which maximises our financial returns;
2. When doing medical procedures, we may want to maximise the effectiveness of the procedure or minimise any potential side effects;
3. When writing computer code, we (normally) want our code to run as fast as possible;
4. When scheduling a train timetable, we may want to maximise the usage of the network, or minimise the cost of running the network, or maximise the robustness to delays of our timetable.
5. When designing a social media app, we want to find the algorithm that encourages people to stay on the app the longest.
6. When controlling a robotic arm, we may want to minimise the time or energy the solution takes.
7. When doing any problem with data, we want to find the model or parameters that best fit the data.
8. When training a neural network, we want to find the parameters of the network to best match the training data.

We make a distinction between problems where the set of possible solutions (admissible states) are continuous or discrete. For example:

1. Choosing between distinct medical procedures is a discrete optimisation problem.
2. Choosing how much of a drug to give is a continuous problem that can be modelled using real numbers.

This part of the module will deal with **continuous optimisation problems**.

All of the continuous optimisation problems above can be expressed in a general mathematical form.

We first need a set $\mathcal{A}$ of admissible states (think of the amount of drug to be delivered or how it is delivered). For continuous problems $\mathcal{A}$ will also be either an interval of the real line $\mathbb{R}$ or a nice subset of the higher dimensional space $\mathbb{R}^n$. Then we can ask what the outcome of that choice is; we model the outcome through a *cost function* (also called potential or loss function), which we label $F\colon \mathcal{A} \to \mathbb{R}$.

Remark. In our examples, where we compute things, we will also have $\mathcal{A} = \mathbb{R}$ or $\mathbb{R}^n$.

The idea of optimisation is to find the best value x^* in $\mathcal{A}$ which minimises F and the value of $F^* = F(x^*)$ of the minimal cost.

Definition 5.1. We will write:

$$\begin{aligned} F^* &= \min_{x\in\mathcal{A}} F(x) \\ x^* &= \arg\min_{x\in\mathcal{A}} F(x). \end{aligned} \tag{5.1}$$

We call x^* the **minimiser** and F^* the **minimising value** or **minimal value**. We note that the argmin may not be a single value but may be a set (containing more than one point). In examples, we often denote one such minimiser by x^*.

Remark 5.1. In this section of the notes, we have included some proofs to demonstrate why many of the results are true. However, they will not be covered in the lectures and are not assessable (i.e. no questions on them will be in the exam). However, we do expect you to understand the results and statements (without proofs).

5.1 Some interesting examples

It is sometimes easy to see where the minimum of a function is by plotting the function.

In one dimension, we can simply plot the graph of the function. Let $F\colon \mathbb{R} \to \mathbb{R}$ be given by

$$F(x) = x^2 - 2x + 2.$$

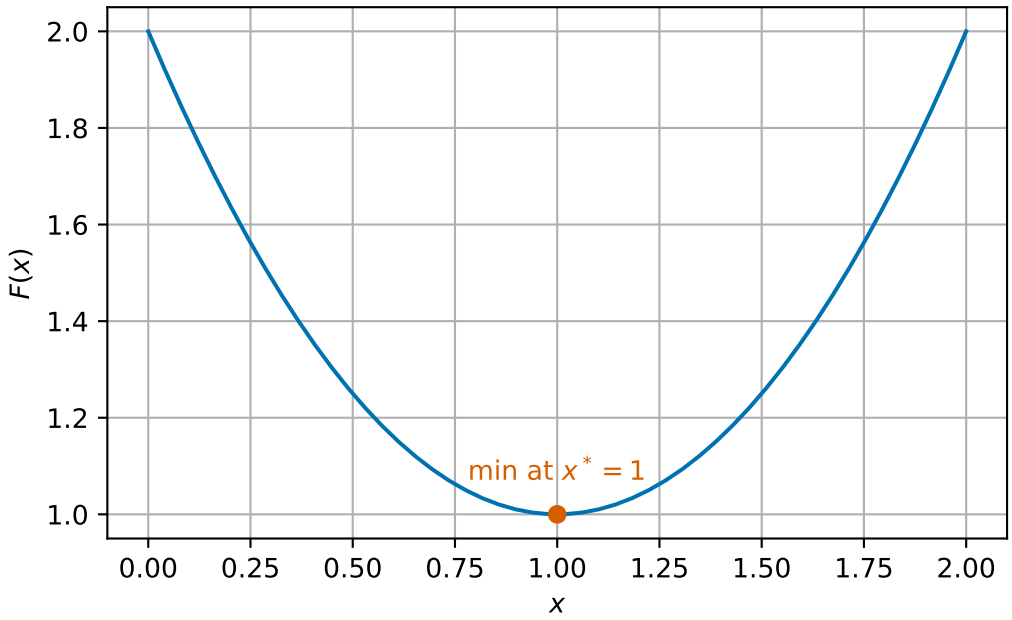


Figure 5.1: Example minimum in one dimension

We can see that the minimum is $F^* = 1$ at $x^* = 1$.

It is hard to visualise functions in higher dimension. One possible idea is to plot slices, where we fix one or more variables at a time to leave a one or two dimensional function. However, this is not often a useful way to help find minimisers. In two dimensions, we can plot a heatmap of a function. This means that for each point in a two dimensional region, we assign a colour based on the value of the function. Each colour corresponds to a different function value. Let $F\colon \mathbb{R}^2 \to \mathbb{R}$ be given by

$$F(\vec{x}) = F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2$$

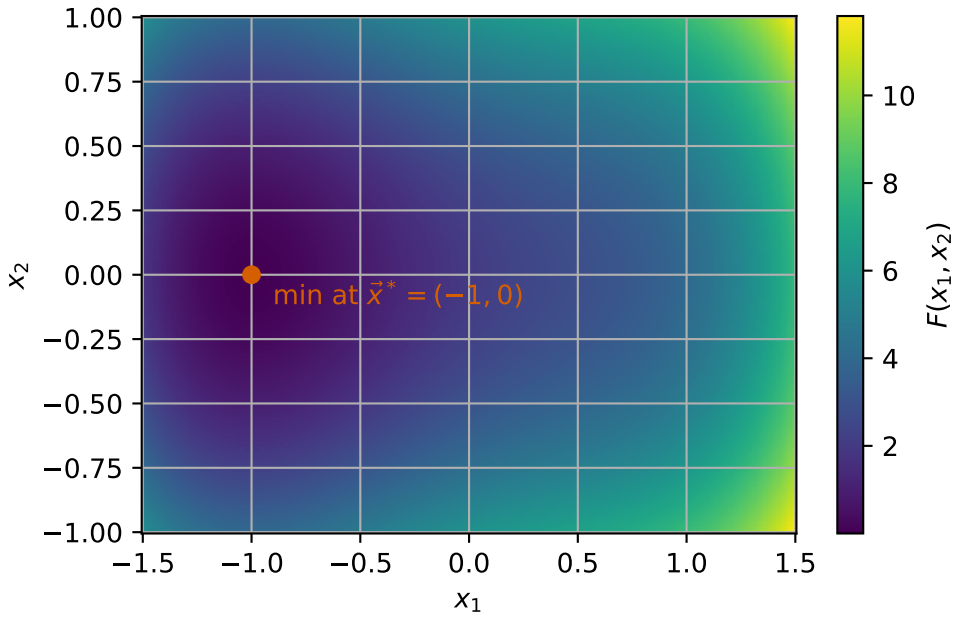


Figure 5.2: Example minimum in two dimensions in isocolour plot

Here we can see a minimum at $\vec{x}^* = (-1, 0)^T$ where $F^* = 0$.

We can also show a similar plot, but now make a three dimensional surface where the height of the surface corresponds to the value of the function.

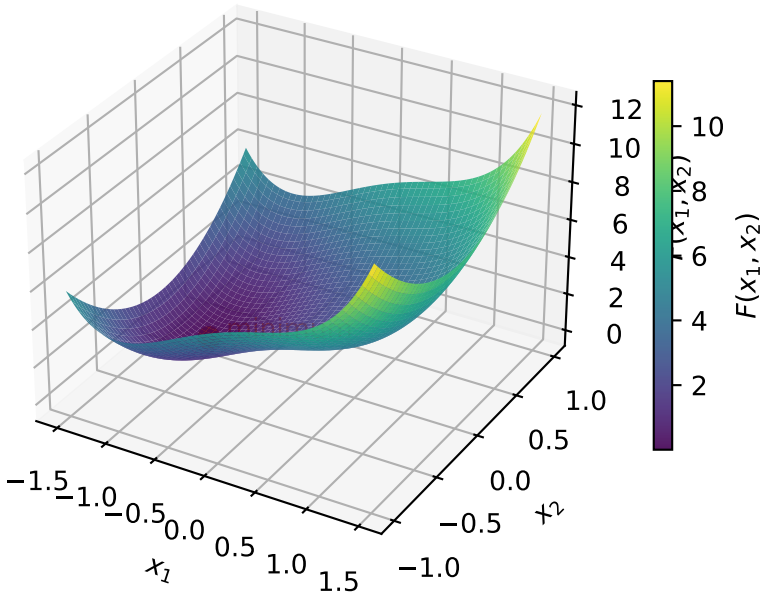


Figure 5.3: Three dimensional surface plot of function F

Before we come to some useful ways to find minimisers, we want to explore some other interesting cases.

Sometimes minimisers are not unique. We can have two values which both have the same minimal value. For

$$F(x) = (x^2 - 1)^2$$

we have two minima at $x^* = -1$ and 1 which both have $F^* = 0$.

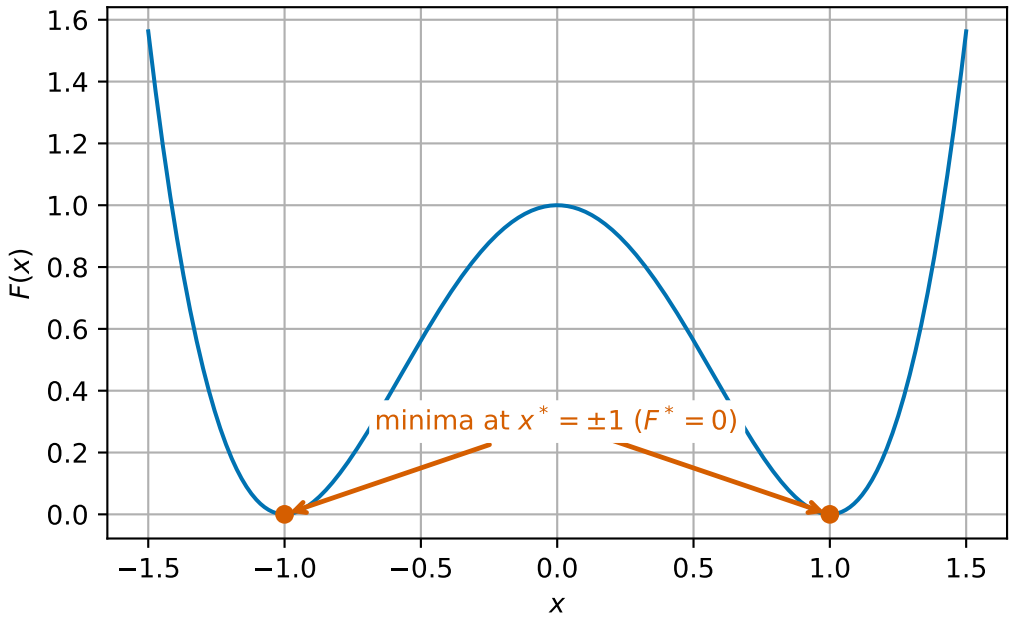


Figure 5.4: An example of a function with two distinct minima

Sometimes we can have infinitely many minimisers. Consider

$$F(x) = 1 + \sin^2(x).$$

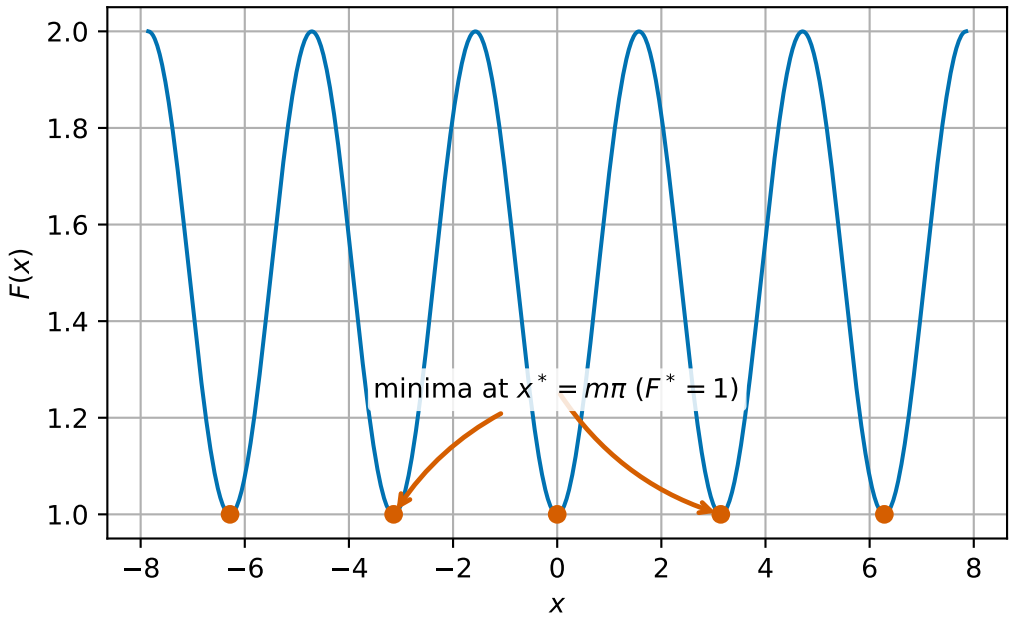


Figure 5.5: An example of a function with infinitely many distinct minima

Here, $x^* = m\pi$ for any $m \in \mathbb{Z}$ which all have $F^* = 1$.

Sometimes, we can have no minimisers at all! For

$$F(x) = -x^2,$$

there is no minimiser in $\mathbb{R}$. Indeed, for any candidate minimiser, we can always find a point where F takes a lesser value.

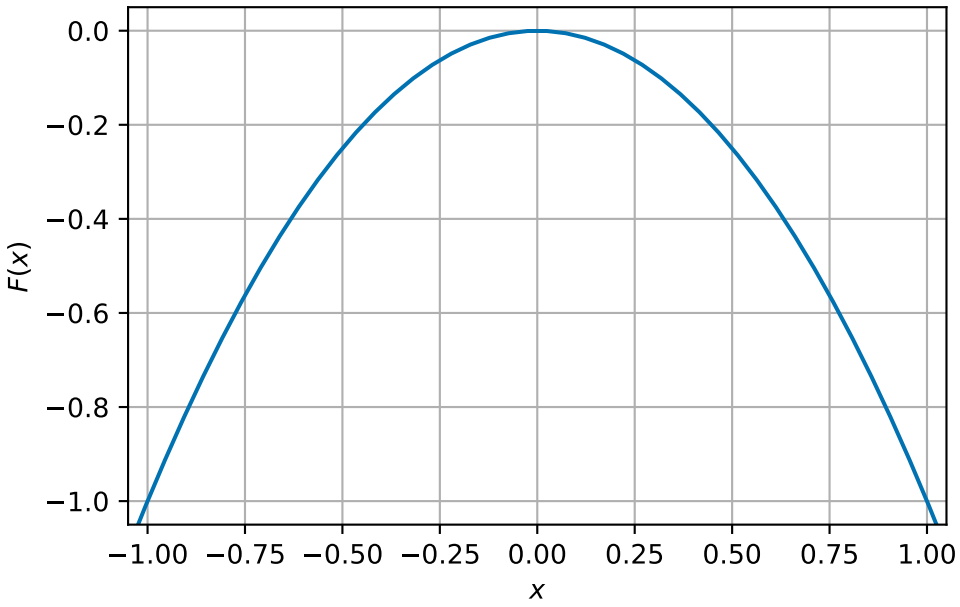


Figure 5.6: An example of a function with no minima

In two (or more) dimensions, the same things can happen. We can also have that minimisers lie on a straight line or curve: For example for $F(x_1, x_2) = (x_1 - x_2^2)^2$ the minimisers lie on the curve $x_1 = x_2^2$ with $F^* = 0$.

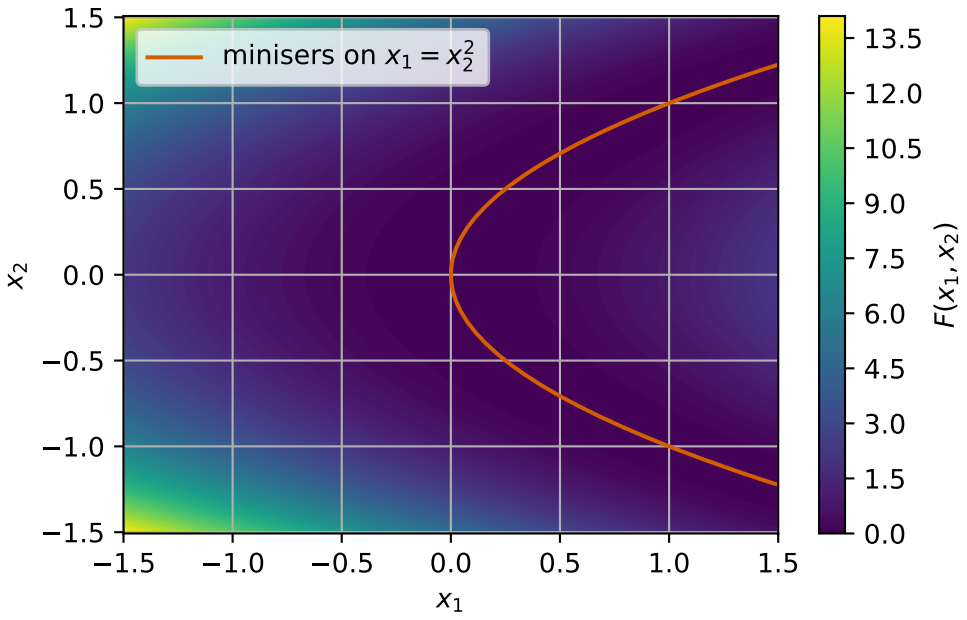


Figure 5.7: An example of a two dimensional function with minima along a curve

5.2 Global and local minimisers

Consider the function

$$F(x) = \frac{1}{4}x^4 - \frac{1}{12}x^3 - \frac{1}{2}x^2 + \frac{1}{4}x + \frac{1}{2}.$$

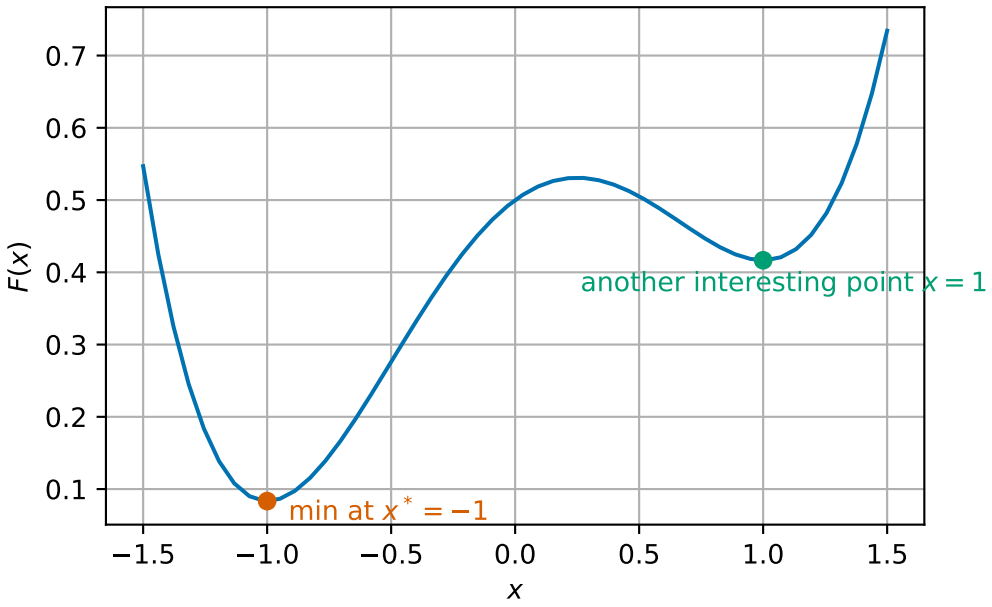


Figure 5.8: Plot of function with local and global minima

This function has a minimum at $x_* = -1$ with value $F_* = \frac{1}{12}$ (orange), but there is another interesting value at $x = 1$ (where $F(x) = 5/12$, green). The point $x = 1$ is clearly not a minimiser of the entire function, but the value at $x = 1$ is less than values nearby. This motivates the idea of a *local minimum*.

Our definitions in (5.1), are **global** (i.e. a global minimiser). By this we mean that we are looking for the value that is the least across all possible values of x. This is a very hard problem! When designing numerical algorithms, we instead often look for **local minimisers**. Later we will see a condition where we can guarantee local minimisers are global minimisers.

Definition 5.2. Scalar domain case: Given a smooth function $F\colon \mathbb{R} \to \mathbb{R}$, we say that a point x^* is a **local minimiser** of F if there exists a value b such that for all $y \in \mathbb{R}$ with $|y - x^*| < b$, $F(x^*) \leq F(y)$.

Vector domain case: Given a smooth function $F\colon \mathbb{R}^n \to \mathbb{R}$, we say that a point $\vec{x}^*$ is a **local minimiser** of F if there exists a value b such that for all $\vec{y} \in \mathbb{R}^n$ with $\|\vec{y} - \vec{x}^*\| < b$, $F(\vec{x}^*) \leq F(\vec{y})$.

5.3 How to determine a local minimiser in the scalar domain case

Looking again at the examples above, we can see that at all (local or global) minimisers the slope of F is 0 (i.e. the graph is horizontal). We can express this mathematically by saying

that the derivative of F is zero or $F'(x_*) = 0$. However, the converse is not true: we see points such as the local minimal where the slope is zero but also other points with zero derivative too!

Proposition 5.1. *Let $F : \mathbb{R} \to \mathbb{R}$ and let x^* be a local minimiser of F. Then $F'(x^*) = 0$.*

Proof. We see that the definition of local minima says that for h small enough that

$$F(x^* + h) - F(x^*) \geq 0,$$

so

$$\frac{F(x^* + h) - F(x^*)}{h} \geq 0 \qquad \text{if} \quad h \geq 0 \tag{5.2}$$

$$\frac{F(x^* + h) - F(x^*)}{h} \leq 0 \qquad \text{if} \quad h \leq 0. \tag{5.3}$$

If F is smooth, F' exists and the limit

$$F'(x^*) = \lim_{h \to 0} \frac{F(x^* + h) - F(x^*)}{h},$$

has a single value. The results of limits say that in this case (5.2) and (5.3) both hold in the limit too. This is only possible if $F'(x^*) = 0$. □

Example 5.1. Let $F(x) = (x - 3)^2 + 3 = x^2 - 6x + 12$. For the completed square form, we see that F has a minimum at $x^* = 3$.

We can compute that $F'(x)$ is given by

$$F'(x) = 2x - 6.$$

If we set $F'(x) = 0$ we can rearrange to see

$$\begin{aligned} F'(x) = 2x - 6 &= 0 \\ 2x &= 6 \\ x &= \frac{6}{2} = 3. \end{aligned}$$

Exercise 5.1. Let $F(x) = \frac{1}{4}x^4 - 2x^3 + \frac{11}{2}x^2 - 6x + 1$. Identify any potential local minima by finding the derivative of F and finding all points with $F'(x) = 0$.

We next revisit the example from the start of this section

$$F(x) = \frac{1}{4}x^4 - \frac{1}{12}x^3 - \frac{1}{2}x^2 + \frac{1}{4}x + \frac{1}{2}.$$

We can compute the derivative as:

$$F'(x) = x^3 - \frac{1}{4}x^2 - x + \frac{1}{4}.$$

Looking at the graph, we can see the F' is zero at three points, $x = -1, 1$ (orange and green) which we have already identified as local minima, but also at $x = \frac{1}{4}$ (purple).

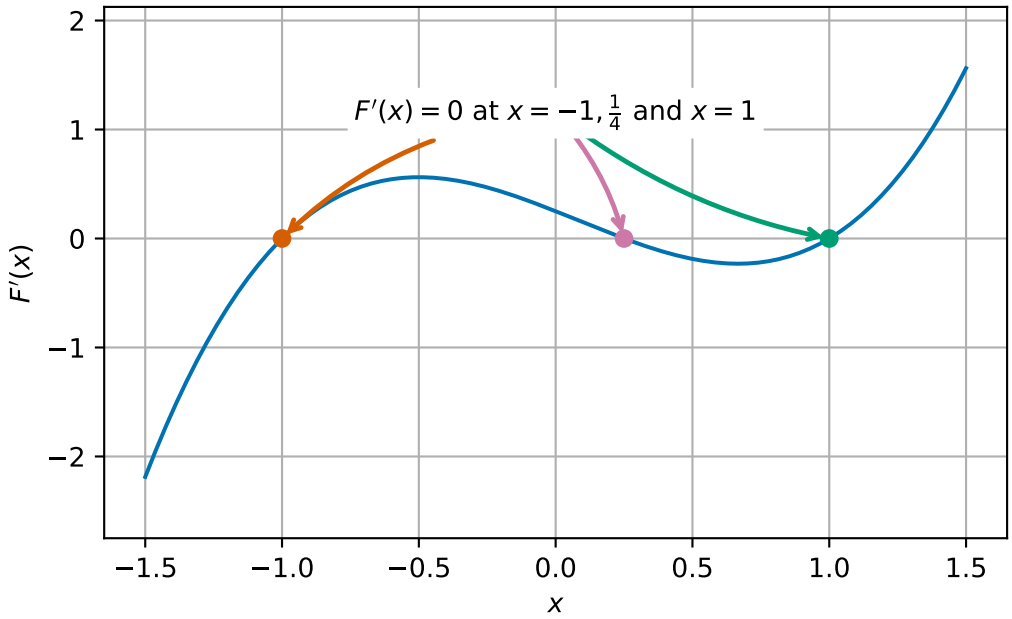


Figure 5.9: Zeros of the derivative of F.

We see that at the local minima (red and green points) the derivative is *increasing* but at the other point (orange), the derivative is decreasing. More formally, we can say that at local minima the second derivative is strictly positive. In this instance, we have

$$F''(x) = 3x^2 - \frac{1}{2}x - 1,$$

so that

$$F''(-1) = 3 \times (-1)^2 - \frac{1}{2} \times (-1) - 1 = 3 + \frac{1}{2} - 1 = \frac{5}{2} = 2.5 > 0$$
$$F''(\tfrac{1}{4}) = 3 \times (\frac{1}{4})^2 - \frac{1}{2} \times \frac{1}{4} - 1 = \frac{3}{16} - \frac{1}{8} - 1 = -\frac{15}{16} = -0.9375 < 0$$
$$F''(1) = 3 \times 1^2 - \frac{1}{2} \times 1 - 1 = 3 - \frac{1}{2} - 1 = \frac{3}{2} = 1.5 > 0.$$

Proposition 5.2. *Let $F\colon \mathbb{R} \to \mathbb{R}$ and $x^* \in \mathbb{R}$ such that $F'(x^*) = 0$ and $F''(x^*) \geq 0$. Then x^* is a local minimiser of F.*

Proof. We only sketch the proof here. The key idea is to use something called Taylor's theorem, which gives us the formula that for x close to x^*, we have that

$$F(x) \approx F(x^*) + (x - x^*)F'(x^*) + \frac{1}{2}(x - x^*)^2 F''(x^*).$$

Applying that $F'(x^*) = 0$ and $F''(x^*) \geq 0$, we have

$$F(x) \approx F(x^*) + \frac{1}{2}(x - x^*)^2 F''(x^*) \geq F(x^*).$$

This proof can be made rigorous with the help of further results from mathematical analysis. □

Next, let us compare two examples:

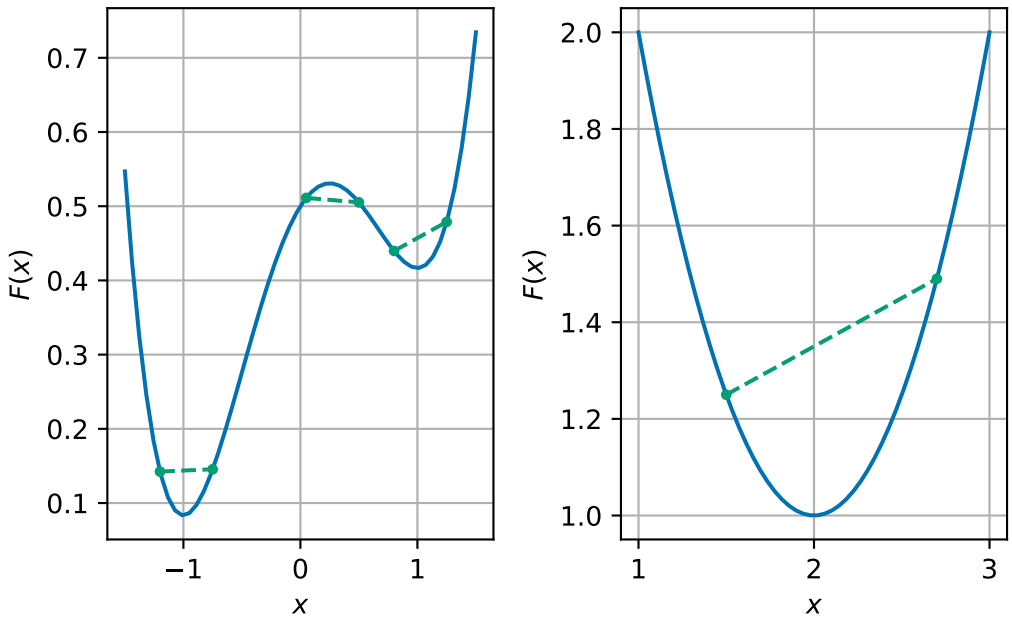


Figure 5.10: Examples of functions and chords between points

For each example, we have chosen different pairs of points and drawn a straight line between them (green dashed line). We see, in the left hand side example, that sometimes the straight line is above the function (blue curve) and sometimes its below, whereas in the right hand side example, the straight line is above the function, and always will be for any pair of points. This idea is expressed mathematically through the idea of *convexity*.

Definition 5.3. A function $F\colon \mathbb{R} \to \mathbb{R}$ is **locally convex in an interval** $B \subset \mathbb{R}$ if for all points $x, y \in B$:

$$aF(x) + (1-a)F(y) \geq F(ax + (1-a)y) \quad \text{for all} \quad a \in [0,1]. \tag{5.4}$$

We say that F is **convex** if it is convex on $\mathbb{R}$ (i.e., on every interval $B \subset \mathbb{R}$).

It is hard to show convexity with this definition directly. Instead, we normally use the follow equivalent formulations:

Lemma 5.1. *Let $F\colon \mathbb{R} \to \mathbb{R}$ be a smooth function. Then the following are equivalent:*

1. *F is locally convex on an interval B*
2. *The derivative of F satisfies*

$$F(x) \geq F(y) + F'(y)(x-y) \quad \textit{for all} \quad x, y \in B. \tag{5.5}$$

3. *The second derivative of F is positive in B:*

$$F''(x) \geq 0 \quad \textit{for all} \quad x \in B. \tag{5.6}$$

Remark 5.2. Condition 2 can be expressed informally as saying that the graph of F lies above its tangent.

Condition 3 links directly to the idea of linking local and global minima through Proposition 5.2.

Proof. We prove (1) is equivalent to (2) and (2) is equivalent to (3).

(1) $\Rightarrow$ (2).

Assume F is (locally) convex on B. Fix $x, y \in B$ and $a \in (0,1]$. By convexity,

$$aF(x) + (1-a)F(y) \;\geq\; F(ay + (1-a)x) = F(y + a(x-y)).$$

Rearranging gives

$$F(x) \;\geq\; \frac{F(y + a(x-y)) - F(y)}{a} + F(y) \;=\; (x-y)\,\frac{F(y+h) - F(y)}{h} + F(y),$$

where we set $h := a(x-y)$. As $a \downarrow 0$ we have $h \to 0$ (with the same sign as $x-y$). Since F is smooth, the difference quotient tends to $F'(y)$ regardless of the sign of h. Thus,

$$F(x) \;\geq\; F(y) + F'(y)(x-y),$$

which is (5.5).

(2) $\Rightarrow$ (1).

We start with $x, y \in B$ and let $z = ax + (1-a)y$, then we have

$$F(x) \geq F(z) + F'(z)(x-z) \quad \text{and} \quad F(y) \geq F(z) + F'(z)(y-z).$$

Multiplying the first inequality by a and the second by $1-a$ and adding gives

$$aF(x) + (1-a)F(y) \geq F(z) + F'(z)(a(x-z) + (1-a)(y-z)).$$

But from the definition of z, we have

$$a(x-z) + (1-a)(y-z) = ax + (1-a)y - z = 0.$$

Hence, we can infer that

$$aF(x) + (1-a)F(y) \geq F(z) = F(ax + (1-a)y).$$

(2) $\Rightarrow$ (3).

We can apply (5.5) to see that for any $x, y \in B$, we have

$$F(x) \geq F(y) + F'(y)(x-y)$$
$$F(y) \geq F(x) + F'(x)(y-x).$$

Adding these inequality and rearranging gives

$$(F'(y) - F'(x))(y-x) \geq 0.$$

This inequality says that F' is nondecreasing on B. If we let $y = x + h$ for $h > 0$, we have

$$\frac{F'(x+h) - F'(x)}{h} \geq 0.$$

Taking the limit $h \to 0$ shows that $F''(x) \geq 0$ (5.6).

(3) $\Rightarrow$ (2).

This result is a little hard and relies on a result from mathematical analysis (the Mean Value Theorem) which is not covered by this course. The key idea is to show that $F''(x) \geq 0$ (5.6) implies that F' is nondecreasing and then if F' is nondecreasing that (5.5) holds. □

Example 5.2. We see that Condition 3 in Lemma 5.1 is often the easiest to show.

1. Take for example $F(x) = x^2 - 4x + 5$. Then

   $$F'(x) = 2x - 4 \quad \text{and} \quad F''(x) = 2 > 0.$$

   Hence we can see that F is convex.

   This example, is somewhat of a prototypical example. It is often helpful when thinking of convex functions to think of something like a quadratic function with positive leading coefficient.

2. (hard) Take $F(x) = \frac{1}{4}x^4 - 3x^3 + 15x^2 - 5x + 4$

$$F'(x) = x^3 - 9x^2 + 30x$$
$$F''(x) = 3x^2 - 18x + 30.$$

We can complete the square to see that

$$F''(x) = 3(x-3)^2 + 3 \geq 3 > 0.$$

We can see that F'' is in fact strictly positive. Hence, we can infer that F is globally convex.

Exercise 5.2. Show that the following functions are or are not convex on $\mathbb{R}$:

1. $F(x) = x^2 - 6x + 12$
2. $F(x) = \frac{1}{6}x^4 - \frac{8}{3}x^3 + 15x^2 + 6x + 1.$

Proposition 5.3. *Let $F\colon \mathbb{R} \to \mathbb{R}$ and x^* be a point such that $F'(x^*) = 0$. If F is locally convex in a region B with $x^* \in B$ then x^* is a local minimiser of F.*

Proof. Let $y \in B$, then from Lemma 5.1, we have

$$F(y) \geq F(x^*) + F'(x^*)(y - x^*).$$

Since $F'(x^*) = 0$, we see that

$$F(y) \geq F(x^*) + F'(x^*)(y - x^*) = F(x^*) + 0(y - x^*) = F(x^*).$$

Hence we can see that x^* is a local minimiser in the interval B. □

Theorem 5.1. *Let $F\colon \mathbb{R} \to \mathbb{R}$ be convex. Then any local minimiser is a global minimiser.*

Proof. Let x^* be a local minimiser of F, then $F'(x^*) = 0$ (from Proposition 5.1). Applying Lemma 5.1, we see that for any y, we have

$$F(y) \geq F(x^*) + F'(x^*)(y - x^*) = F(x^*).$$

Hence x^* is in fact a global minimiser. □

Example 5.3. Take the same example as before $F(x) = x^2 - 4x + 5$. Then

$$F'(x) = 2x - 4.$$

We can easily see that $F'(2) = 0$ hence $x^* = 2$ is a local and indeed a global minimiser since F is convex.

We can also see this by completing the square:

$$F(x) = x^2 - 4x + 5 = (x-2)^2 + 1.$$

We can read off that F has a minimum at the same point that $(x-2)^2 = 0$, i.e. that $x^* = 2$.

Remark 5.3. Minimisers of convex functions are not necessarily unique. Consider $F\colon \mathbb{R} \to \mathbb{R}$ given by

$$F(x) = \begin{cases} (x+1)^4 & \text{for} \quad x \leq -1 \\ 0 & \text{for} \quad -1 < x < 1 \\ (x-1)^4 & \text{for} \quad x \geq 1. \end{cases}$$

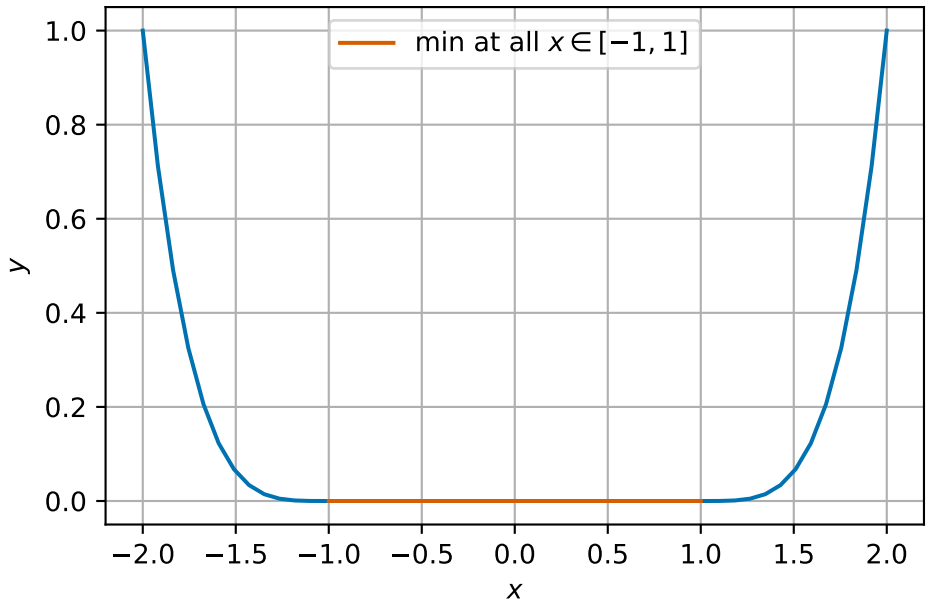


Figure 5.11: Example of a convex function with non-unique minima

F is a convex, smooth function, but all points $x \in [-1, 1]$ are minimisers.

5.4 Summary

Optimisation is a wide area of numerical computation with many different application areas. In this module, we will only consider continuous optimisation problems. We have provided several examples of how different types of minima can occur in one and two dimensions using plots to identify minima. In one dimension, we have provided a characterisation of local minima and how to link them to global minima using the idea convexity.

6 Root finding methods for one dimensional optimisation

Module learning objective

1. Explain why numerical methods are required for finding minima/solving nonlinear equations in one dimension,
2. Explain the idea of an iterative method,
3. Apply the Bisection Method and understand its properties,
4. Apply Newton's method and understand its properties.

In one dimension, we will proceed with a simple idea: to find candidate minima of functions $F : \mathbb{R} \to \mathbb{R}$, we will find points at which the derivative of F is 0. To help us with notation, in this section we will write $f := F'$. The techniques developed in this section related to both optimisation of one dimensional optimisation problems, but also solving general scalar nonlinear equations.

> Given a smooth function $f(x)$, the problem is to find a point x^* such that $f(x^*) = 0$. That is, x^* is a solution of the equation $f(x) = 0$ and is called a **root of** $f(x)$.

Remark 6.1. In one dimension, a local minimum occurs at point where $F'(x) = 0$, but not every root of F' is a minimum. The focus on this lecture is finding possible minima.

Example 6.1.

1. The linear equation $f(x) = ax + b = 0$ has a single solution at $x^* = \frac{-b}{a}$.
2. The quadratic equation $f(x) = ax^2 + bx + c = 0$ is nonlinear, but simple enough to have a known formula for the solutions:
$$x^* = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$
3. A general nonlinear equation $f(x) = 0$ rarely has a formula like those above which can be used to calculate its roots.

It is also sometimes better to use numerical methods to solve an equation even if an exact formula exists:

- the exact formula may have difficulties due to rounding errors;
- the exact formula may be more expensive to compute.

Example 6.2. We want to solve the quadratic equation

$$f(x) = x^2 - (m + \frac{1}{m})x + 1 = 0,$$

for large values of m. The exact solutions are m and $\frac{1}{m}$.

```
# implementation of quadratic formula
def quad(a, b, c):
    """
    Return the roots of the quadratic polynomial a x^2 + b x + c = 0.
    """
    d = np.sqrt(b * b - 4 * a * c)
    return (-b + d) / (2 * a), (-b - d) / (2 * a)


# test for m = 1e8
m = 1e8
roots = quad(1, -(m + 1 / m), 1)

print(f"roots {roots[0]:.4e} and {roots[1]:.4e}")
```

```
roots 1.0000e+08 and 1.4901e-08
```

These values look reasonable, but let's check the errors:

```
# absolute and relative errors
exact_roots = (m, 1 / m)

abs_error = (abs(roots[0] - exact_roots[0]), abs(roots[1] - exact_roots[1]))
rel_error = (abs_error[0] / m, abs_error[1] / (1 / m))

print(f"abs errors {abs_error[0]:.4e} {abs_error[1]:.4e}")
print(f"rel errors {rel_error[0]:.4e} {rel_error[1]:.4e}")
```

```
abs errors 0.0000e+00 4.9012e-09
rel errors 0.0000e+00 4.9012e-01
```

The absolute errors are quite small, but the relative error for the second root is huge - almost 50%!

6.1 Example problems

We will test our methods on the following problems.

Example 6.3 (Square root example)**.** We want to find the value of the square root of 2. We can solve this by finding the (positive) root of

$$f(x) = x^2 - 2.$$

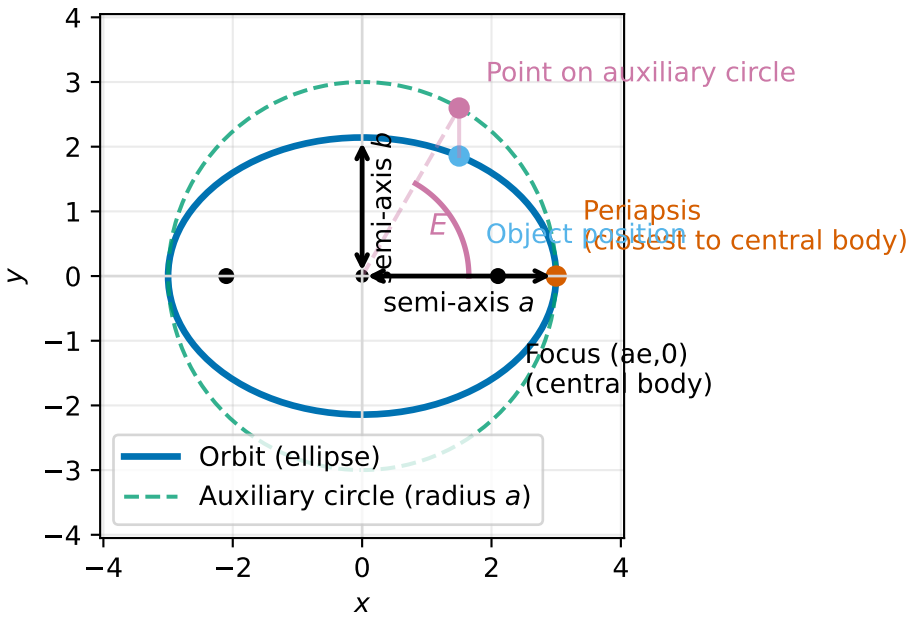
This problem is interesting in that we know there are two solutions, but we only want the positive one. Our algorithms should take that into account.

Example 6.4 (Kepler's equation, elliptic orbits)**.** When an object moves under gravity around a planet or star, its path is often well-approximated by an ellipse. To work out where the object is at a given time, we use a standard relationship called Kepler's equation. The key point here is: it reduces to finding a root of a function $f(E) = 0$.

An ellipse has two semi-axes a and b and we assume that $a > b$. We denote by $e = \sqrt{1 - \frac{b^2}{a^2}}$ the eccentricity of the ellipse. The closest point on the ellipse to the central body (i.e. planet or star, which is at one of the foci of the ellipse) of the object's trajectory is called the periapsis.

To connect time to position, it's convenient to use an angle-like variable that increases at a constant rate - this variable is called the mean anomaly, M (measured in radians). $M = 0$ corresponds to periapsis and $M = \pi$ corresponds to half an orbit later.

There is a helpful intermediate angle called the eccentric anomaly which we write as E. Since M is not the actual geometric angle around the ellipse, we use E as a useful angle from which we can compute quantities such that the coordinates of the object.



For an elliptic orbit with eccentricity $0 \leq e < 1$, the mean anomaly M is related to the eccentric anomaly E by Kepler's equation:

$$E - e\sin(E) = M.$$

The problem is: Given e (eccentricity) and M (mean anomaly at a given time, in radians), find E^* (eccentric anomaly). We can write this as a root finding problem with

$$f(E) = E - e\sin(E) - M.$$

We should be careful to find a solution only in $[0, 2\pi)$. In our tests we will use $e = 0.7, M = 2.0$. Note that

$$f'(E) = 1 - e\cos E \geq 1 - e > 0,$$

so f is strictly increasing and we know we only have one solution.

6.2 Iterative methods

All our methods will use the idea of **iteration** (or **iterative improvement**). We saw this idea when we were using Jacobi and Gauss-Seidel iteration to solve systems of linear equations.

- Given an initial estimate of the root $x^{(0)}$, try to generate a better approximate $x^{(1)}$ of the root.
- Once $x^{(1)}$ has been computed, it can be used to compute another estimate $x^{(2)}$ in the same way (and so on…).

- Generally, $x^{(i)}$ is used to compute $x^{(i+1)}$ (although other previous estimates may be used as well).
- There are many methods for doing this!

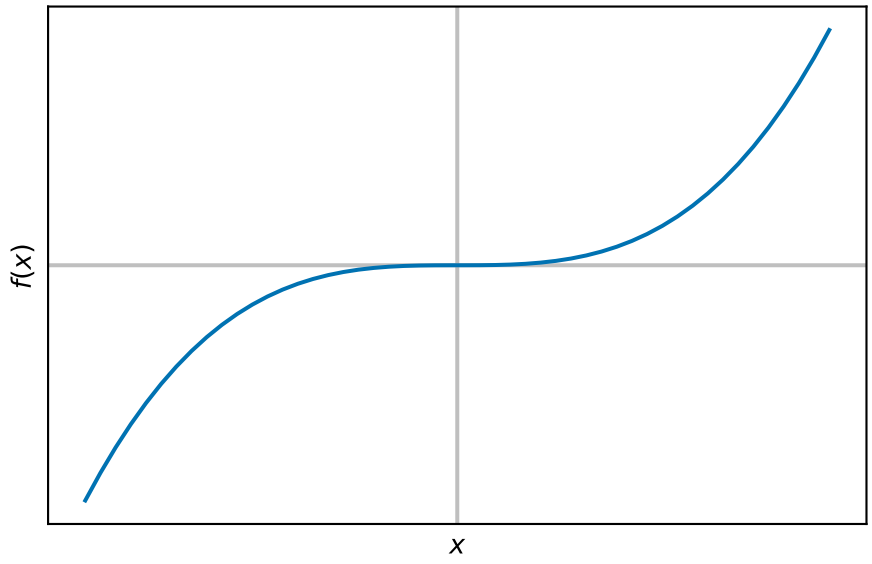
There are a few issues we should consider with iterative methods:

1. Does the iteration converge?
2. In other words, will $x^{(i+1)}$ be a better estimate than $x^{(i)}$? In what sense – e.g. $|f(x^{(i)})|$ smaller or $|x^{(i)} - x^*|$ smaller?
3. Is there a way to check this as part of the computation? Can we prove this?
4. If the iteration does converge, then how quickly does this happen?
5. How should we decide when to stop the iteration?

Remark 6.2. We briefly comment that iteration is *different* to the idea of time stepping we met when solving differential equations. The number of time steps is a given parameter when solving a differential equation but we (normally) do not know how many iterations of a root finder are needed to find a solution (for a particular stopping criterion).

6.3 Bisection method

The first method that we introduce is the bisection method. This is a simple, slow but very robust method. It's based on a simple idea: if a function is smooth (continuous) on an interval and changes sign between the two end points (either positive to negative or negative to positive), then it must be zero some point in between. (There is a mathematical analysis theorem called the Intermediate Value Theorem that gives us this result).



6.3.1 The algorithm

The algorithm steps are:

1. Suppose we *know* there is a root x^* (i.e with $f(x^*) = 0$) between two end points a and b (we sometimes call $[a, b]$ the *bracket*).
2. Call $c = \frac{a+b}{2}$, then the solution is either between a and c or between c and b.
3. If the solution is between a and c:
    - then go to step 1 with the end points as a and c,
    - otherwise, go to step 1 with the end points c and b.

We repeat this process until $b - a < TOL$ where TOL is a user-supplied value.

We can check if the root is between a and b by checking if the signs of $f(a)$ and $f(b)$ are different: i.e. $f(a)f(b) < 0$.

6.3.2 Convergence analysis

Assume that k steps have been taken from an initial bracket (a, b). We notice that at each step, the bracket size is halved. This means after k steps, the length of the interval will be $\frac{b-a}{2^k}$.

For large initial bracket size (i.e. large $b-a$) or small TOL (high accuracy requirement) k will be large.

6.3.3 Examples

```
def bisection(f, a, b, tol):
    """
    Find a root of a scalar function using the bisection method.

    The bisection method is a bracketing root-finder. Given an interval
    ``[a, b]`` such that ``f(a)`` and ``f(b)`` have opposite signs, the method
    repeatedly halves the interval and selects the subinterval that still
    brackets a root. The returned estimate is the midpoint of the final
    interval once its width is less than ``tol``.

    Parameters
    ----------
    f : callable
        Scalar function of one variable, ``f(x)``, whose root is sought.
```

```
        Must be defined and (ideally) continuous on the interval ``[a, b]``.
    a : float
        Left endpoint of the initial bracket.
    b : float
        Right endpoint of the initial bracket. Must satisfy ``b > a``.
    tol : float
        Absolute tolerance on the interval width. Iteration stops when
        ``b - a <= tol``.

    Returns
    -------
    data: list[dict]
        Iteration data for analysing results

    Raises
    ------
    AssertionError
        If the initial interval does not bracket a root, i.e. if
        ``f(a) * f(b) >= 0``. Note that with the current implementation this
        also rejects cases where ``f(a) == 0`` or ``f(b) == 0`` (an endpoint is
        exactly a root).
    """
    # initial bracket check
    assert f(a) * f(b) < 0.0, f"({a}, {b}) does not bracket the root"

    data = []

    # the iteration
    it = 0
    while b - a > tol:
        # find midpoint
        c = (a + b) / 2
        # update bracket
        if f(a) * f(c) > 0.0:
            a = c
        else:
            b = c

        it += 1

        data.append({"it": it, "a": a, "b": b, "f(a)": f(a), "f(b)": f(b)})

```

```
57     return data
```

$$f(x) = x^2 - 2.$$

```
'|   it |       a |       b |         f(a) |        f(b) |\n|-----:|--------:|--------:|-----
```

We see that it takes 14 iterations to find the solution to a tolerance of 10^{-4} for the square root problem.

$$f(x) = x - 0.7\sin(x) - 2.0$$

```
'|   it |       a |       b |         f(a) |        f(b) |\n|-----:|--------:|--------:|-----
```

We see that it takes 16 iterations to find the solution to a tolerance of 10^{-4} for the Kepler problem

Exercise 6.1. For the square root problem, for which of the following brackets do you expect the algorithm to find a solution? For cases where you expect the algorithm to find a solution, how many iterations do you expect for a tolerance of 10^{-4}?

1. $(1.0, 1.5)$
2. $(0.0, 4.0)$
3. $(-4.0, 4.0)$
4. $(-2.0, 0.0)$

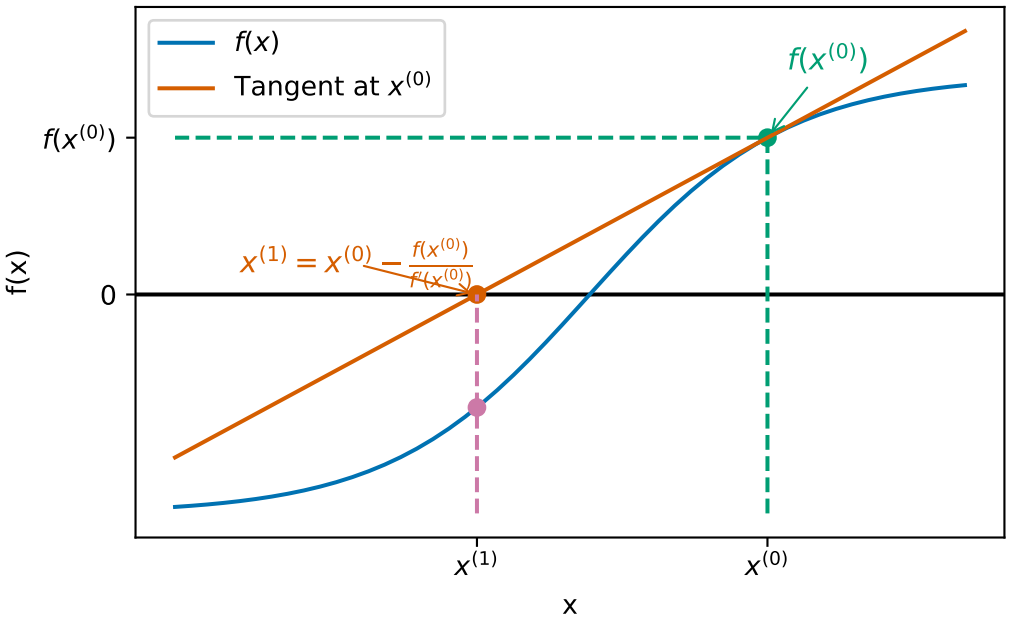
6.3.4 Weaknesses of the bisection algorithm

Bisection is a reliable method for finding solutions of $f(x) = 0$ provided an initial bracket can be found. It is far from perfect however:

- finding an initial bracket is not always easy;
- it can take a very large number of iterations to obtain an accurate answer;
- it can never find a solution of $f(x)$ for which f does not change sign (e.g. $f(x) = (x-1)^2$) since we cannot find a bracket for the root!

6.4 Newton's method

Newton's method (or the Newton-Raphson iteration) is an alternative algorithm for solving $f(x) = 0$. On the positive side, it does not need an initial bracket (just one initial value), it converges much more quickly than bisection and it can solve problems where the solution is also a turning point. On the negative side, it is not guaranteed to converge (unlike bisection with a good initial bracket).



We start at our initial guess $x^{(0)}$ and find the corresponding point on the graph of the function $(x^{(0)}, f(x^{(0)}))$. We find the tangent to the graph at $(x^{(0)}, f(x^{(0)}))$ and follow this line to the x-axis. Where the tangent line intersects the x-axis is our new value for $x^{(1)}$.

In the picture, we can see two ways to calculate the tangent:

1. We can use the derivative of f to see that the slope of the tangent is $f'(x^{(0)})$.
2. We can see that we have a right-angled triangle with vertices at $(x^{(1)}, 0)$, $(x^{(0)}, 0)$ and $(x^{(0)}, f(x^{(0)})$ which has slope

$$\frac{\text{change in } y}{\text{change in } x} = \frac{f(x^{(0)}) - 0}{x^{(0)} - x^{(1)}}.$$

Equating the two slopes, we see that

$$f'(x^{(0)}) = \frac{f(x^{(0)})}{x^{(0)} - x^{(1)}},$$

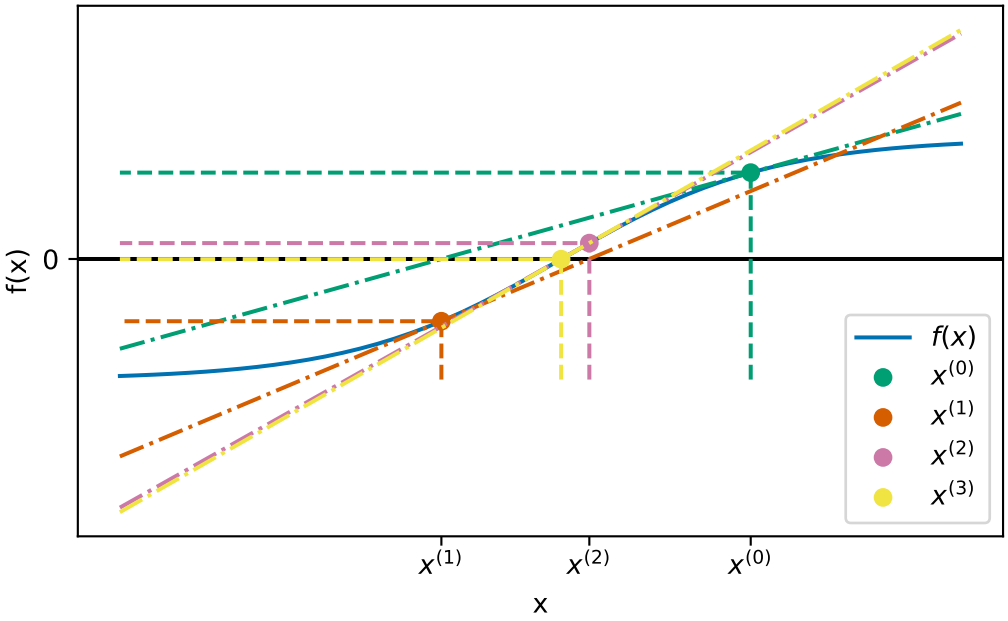
which we can rearrange to

$$x^{(1)} = x^{(0)} - \frac{f(x^{(0)})}{f'(x^{(0)})}.$$

This is the formula for a single Newton update!

Once we have our updated estimate $x^{(1)}$, we can continue the iteration:

$$x^{(i+1)} = x^{(i)} - \frac{f(x^{(i)})}{f'(x^{(i)})}.$$



This is quite a messy plot but it appears the Newton's method converges very quickly!

Remark 6.3.

1. The iteration formula requires an initial guess at the solution $x^{(0)}$.
2. In order to apply Newton's method, we need to be able to compute an expression for the derivative of $f(x)$ - this may not always be possible or easy.
3. You will see a variant of Newton's method that allows the derivative to be approximated in the lab session.
4. We typically use the absolute value of $f(x^{(i)})$ as the variable we compare against a tolerance in a stopping criteria.

6.4.1 Examples

```
def newton(f, df, x0, tol, maxiter=20):
    """
    Find a root of a scalar function using Newton's method.

    Newton's method (also called the Newton-Raphson method) generates a sequence
    of iterates
    ``x_{k+1} = x_k - f(x_k) / df(x_k)``
    which (when it converges) typically converges rapidly to a root ``x*`` such
    that ``f(x*) = 0``. This implementation stops when ``abs(f(x)) <= tol`` or
    when ``maxiter`` iterations have been performed.

    Parameters
    ----------
    f : callable
        Scalar function of one variable, ``f(x)``, whose root is sought.
    df : callable
        Derivative of ``f``, i.e. ``df(x) = f'(x)``. Must be defined at each
        iterate visited by the algorithm.
    x0 : float
        Initial guess for the root.
    tol : float
        Absolute tolerance on the residual. Iteration stops when
        ``abs(f(x)) <= tol``.
    maxiter : int, optional
        Maximum number of iterations to perform. Default is 20.

    Returns
    -------
    data: list[dict]
        Iteration data for analysing results
    """
    data = []

    # the iteration
    it = 0
    x = x0
    data.append({"it": it, "x": x, "f(x)": f(x), "f'(x)": df(x)})

    while abs(f(x)) > tol and it < maxiter:
        x_new = x - f(x) / df(x)
```

```
        x = x_new
        it += 1

        data.append({"it": it, "x": x, "f(x)": f(x), "f'(x)": df(x)})

    return data
```

$$f(x) = x^2 - 2.$$

```
"|   it |       x |        f(x) |   f'(x) |\n|-----:|--------:|------------:|--------:|\n|
```

We see that it takes *only* 3 iterations to find the solution to a tolerance of 10^{-4} for the square root problem.

$$f(x) = x - 0.7\sin(x) - 2.0$$

```
"|   it |       x |        f(x) |   f'(x) |\n|-----:|--------:|------------:|--------:|\n|
```

We see that it takes 3 iterations to find the solution to a tolerance of 10^{-4} for the Kepler problem

Example 6.5. Consider $f(x) = x^2 - 5$ and $x^{(0)} = 2$. This gives $f'(x) = 2x$ so the iterative formula is:

$$x^{(i+1)} = x^{(i)} - \frac{(x^{(i)})^2 - 5}{2x^{(i)}} = \frac{(x^{(i)})^2 + 5}{2x^{(i)}}.$$

The iteration then proceeds as:

$$x^{(1)} = \frac{(x^{(0)})^2 + 5}{2x^{(0)}} = \frac{2^2 + 5}{2 \times 2} = \frac{9}{4} = 2.25.$$
$$x^{(2)} = \frac{(x^{(1)})^2 + 5}{2x^{(1)}} = \frac{(2.25)^2 + 5}{2 \times 2.25} = \frac{161}{72} \approx 2.23611.$$

Note that $\sqrt{5} = 2.23607\ldots$.

Exercise 6.2. Write down an expression for the Newton iteration formula for the following functions and then carry out 2 iterations using the resulting iteration and given initial value $x^{(0)}$.

1. $f(x) = x^3 - 2x - 5$ with $x^{(0)} = 2$.
2. $f(x) = x^3 - 2$ with $x^{(0)} = 1$.

6.4.2 Challenges for Newton's method

The first obvious challenge for Newton's method comes from the idea that we divide by something that might be zero in our update formula – what if $f'(x^{(i)}) = 0$? In this case, there isn't much we can do and Newton's method will fail.

Example 6.6. Consider applying Newton's method to the function $f(x) = x^3 + 2x^2 + x + 1$ with $x^{(0)} = 0$. This gives $f'(x) = 3x^2 + 4x + 1$. Starting from $x^{(0)} = 0$, we get:

$$x^{(1)} = x^{(0)} - \frac{f(x^{(0)})}{f'(x^{(0)})} = 0 - \frac{0^3 + 2 \times 0 + 0 + 1}{3 \times 0^2 + 4 \times 0 + 1} = 0 - \frac{1}{1} = -1.$$

But we see that

$$f'(x^{(1)}) = 3 \times (-1)^2 + 4 \times -1 + 1 = 0.$$

So the iteration cannot proceed.

Another similar case is what if the derivative at the root is zero (i.e., $f'(x^*) = 0$), in this case the iteration can still proceed but the iteration is *slower*.

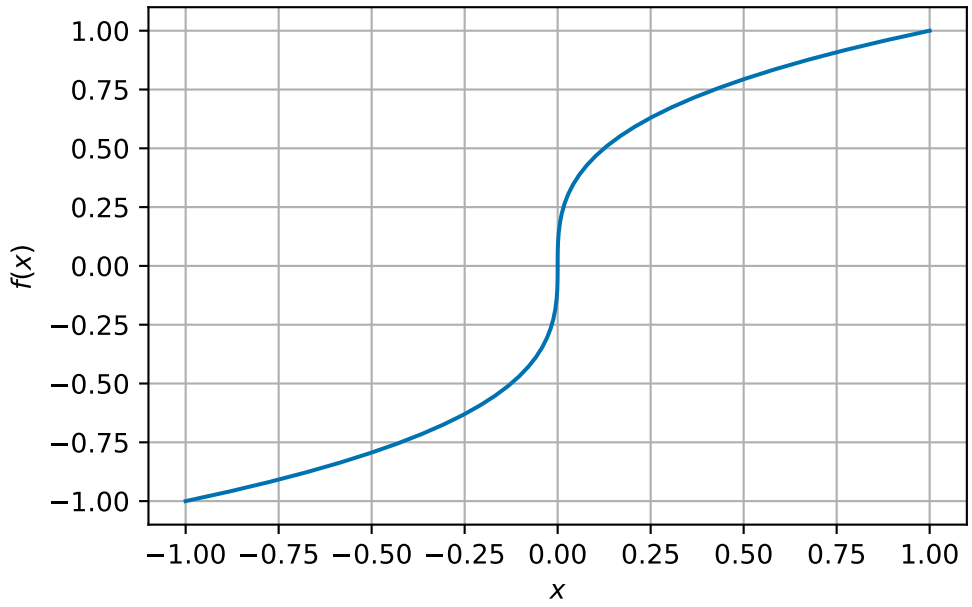
Consider $f(x) = (x-1)^2 = x^2 - 2x + 1$:

```
"|   it |       x |        f(x) |    f'(x) |\n|-----:|--------:|------------:|---------:|\n|
```

It takes 7 iterations to find the root in this case (more than the other Newton examples). We note here also that the convergence criterion is $|f(x^{(i)})| < TOL$ which does not guarantee that $|x^* - x^{(i)}| < TOL$!

There are also cases where Newton's method does not converge even when a root exists!

Example 6.7. Consider the function $f(x) = x^{1/3}$. Clearly f has a root at $x^* = 0$ ($f(0) = 0$).



The derivative is $f'(x) = \frac{1}{3}x^{-2/3}$ and the Newton iteration is

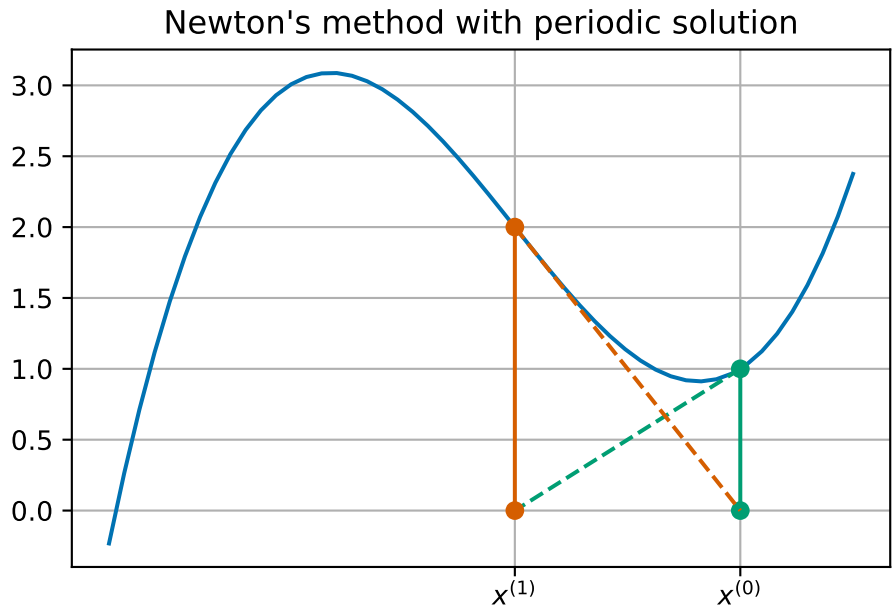
$$x^{(i+1)} = x^{(i)} - \frac{(x^{(i)})^{1/3}}{\frac{1}{3}(x^{(i)})^{-2/3}} = x^{(i)} - 3x^{(i)} = -2x^{(i)}.$$

We can see that the general term in this sequence is $x^{(i)} = (-2)^i x^{(0)}$ – this is clearly not going to converge to 0!

```
"|   it |    x |     f(x) |      f'(x) |\n|-----:|-----:|---------:|-----------:|\n|    0 |
```

It is even possible for the iteration to simply cycle between two values repeatedly.

Example 6.8. Consider the function $f(x) = x^3 - 2x + 2$. This has derivative $f'(x) = 3x^2 - 2$.



Starting an iteration at $x^{(0)} = 1$ gives:

```
"|   it |   x |   f(x) |   f'(x) |\n|-----:|----:|-------:|--------:|\n|    0 |   1 |      1
```

6.5 Summary

We have presented two methods for finding roots of nonlinear equations: the bisection method and Newton's method. We summarise their properties in the following table:

	Bisection	Newton
Simple algorithm	yes	yes
Starting values	bracket	one
Iterations	lots	normally fewer
Convergence	with good bracket	not always
Turning point roots	no	slower
Use derivative	no	yes

There are many more methods available - you will see one on the lab exercise sheet.

7 What about functions with higher dimension domains?

Module learning objective

1. Compute directional derivatives, partial derivatives, gradients and Hessians of functions $F\colon \mathbb{R}^n \to \mathbb{R}$,
2. Compute the Jacobian of a function $\vec{V}\colon \mathbb{R}^n \to \mathbb{R}^m$,
3. Give and apply a condition that shows that a function $F\colon \mathbb{R}^n \to \mathbb{R}$ is convex.

When we were working with a function $F\colon \mathbb{R} \to \mathbb{R}$, we had the key idea to find the points where the derivative of F is 0. We have developed theory to tell us when these points are local or global minima and some methods to find these points.

> When working with functions with two-dimensional (or more) domains, how do we compute derivatives?

Throughout this section we will work with the example function:

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2.$$

This function has a unique minimum at $\vec{x}^* = (-1, 0)$ where $F* = 0$.

7.1 Directional derivatives

Somehow we want to use the idea that if you move away from the minimum in *any* direction then the function increases. Mathematically, we could write this as saying: A point $\vec{x}^*$ is a local minimum if, for *any* direction $\vec{v} \in \mathbb{R}^n$, the value $h = 0$ is a local minimum of:

$$\gamma_{\vec{v}}(h) = F(\vec{x}^* + h\vec{v}) \geq F(\vec{x}^*) \quad \text{for all small} \quad h.$$

We could try to use our ideas from one dimension then by considering the function $\gamma_{\vec{v}}\colon \mathbb{R} \to \mathbb{R}$. This is a well-defined one-dimensional function so we can apply all our theory as before.

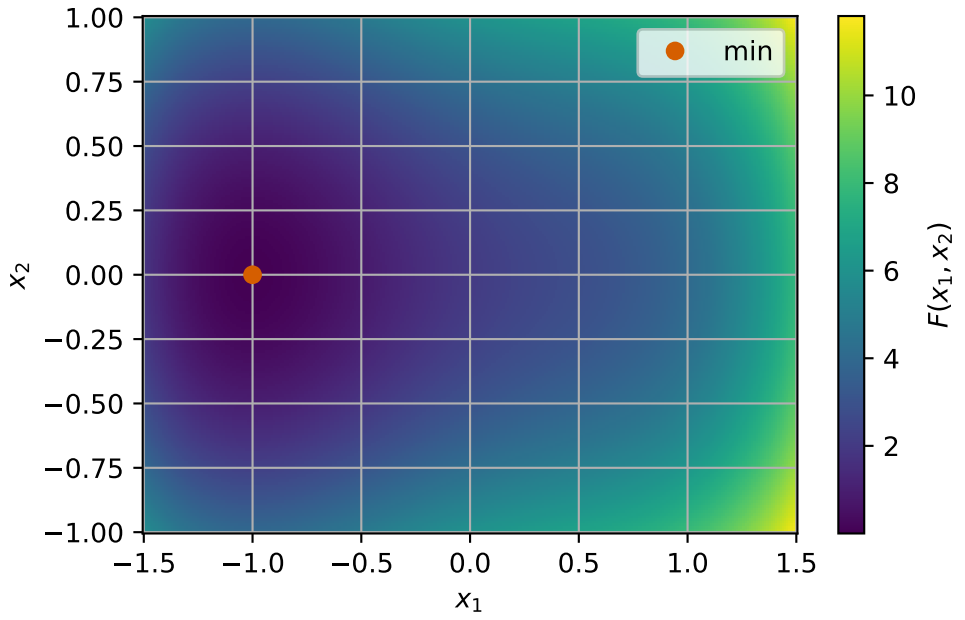


Figure 7.1: Isocolour plot of F showing unique minimum.

Example 7.1. Consider the function

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2$$

and the directions $\vec{v} = (1, 0)^T, (0, 1)^T$ and $(1, 1)^T$. Find $\gamma_{\vec{v}}(h)$ and $\gamma'_{\vec{v}}(h)$ at $\vec{x}^* = (-1, 0)$.

1. For $\vec{v} = (1, 0)^T$, we have

$$\begin{aligned}
\gamma_{\vec{v}}(h) &= F(-1 + h, 0) \\
&= (-1 + h)^4 - (-1 + h)^2 + 2(-1 + h) + 4 \times 0^2 + 2 \\
&= (-1)^4 + 4 \times (-1)^3 \times h + 6 \times (-1)^2 \times h^2 + 4 \times (-1) \times h^3 + h^4 \\
&\quad - (-1)^2 - 2 \times (-1) \times h - h^2 + 2 \times (-1) + 2 \times h + 0 + 2 \\
&= 1 - 4h + 6h^2 - 4h^3 + h^4 - 1 + 2h - h^2 - 2 + 2h + 2 \\
&= h^4 - 4h^3 + 5h^2,
\end{aligned}$$

   so

$$\gamma'_{\vec{v}}(h) = 4h^3 - 12h^2 + 10h.$$

2. For $\vec{v} = (0, 1)^T$, we have

$$\begin{aligned}
\gamma_{\vec{v}}(h) &= F(-1, 0 + h) \\
&= (-1)^4 - (-1)^2 + 2 \times (-1) + 4h^2 + 2 \\
&= 1 - 1 - 2 + 4h^2 + 2 = 4h^2,
\end{aligned}$$

so

$$\gamma'_{\vec{v}}(h) = 8h.$$

3. For $\vec{v} = (1,1)^T$, we have

$$\begin{aligned}\gamma_{\vec{v}}(h) &= F(-1+h, 0+h)\\ &= (-1+h)^4 - (-1+h)^2 + 2(-1+h) + 4 \times h^2 + 2\\ &= (h^4 - 4h^3 + 5h^2) + (4h^2)\\ &= h^4 - 4h^3 + 9h^2,\end{aligned}$$

so

$$\gamma'_{\vec{v}}(h) = 4h^3 - 12h^2 + 18h.$$

We note that each derivative depends on h and that for each derivative we have $\gamma'_{\vec{v}}(0) = 0$ so perhaps we do have a candidate local minimum of F? (c.f. Proposition 5.1)

The object we have been computing in this example is called the *directional derivative*:

Definition 7.1. Let $F\colon \mathbb{R}^n \to \mathbb{R}$. We use the notation $F'_{\vec{v}}$ for **directional derivative of** F **at** $\vec{x}$ **in a direction** $\vec{v}$ (with $\vec{v} \neq \vec{0}$) and this is given by:

$$F'_{\vec{v}}(\vec{x}) = \gamma'_{\vec{v}}(0)\frac{1}{\|\vec{v}\|}.$$

So we might want to think about building theory based on directional derivatives. However, as we have seen in the picture of F above, there are still lots of directions to check. We have checked 3 different directions but there are still infinitely many to do!

There is something else we can do, and the calculations for the direction $\vec{v} = (1,1)^T$ give us a clue! We saw that

$$\gamma'_{(1,1)^T}(h) = \gamma'_{(1,0)^T}(h) + \gamma'_{(0,1)^T}(h).$$

This is also true in general: For directions $\vec{v}$ and $\vec{w}$ and a number a

$$\begin{aligned}\gamma'_{\vec{v}+\vec{w}}(h) &= \gamma'_{\vec{v}}(h) + \gamma'_{\vec{w}}(h)\\ \gamma'_{a\vec{v}}(h) &= a\gamma'_{\vec{v}}(h).\end{aligned}$$

So if we can find a basis (remember that term from semester 1?!?), we can write any directional derivative in terms of the basis vectors. We can make a simple choice for this: the coordinate axes!

Let $\vec{e}^{(j)}$ be the vector with 1 in the jth place and 0 in every other place for $j = 1, 2, \ldots n$. For example in $\mathbb{R}^2$, we have $\vec{e}^{(1)} = (1, 0)^T$ and $\vec{e}^{(2)} = (0, 1)^T$. These form a basis. So any direction $\vec{v}$ can be written as

$$\vec{v} = (v_1, v_2, \ldots, v_n) = \sum_{j=1}^{n} v_j \vec{e}^{(j)},$$

so that

$$\gamma'_{\vec{v}}(h) = \sum_{j=1}^{n} v_j \gamma'_{\vec{e}^{(j)}}(h).$$

Then, using the fact that $\|\vec{e}^{(j)}\| = 1$, we see that:

$$F'_{\vec{v}}(\vec{x}) = \gamma'_{\vec{v}}(0)\frac{1}{\|\vec{v}\|} = \frac{1}{\|\vec{v}\|}\sum_{j=1}^{n} v_j \gamma'_{\vec{e}^{(j)}}(0) = \frac{1}{\|\vec{v}\|}\sum_{j=1}^{n} v_j F'_{\vec{e}^{(j)}}(\vec{x}).$$

This idea is special enough that we give the terms $F'_{\vec{e}^{(j)}}$ a special name: *partial derivatives*, and there is a slightly simpler way to calculate them.

7.2 Partial derivatives

Definition 7.2. Let $F\colon \mathbb{R}^n \to \mathbb{R}$. Then the partial derivative of F at a point $\vec{x}$ with respect to the jth variable x_j is defined as:

$$\frac{\partial F}{\partial x_j}(\vec{x}) = \lim_{h \to 0} \frac{F(\vec{x} + h\vec{e}^{(j)}) - F(\vec{x})}{h}.$$

Remark 7.1. The notation we use includes the curved ∂ symbol which looks like a curvy "d" and is pronounced "partial".

We use the same examples as the previous section to give an idea of how to calculate a partial derivative:

Example 7.2. Consider once more the function

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2.$$

We want to compute the partial derivative with respect to x_1.

The definition tells us to compute:

$$\begin{aligned}
&\frac{\partial F}{\partial x_1}(\vec{x}) \\
&= \lim_{h\to 0} \frac{F(\vec{x}+h\vec{e}^{(1)}) - F(\vec{x})}{h} \\
&= \lim_{h\to 0} \frac{((x_1+h)^4 - (x_1+h)^2 + 2(x_1+h) + 4x_2^2 + 2) - (x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2)}{h} \\
&= \lim_{h\to 0} \frac{(x_1^4 + 4x_1^3h + 6x_1^2h^2 + 4x_1h^3 + h^4 - x_1^2 - 2x_1h - h^2 + 2x_1 + 2h + 4x_2^2 + 2) - (x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2)}{h} \\
&= \lim_{h\to 0} \frac{h(4x_1^3 - 2x_1 + 2) + h^2(6x_1^2 - 1) + h^3(4x_1)}{h} \\
&= \lim_{h\to 0} (4x_1^3 - 2x_1 + 2) + h(6x_1^2 - 1) + h^2(4x_1) \\
&= 4x_1^3 - 2x_1 + 2.
\end{aligned}$$

But we notice that when we do this calculation, we are effectively finding the usual derivative with respect to x_1 but ignoring terms with x_2. This is an approach we can follow.

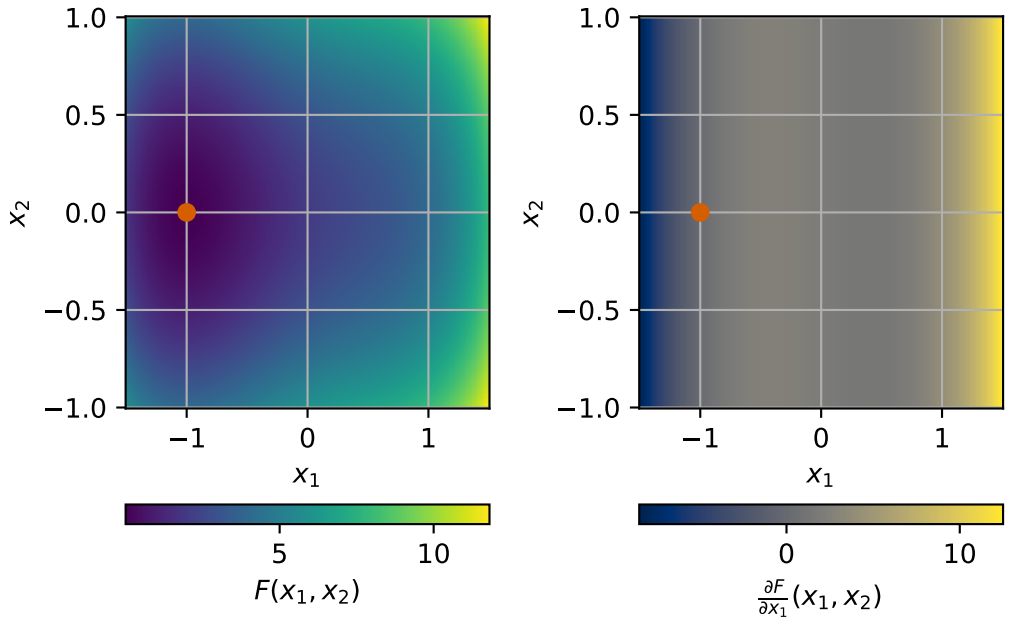
Strictly, when computing partial derivatives with respect to a variable x_j, we compute the usual derivative with respect to x_1 and treat all other variables as *constants*: i.e. $\frac{\partial}{\partial x_1} x_2 = 0$.

In this example, we would compute

$$\begin{aligned}
&\frac{\partial F}{\partial x_1}(\vec{x}) \\
&= \frac{\partial}{\partial x_1}(x_1^4) - \frac{\partial}{\partial x_1}x_1^2 + 2\frac{\partial}{\partial x_1}x_1 + 4\frac{\partial}{\partial x_1}x_2^2 + \frac{\partial}{\partial x_1}2 \\
&= 4x_1^3 - 2x_1 + 2 + 0 + 0 = 4x_1^3 - 2x_1 + 2.
\end{aligned}$$

We see we get the same result as using limits.

Exercise 7.1. Compute $\frac{\partial F}{\partial x_2}$ for this example using either approach.



The ideas in these examples let us see that many of the properties of usual derivatives carry over to partial derivatives too:

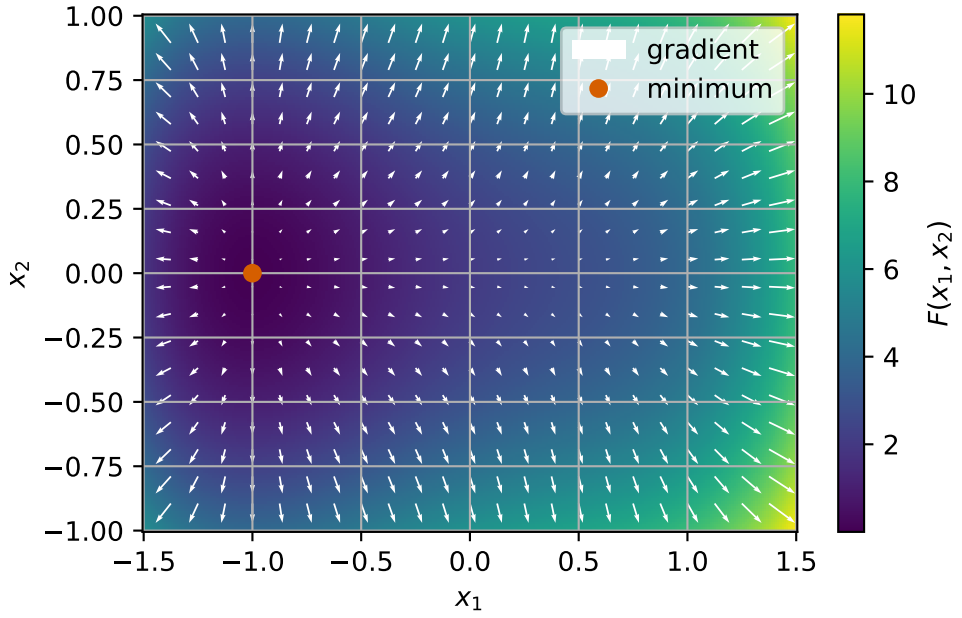
1. sum rule:
$$\frac{\partial}{\partial x_j}(aF(\vec{x}) + bG(\vec{x}) = a\frac{\partial}{\partial x_j}F(\vec{x}) + b\frac{\partial}{\partial x_j}G(\vec{x})$$
2. product rule:
$$\frac{\partial}{\partial x_j}(F(\vec{x})G(\vec{x})) = G(\vec{x})\frac{\partial}{\partial x_j}F(\vec{x}) + F(\vec{x})\frac{\partial}{\partial x_j}G(\vec{x})$$

7.3 Gradient

From a calculation point of view, the gradient is really just a stack of partial derivatives put together. However, it turns out to be a useful construction beyond this too.

Definition 7.3. Let $F\colon \mathbb{R}^n \to \mathbb{R}$. Then the **gradient of** F is a function $\nabla F : \mathbb{R}^n \to \mathbb{R}^n$ which is given by
$$\nabla F(\vec{x}) = \begin{pmatrix} \frac{\partial F}{\partial x_1}(\vec{x}) \\ \frac{\partial F}{\partial x_2}(\vec{x}) \\ \vdots \\ \frac{\partial F}{\partial x_n}(\vec{x}) \end{pmatrix}.$$

Example 7.3. With the same F as before, we can add white arrows to our previous plot to show the gradient of F. At various points $\vec{x}$ in the domain, we plot an arrow starting at $\vec{x}$ in the direction $\nabla F(\vec{x})$. The size of the arrow corresponds to the magnitude of the gradient.



We note that the gradient seems to point away from the minimum at every point and seems to be small (perhaps even the zero vector?) at the minimum. Indeed:

$$\nabla F(\vec{x}) = (4x_1^3 - 2x_1 + 2, 8x_2),$$

so

$$\nabla F(-1, 0) = (4 \times (-1)^3 - 2 \times (-1) + 2, 8 \times 0) = (-4 + 2 + 2, 0) = (0, 0).$$

We again see that the gradient follows some simple results: for functions $F, G \colon \mathbb{R}^n \to \mathbb{R}$ and numbers a, b we have

1. sum rule:
$$\nabla(aF(\vec{x}) + bG(\vec{x})) = a\nabla F(\vec{x}) + b\nabla G(\vec{x}).$$
2. product rule:
$$\nabla(F(\vec{x})G(\vec{x})) = G(\vec{x})\nabla F(\vec{x}) + F(\vec{x})\nabla G(\vec{x}).$$

One reason that the gradient is useful is the following lemma that connects the gradient to directional derivatives:

Lemma 7.1. *Let $F\colon \mathbb{R}^n \to \mathbb{R}$ be a smooth function. Then for any point $\vec{x}$ and any direction $\vec{v}$ we have*

$$F'_{\vec{v}}(\vec{x}) = \nabla F(\vec{x}) \cdot \frac{\vec{v}}{\|\vec{v}\|}.$$

Proof. We sketch the key ideas here only.

Write $\vec{v} = (v_1, \dots, v_n)$ then for small h, $F(\vec{x} + h\vec{v}) - F(\vec{x})$ can be built up by changing one coordinate at a time:

$$\begin{aligned}
&F(\vec{x} + h\vec{v}) - F(\vec{x}) \\
&=\big(F(x_1 + hv_1, \dots, x_n + hv_n) - F(x_1 + hv_1, \dots, x_{n-1} + hv_{n-1}, x_n)\big) \\
&\quad + \big(F(x_1 + hv_1, \dots, x_{n-1} + hv_{n-1}, x_n) - F(x_1 + hv_1, \dots, x_{n-2} + hv_{n-2}, x_{n-1}, x_n)\big) \\
&\quad + \cdots \\
&\quad + \big(F(x_1 + hv_1, x_2, \dots, x_n) - F(x_1, x_2, \dots, x_n)\big).
\end{aligned}$$

Each bracket changes only one variable, so it behaves like a one-variable derivative:

$$F(\vec{x} + h\vec{v}) - F(\vec{x}) \approx h \sum_{j=1}^{n} \frac{\partial F}{\partial x_j}(\vec{x})\, v_j.$$

Dividing by h and letting $h \to 0$ gives

$$\gamma'_{\vec{v}}(0) = \sum_{j=1}^{n} \frac{\partial F}{\partial x_j}(\vec{x})\, v_j = \nabla F(\vec{x}) \cdot \vec{v}.$$

Then divide by $\|\vec{v}\|$ gives:

$$F'_{\vec{v}}(\vec{x}) = \nabla F(\vec{x}) \cdot \frac{\vec{v}}{\|\vec{v}\|}.$$

□

We also have a geometric understanding of what the gradient is in a similar way to the derivative of a function of a scalar variable representing the slope of the function.

Lemma 7.2. *Let $F\colon \mathbb{R}^n \to \mathbb{R}$ be a smooth function. The gradient of F represents the direction in which F increase the most rapidly.*

Proof. The direction of F which increases the fastest can be found by finding the direction $\vec{v}$ which gives the greatest value for:

$$F'_{\vec{v}}(\vec{x})$$

at an arbitrary point $\vec{x}$.

Using Lemma 7.1, we have

$$F'_{\vec{v}}(\vec{x}) = \nabla F(\vec{x}) \cdot \frac{\vec{v}}{\|\vec{v}\|}.$$

We can see that this is maximised for $\vec{v} = \nabla F(\vec{x})$ (e.g. using the dot product formula $\vec{a} \cdot \vec{b} = \|\vec{a}\|\|\vec{b}\| \cos\theta$) where

$$F'_{\vec{v}}(\nabla F(\vec{x})) = \nabla F(\vec{x}) \cdot \frac{\nabla F(\vec{x})}{\|\nabla F(\vec{x})\|} = \frac{\|\nabla F(\vec{x})\|^2}{\|\nabla F(\vec{x})\|} = \|\nabla F(\vec{x})\|.$$

In the final calculation we assume that $\|\nabla F(\vec{x})\| \neq 0$. Otherwise, $\nabla F(\vec{x}) = \vec{0}$ and all directional derivatives are zero so any $\vec{v}$ can maximise the expression. □

Remark 7.2. This result is very significant. It gives us an idea of how to manipulate a point towards a minimum - we can move in the direction opposite the gradient (i.e. $-\nabla F(\vec{x})$) and we will be following the steepest descent direction.

7.4 The Hessian

For scalar domain problems, we used the second derivative to characterise convexity of functions. For functions with a higher dimension domain, we can also use second derivatives - only now there are lots of them.

Definition 7.4. Let $F \colon \mathbb{R}^n \to \mathbb{R}$ be a smooth function. Then the **Hessian matrix** of F is a square $n \times n$ matrix of second partial derivatives defined and arranged as:

$$\nabla^2 F = \begin{pmatrix} \frac{\partial^2 F}{\partial x_1^2} & \frac{\partial^2 F}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 F}{\partial x_1 \partial x_n} \\ \frac{\partial^2 F}{\partial x_2 \partial x_1} & \frac{\partial^2 F}{\partial x_2^2} & \cdots & \frac{\partial^2 F}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 F}{\partial x_n \partial x_1} & \frac{\partial^2 F}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 F}{\partial x_n^2}. \end{pmatrix}$$

Here, and elsewhere, we write

$$\frac{\partial^2 F}{\partial x_i \partial x_j} = \frac{\partial}{\partial x_i}\left(\frac{\partial}{\partial x_j} F\right),$$

to denote first taking the partial derivative with respect to x_j and then the partial derivative with respect to x_i.

Example 7.4. We want to compute the Hessian of

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2.$$

We already have the gradient:

$$\nabla F(x_1, x_2) = \begin{pmatrix} 4x_1^3 - 2x_1 + 2 \\ 8x_2 \end{pmatrix}.$$

Then for each term in the Hessian we compute:

$$\begin{aligned} \frac{\partial^2}{\partial x_1^2} F(x_1, x_2) &= \frac{\partial}{\partial x_1}(4x_1^3 - 2x_1 + 2) && = 12x_1^2 - 2 \\ \frac{\partial^2}{\partial x_1 \partial x_2} F(x_1, x_2) &= \frac{\partial}{\partial x_1}(8x_2) && = 0 \\ \frac{\partial^2}{\partial x_2 \partial x_1} F(x_1, x_2) &= \frac{\partial}{\partial x_2}(4x_1^3 - 2x_1 + 2) && = 0 \\ \frac{\partial^2}{\partial x_2^2} F(x_1, x_2) &= \frac{\partial}{\partial x_2}(8x_2) && = 8. \end{aligned}$$

So the Hessian is given by:

$$\nabla^2 F(x_1, x_2) = \begin{pmatrix} 12x_1^2 - 2 & 0 \\ 0 & 8 \end{pmatrix}.$$

Exercise 7.2. Compute the Hessian of

$$F(x_1, x_2) = 2x_1^2 + 2x_2^2 - 2x_1 x_2.$$

We notice that for both the example and exercise the Hessian is a symmetric matrix. This is true in general!

Lemma 7.3. *For a smooth function $F \colon \mathbb{R}^n \to \mathbb{R}$, we have*

$$\frac{\partial^2 F}{\partial x_i \partial x_j} = \frac{\partial^2 F}{\partial x_j \partial x_i},$$

hence the Hessian is a symmetric matrix.

Proof. A full proof is beyond this course.

One intuition comes from the idea of walking small steps on the graph of the function. Walking the small steps in different orders gives the same change in height (F) so the second derivatives do not see which order the derivatives come. □

7.5 What does this say about optimisation?

We start with a definition of local minima here which matches the scalar domain case. To help writing definitions more compactly we introduce the idea of a ball. We use the notation $B(\vec{x}, R)$ to denote the ball of radius R centered at $\vec{x}$ which is given by

$$B(\vec{x}, r) = \{\vec{y} \in \mathbb{R}^n : \|\vec{y} - \vec{x}\| < r\}.$$

The idea of a ball generalises an interval to higher dimensions.

Definition 7.5. Let $F \colon \mathbb{R}^n \to \mathbb{R}$. We say a point $\vec{x}^*$ is a **local minimum** of F if for a small radius $r > 0$, we have that

$$F(\vec{x}^*) \leq F(\vec{x}) \quad \text{for all} \quad \vec{x} \in B(\vec{x}^*, r)$$

Remark 7.3. Comparing to Definition 5.2, we see that the biggest change is swapping the absolute value and an interval for a Euclidean norm and a ball.

Lemma 7.4. *Let $F \colon \mathbb{R}^n \to \mathbb{R}$ be a smooth function. Let x^* be a local minimum of F, then we have*

$$F'_{\vec{v}}(x^*) = 0 \quad \textit{for all non-zero directions} \quad \vec{v}.$$

and $\nabla F(x^*) = \vec{0}$.

Proof. Because $\vec{x}^*$ is a local minimum of F, for all sufficiently small h we have

$$F(\vec{x}^* + h\vec{v}) \geq F(\vec{x}^*),$$

or equivalently for $\gamma_{\vec{v}}$, we have

$$\gamma_{\vec{v}}(h) \geq \gamma_{\vec{v}}(0) \quad \text{for } h \text{ small enough.}$$

That is that $h = 0$ is a local minimiser of $\gamma_{\vec{v}}$. Since we have found a local minimiser, we know that (Proposition 5.1)

$$0 = \gamma'_{\vec{v}}(0) = F'_{\vec{v}}(\vec{x}^*)\|\vec{v}\|.$$

Hence, so long as $\vec{v} \neq 0$, we have $F'_{\vec{v}}(\vec{x}^*) = 0$.

Since we have all directional derivatives are 0 so in particular all partial derivatives are zero since they are the directional derivatives in the coordinate directions. Since the gradient is just a stack of partial derivatives we see that the gradient is the zero vector too. □

Previously to map from local to global minima, we used the idea of convexity. We can do the same again here:

Definition 7.6. Let $F\colon \mathbb{R}^n \to \mathbb{R}$ be a function. We say that F is **locally convex on a ball** B if for all $\vec{x}, \vec{y} \in B$, we have

$$aF(\vec{x}) + (1-a)F(\vec{y}) \geq F(a\vec{x} + (1-a)\vec{y}) \quad \text{for all} \quad a \in [0,1].$$

We say that F is **convex** if it is convex on $\mathbb{R}^n$ (i.e. convex for any ball $B \subset \mathbb{R}^n$).

Similar to **?@lem-convex-equiv**, we have the following equivalence characterisations of convexity on a ball:

Lemma 7.5. *Let $F\colon \mathbb{R}^n \to \mathbb{R}$ be a smooth function. Then the following are equivalent:*

1. *F is locally convex on a ball B*
2. *The gradient of F satisfies:*

$$F(\vec{x}) \geq F(\vec{y}) + \nabla F(\vec{y}) \cdot (\vec{x} - \vec{y}) \quad \textit{for all} \quad \vec{x}, \vec{y} \in B.$$

3. *The eigenvalues of the Hessian of $F(\vec{x})$ are non-negative for any $\vec{x} \in B$. (This property is normally stated by writing that $\nabla^2 F$ is* positive semi-definite*).*

Proof. We prove the implications (1) $\implies$ (2) $\implies$ (3) $\implies$ (1).

Throughout, let

$$\phi(t) = F(\vec{y} + t(\vec{x} - \vec{y})), \qquad t \in [0,1].$$

Since B is a ball, if $\vec{x}, \vec{y} \in B$, then

$$\vec{y} + t(\vec{x} - \vec{y}) \in B \qquad \text{for all } t \in [0,1].$$

(1) $\implies$ (2)

Assume F is convex on B. Then for any $\vec{x}, \vec{y} \in B$, the one-variable function

$$\phi(t) = F(\vec{y} + t(\vec{x} - \vec{y}))$$

is convex on $[0,1]$.

For a differentiable convex function of one variable, we have the supporting line inequality

$$\phi(1) \geq \phi(0) + \phi'(0).$$

Now

$$\phi(0) = F(\vec{y}), \qquad \phi(1) = F(\vec{x}),$$

and, using the previous lemma on directional derivatives (Lemma 7.1, we see

$$\phi'(0) = \nabla F(\vec{y}) \cdot (\vec{x} - \vec{y}).$$

Therefore

$$F(\vec{x}) \geq F(\vec{y}) + \nabla F(\vec{y}) \cdot (\vec{x} - \vec{y}),$$

which is exactly (2).

(2) $\implies$ (3)

Assume (2). Fix a point $\vec{y} \in B$ and a vector $\vec{v} \in \mathbb{R}^n$. For small t, the point $\vec{y} + t\vec{v}$ lies in B, so applying (2) with

$$\vec{x} = \vec{y} + t\vec{v}$$

gives

$$F(\vec{y} + t\vec{v}) \geq F(\vec{y}) + t\,\nabla F(\vec{y}) \cdot \vec{v}.$$

Define

$$\psi(t) = F(\vec{y} + t\vec{v}) - F(\vec{y}) - t\,\nabla F(\vec{y}) \cdot \vec{v}.$$

Then $\psi(t) \geq 0$ for all sufficiently small t, and $\psi(0) = 0$. Hence $t = 0$ is a local minimum of ψ, so

$$\psi''(0) \geq 0.$$

But

$$\psi''(0) = \vec{v}^T \nabla^2 F(\vec{y})\, \vec{v}.$$

Therefore

$$\vec{v}^T \nabla^2 F(\vec{y})\, \vec{v} \geq 0 \qquad \text{for all } \vec{v} \in \mathbb{R}^n.$$

It can be shown that this implies that the eigenvalues of $\nabla^2 F(\vec{y})$ are all non-negative, i.e. that $\nabla F^2(\vec{y})$ is positive-semi-definite.

(3) $\implies$ (1)

Assume (3). Let $\vec{x}, \vec{y} \in B$, and define

$$\phi(t) = F(\vec{y} + t(\vec{x} - \vec{y})), \qquad t \in [0, 1].$$

Then

$$\phi''(t) = (\vec{x} - \vec{y})^T \nabla^2 F(\vec{y} + t(\vec{x} - \vec{y}))(\vec{x} - \vec{y}).$$

By assumption, the Hessian is positive semidefinite at every point of B, so

$$\phi''(t) \geq 0 \qquad \text{for all } t \in [0, 1].$$

Thus ϕ is a convex one-variable function on $[0, 1]$. Therefore, for every $a \in [0, 1]$,

$$\phi(a) \leq (1 - a)\phi(0) + a\phi(1).$$

Substituting back,

$$F((1 - a)\vec{y} + a\vec{x}) \leq (1 - a)F(\vec{y}) + aF(\vec{x}).$$

This is exactly the statement that F is convex on B. Hence (1) holds.

Therefore, (1), (2), and (3) are equivalent. $\square$

Example 7.5. The function given by

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2$$

is not convex.

We have computed the Hessian as:

$$\nabla^2 F(x_1, x_2) = \begin{pmatrix} 12x_1^2 - 2 & 0 \\ 0 & 8 \end{pmatrix}.$$

This is a diagonal matrix so we can read off the eigenvalues as:

$$\lambda_1 = 12x_1^2 - 2 \quad \text{and} \quad \lambda_2 = 8.$$

We can see that λ_1 is negative for $|x_1| < 1/\sqrt{6}$ hence F is not (globally) convex.

Exercise 7.3. Is F given by

$$F(x_1, x_2) = 2x_1^2 + 2x_2^2 - 2x_1x_2$$

convex? First, find the Hessian of F and then the eigenvalues of $\nabla^2 F(\vec{x})$. What do they show you?

Remark 7.4. In exam questions, we will only expect you to be able to find eigenvalues in simple cases – e.g. for a 2×2 matrix or a diagonal matrix. We will not expect you to apply the QR factorsation for larger matrices.

7.6 What about a chain rule?

When introducing partial derivatives, we say that many of the rules from usual derivatives passed over including the sum rule and product rule. However, we did not mention the chain rule. Things are a little more complicated for partial derivatives here.

Example 7.6. Consider $F \colon \mathbb{R}^2 \to \mathbb{R}$ given by

$$F(x_1, x_2) = (2x_1 - 3)^2 + (3x_2 + 4)^2.$$

First, we can compute directly that

$$F(x_1, x_2) = 4x_1^2 - 12x_1 + 9x_2^2 + 24x_2 + 25.$$

Then

$$\frac{\partial F}{\partial x_1}(x_1, x_2) = 8x_1 - 12$$
$$\frac{\partial F}{\partial x_2}(x_1, x_2) = 18x_2 + 24.$$

But we can also see that we could write

$$(y_1, y_2) = \vec{V}(x_1, x_2) = (2x_1 - 3, 3x_2 + 4)$$

and then

$$F(x_1, x_2) = W(y_1, y_2) = y_1^2 + y_2^2.$$

so that $F = W \circ \vec{V}$ - i.e. $F(x_1, x_2) = W(\vec{V}(x_1, x_2))$. We might want to form a chain rule so that derivatives of F are somehow related to derivatives of $\vec{V}_1$ and W.

We can compute partial derivatives of W already. We need to answer the question:

1. What could it mean to say derivatives of $\vec{V} : \mathbb{R}^2 \to \mathbb{R}^2$?
2. How could we combine derivatives of $\vec{V}$ and W to get the right result?

Let's try to answer question 2 first.

Suppose we want to compute:

$$\frac{\partial F}{\partial x_j}(\vec{x}) = \lim_{h \to 0} \frac{F(\vec{x} + h\vec{e}^{(j)}) - F(\vec{x})}{h}.$$

Now

$$F(\vec{x} + h\vec{e}^{(j)}) - F(\vec{x}) = W(\vec{V}(\vec{x} + h\vec{e}^{(j)})) - W(\vec{V}(\vec{x})).$$

So the whole problem becomes: how does W change when *all* of its inputs (y_1, y_2) change by a small amount. We handle that by changing the inputs one at a time.

We have $\vec{y} = (y_1, y_2) = \vec{V}(x_1, x_2)$ and we also add the notation:

$$\begin{aligned}\vec{d}(h) = (d_1(h), d_2(h)) &= \vec{V}(\vec{x} + h\vec{e}^{(j)}) - \vec{V}(\vec{x}) \\ &= \vec{V}(x_1 + he_1^{(j)}, x_2 + he_2^{(j)}) - \vec{V}(x_1, x_2).\end{aligned}$$

Then

$$\vec{V}(\vec{x} + h\vec{e}^{(j)}) = \vec{y} + \vec{d}(h).$$

So we can compute that

$$\begin{aligned}&F(\vec{x} + h\vec{e}^{(j)}) - F(\vec{x}) \\ &= W(\vec{V}(\vec{x} + h\vec{e}^{(j)})) - W(\vec{V}(\vec{x})) \\ &= W(\vec{y} + \vec{d}(h)) - W(\vec{y}) \\ &= W(y_1 + d_1(h), y_2 + d_2(h)) - W(y_1, y_2) \\ &= (W(y_1 + d_1(h), y_2 + d_2(h)) - W(y_1 + d_1(h), y_2)) \\ &\quad + (W(y_1 + d_1(h), y_2) - W(y_1, y_2)).\end{aligned}$$

Thus, from the sum rule for limits, we have that

$$\begin{aligned}
&\lim_{h\to 0}\frac{F(\vec{x}+h\vec{e}^{(j)})-F(\vec{x})}{h}\\
&=\lim_{h\to 0}\frac{W(y_1+d_1(h),y_2+d_2(h))-W(y_1+d_1(h),y_2)}{h}\\
&\quad+\lim_{h\to 0}\frac{W(y_1+d_1(h),y_2)-W(y_1,y_2)}{h}.
\end{aligned}$$

We note that each limit looks a lot like a single partial derivative which follows something very similar to the one-dimensional chain rule. Using further results from Mathematical Analysis (Mean Value Theorem), we can see that

$$\lim_{h\to 0}\frac{F(\vec{x}+h\vec{e}^{(j)})-F(\vec{x})}{h}=\frac{\partial W}{\partial y_2}(y_1,y_2)d_2'(0)+\frac{\partial W}{\partial y_1}(y_1,y_2)d_1'(0).$$

Finally, we compute that

$$d_i'(0)=\lim_{h\to 0}\frac{V_i(x_1+he_1^{(j)},x_2+he_2^{(j)})-V_i(x_1,x_2)}{h}=\frac{\partial V_i}{\partial x_j}(\vec{x}).$$

Putting everything together, we have

$$\frac{\partial F}{\partial x_j}(\vec{x})=\frac{\partial W}{\partial y_1}(\vec{y})\frac{\partial V_1}{\partial x_j}(\vec{x})+\frac{\partial W}{\partial y_2}(\vec{y})\frac{\partial V_2}{\partial x_j}(\vec{x}).$$

Let's test this formula on our example.

$$\begin{aligned}
&\frac{\partial V_1}{\partial x_1}=2 && \frac{\partial V_1}{\partial x_2}=0\\
&\frac{\partial V_2}{\partial x_1}=0 && \frac{\partial V_2}{\partial x_2}=3\\
&\frac{\partial W}{\partial y_1}=2y_1 && \frac{\partial W}{\partial y_2}=2y_2.
\end{aligned}$$

Then, we compute that

$$\begin{aligned}
\frac{\partial F}{\partial x_1}(\vec{x})&=\frac{\partial W}{\partial y_1}(\vec{y})\frac{\partial V_1}{\partial x_1}(\vec{x})+\frac{\partial W}{\partial y_2}(\vec{y})\frac{\partial V_2}{\partial x_1}(\vec{x})\\
&=2y_1\times 2+2y_2\times 0\\
&=4\times(2x_1-3)+0\times(3x_2+4)=8x_1-12,
\end{aligned}$$

and

$$\begin{aligned}
\frac{\partial F}{\partial x_2}(\vec{x})&=\frac{\partial W}{\partial y_1}(\vec{y})\frac{\partial V_1}{\partial x_2}(\vec{x})+\frac{\partial W}{\partial y_2}(\vec{y})\frac{\partial V_2}{\partial x_2}(\vec{x})\\
&=2y_1\times 0+2y_2\times 3\\
&=0\times(2x_1-3)+6\times(3x_2+4)=18x_2+24.
\end{aligned}$$

We see that the results agree!

We return to the general case. Similar calculations to the above example give us this chain rule:

Lemma 7.6. *Let $F\colon \mathbb{R}^n \to \mathbb{R}$ be a smooth function given by $F = W \circ \vec{V}$ where $\vec{V}\colon \mathbb{R}^n \to \mathbb{R}^m$ and $W\colon \mathbb{R}^m \to \mathbb{R}$ are both smooth. Then*

$$\frac{\partial F}{\partial x_j}(\vec{x}) = \sum_{i=1}^{m} \frac{\partial W}{\partial y_i}(\vec{V}(\vec{x}))\frac{\partial V_i}{\partial x_j}(\vec{x}).$$

We have previously seen that the terms $\frac{\partial W}{\partial y_i}$ form part of the *gradient* of W. There is a similar object called the Jacobian for vector-valued functions. Since $\vec{V}$ has m output variables and n input variables, we store its partial derivative in a matrix with m rows and n columns:

Definition 7.7. Let $\vec{V}\colon \mathbb{R}^n \to \mathbb{R}^m$. Then the **Jacobian** of $\vec{V}$ at $\vec{x}$ is the $m \times n$ matrix given by:

$$J_{\vec{V}} = \left(\frac{\partial \vec{V}}{\partial x_1} \cdots \frac{\partial \vec{V}}{\partial x_n}\right) = \begin{pmatrix} (\nabla V_1)^T \\ \vdots \\ (\nabla V_m)^T \end{pmatrix} = \begin{pmatrix} \frac{\partial V_1}{\partial x_1} & \cdots & \frac{\partial V_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial V_m}{\partial x_1} & \cdots & \frac{\partial V_m}{\partial x_n} \end{pmatrix}.$$

That is the Jacobian is the matrix:

- with the jth columns given by the partial derivative of $\vec{V}$ with respect to x_j
- with the ith row given by the transpose of the gradient of V_i
- with the (i, j) entry given by $\frac{\partial V_i}{\partial x_j}$.

With this notation the chain rule (Lemma 7.6) can be written as the matrix vector product

$$\nabla F(\vec{x}) = \left(J_{\vec{V}}(\vec{x})\right)^T \nabla W(\vec{V}(\vec{x})).$$

7.7 Summary

We have introduced derivatives for functions $F\colon \mathbb{R}^n \to \mathbb{R}$:

- $F'_{\vec{v}}$: the directional derivative,
- $\frac{\partial}{\partial x_j}F$: partial derivative,
- ∇F: gradient,
- $\nabla^2 F$: Hessian.

We have also introduced the Jacobian of a function $\vec{V}\colon \mathbb{R}^n \to \mathbb{R}^m$ and used this to form a chain rule.

These are all important objects in their own right with a wide variety of uses. We will use them in the context of optimisation in this module. We have shown some ideas of how to show that a function F is convex depending on the eigenvalues of the Hessian of F.

8 Optimisation in $\mathbb{R}^n$

Module learning objective

1. Define and apply the gradient descent algorithm for functions $F\colon \mathbb{R}^n \to \mathbb{R}$,
2. Explain some of the properties of the gradient descent algorithm,
3. Relate the gradient descent algorithm to the gradient flow differential equation,
4. Apply the Newton method to solve optimsation problems.

TODO change step size to α

We return to our basic optimisation problem. We want to find a minimiser x^* of the function $F\colon \mathbb{R}^n \to \mathbb{R}$:

$$x^* = \arg\min_{x \in \mathbb{R}^n} F(x).$$

In this lecture, we are going to propose an iterative algorithm for approximating solutions called **gradient descent**. Roughly speaking, if F is smooth and convex, then for a sufficiently small step size the method converges to a minimiser. If F is not convex, gradient descent may still be useful, but in general it is only guaranteed to seek a point with $\nabla F = 0$; this may be a local minimiser but may not be!

Remark 8.1. The methods used to train neural network models are based on gradient descent. We will see an example of this later.

We first illustrate the algorithm in one dimension:

Example 8.1. Consider

$$F(x) = \frac{1}{4}x^4 - \frac{1}{12}x^3 - \frac{1}{2}x^2 + \frac{1}{4}x + \frac{1}{2}.$$

We will need the derivative of F given by:

$$F'(x) = x^3 - \frac{1}{4}x^2 - x + \frac{1}{4}.$$

We will start from $x^{(0)} = 0$. We note that

$$F(x^{(0)}) = \frac{1}{4} \times (0)^4 - \frac{1}{12} \times (0)^3 - \frac{1}{2} \times (0)^2 + \frac{1}{4} \times (0) + \frac{1}{2} = \frac{1}{2}$$

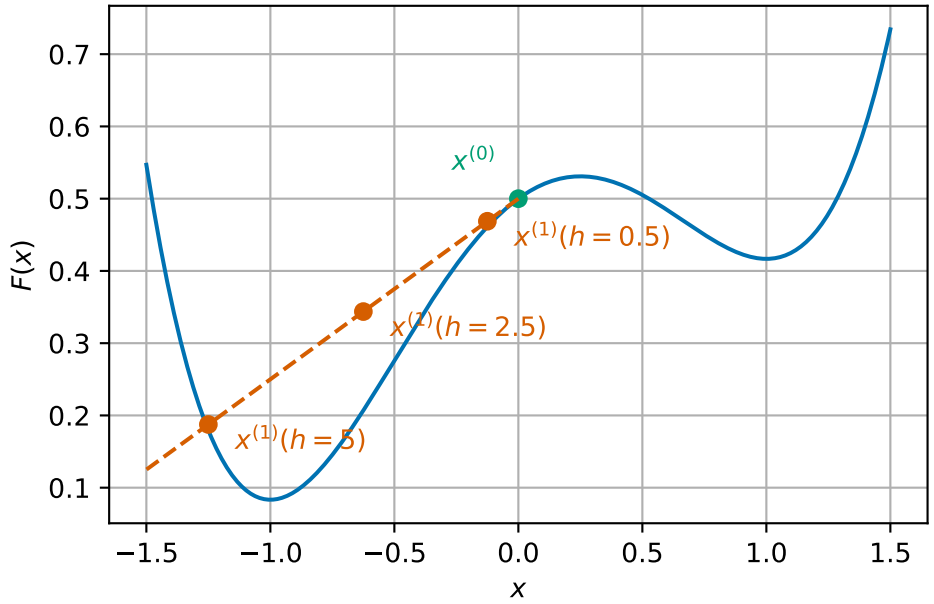
At $x^{(0)} = 0$, we see that the derivative of F is positive:

$$F'(0) = (0)^3 - \frac{1}{4} \times (0)^2 - (0) + \frac{1}{4} = \frac{1}{4} > 0.$$

Hence, to decrease F and get closer to the minimiser, we will take a *small* step in the opposite direction, i.e. in the $-F'(x^{(0)})$ direction. We use the update formula:

$$\begin{aligned} x^{(1)} &= x^{(0)} - hF'(x^{(0)}) \\ &= 0 - h \times \frac{1}{4}. \end{aligned}$$

Here $h > 0$ is the step size that we have to choose as a parameter in our algorithm.



This plot shows our scenario. In this case, $h = \frac{1}{2}$ seems a good choice so we set

$$x^{(1)} = x^{(0)} - hF'(x^{(0)}) = 0 - \frac{1}{2} \times \frac{1}{4} = -\frac{1}{8},$$

and note that

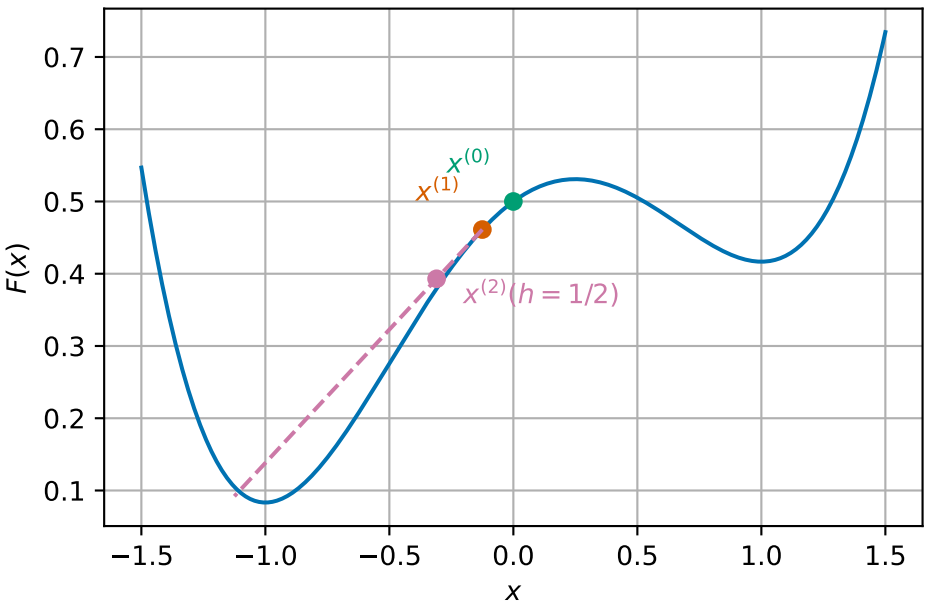
$$\begin{aligned} F(x^{(1)}) &= \frac{1}{4} \times \left(-\frac{1}{8}\right)^4 - \frac{1}{12} \times \left(-\frac{1}{8}\right)^3 - \frac{1}{2} \times \left(-\frac{1}{8}\right)^2 + \frac{1}{4} \times \left(-\frac{1}{8}\right) + \frac{1}{2} \\ &= \frac{22667}{49152} = 0.461161295572917. \end{aligned}$$

Then, we iterate to find $x^{(2)}$:

$$\begin{aligned} x^{(2)} &= x^{(1)} - hF'(x^{(1)}) \\ &= -\frac{1}{8} - \frac{1}{2} \times \left(\left(-\frac{1}{8} \right)^3 - \frac{1}{4} \times \left(-\frac{1}{8} \right)^2 - \left(-\frac{1}{8} \right) + \frac{1}{4} \right) \\ &= -\frac{1}{8} - \frac{1}{2} \times \left(\frac{189}{512} \right) \\ &= -\frac{317}{1024} = -0.309570312500000, \end{aligned}$$

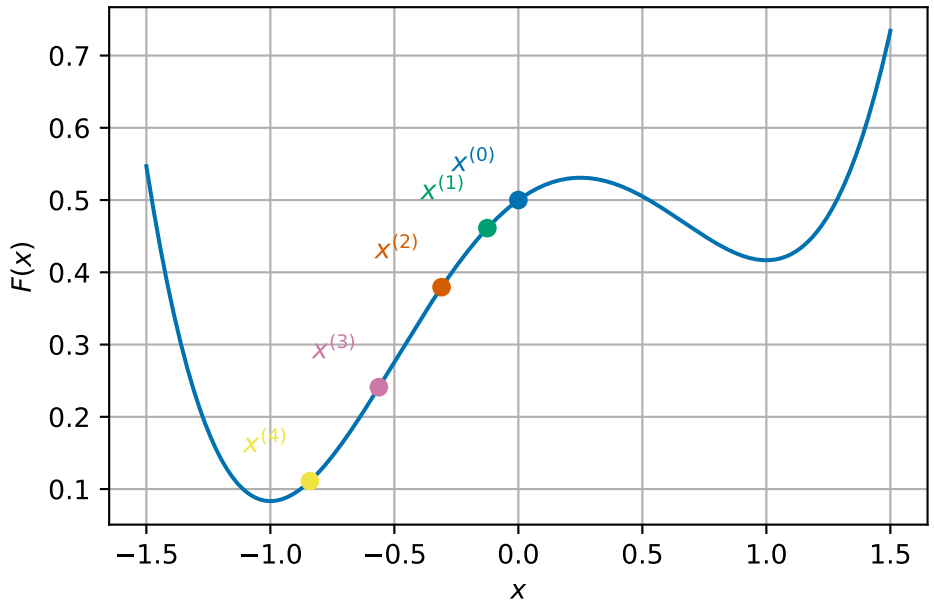
and note that

$$\begin{aligned} F(x^{(2)}) &= \frac{1}{4} \times \left(-\frac{317}{1024} \right)^4 - \frac{1}{12} \times \left(-\frac{317}{1024} \right)^3 - \frac{1}{2} \times \left(-\frac{317}{1024} \right)^2 + \frac{1}{4} \times \left(-\frac{317}{1024} \right) + \frac{1}{2} \\ &= \frac{5006632820723}{13194139533312} = 0.379458835347502. \end{aligned}$$



Continuing the iteration, we see that the sequence converges to the minimiser.

```
'|   it |          x |       F(x) |\n|-----:|-----------:|-----------:|\n|    0 | -0.125     | 0.4
```



We can use a similar idea when working in $\mathbb{R}^n$. The key idea is to note that the gradient of $F\colon \mathbb{R}^n \to \mathbb{R}$ points in the direction of the steepest increase in F (Lemma 7.2). So, starting at a point $\vec{x}^{(0)}$, if we want to reduce F, we should take a small step in the direction $-\nabla F(\vec{x}^{(0)})$. This is the **gradient descent algorithm** which, for a given step size h, uses the update formula:

$$\vec{x}^{(i+1)} = \vec{x}^{(i)} - h\nabla F(\vec{x}^{(i)}) \quad \text{for} \quad i = 0, 1, 2, \dots .$$

Example 8.2. Consider the function $F\colon \mathbb{R}^2 \to \mathbb{R}$ given by

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2.$$

We previously computed (Example 7.3) that

$$\nabla F(x_1, x_2) = (4x_1^3 - 2x_1 + 2, 8x_2).$$

So the gradient descent update formula is:

$$\begin{pmatrix} x_1^{(i+1)} \\ x_2^{(i+1)} \end{pmatrix} = \begin{pmatrix} x_1^{(i)} \\ x_2^{(i)} \end{pmatrix} - h \begin{pmatrix} 4(x_1^{(i)})^3 - 2x_1^{(i)} + 2 \\ 8x_2^{(i)} \end{pmatrix}.$$

Let $x^{(0)} = \begin{pmatrix} 1 \\ \frac{3}{4} \end{pmatrix}$ and $h = 1/4$.

Then we can compute that

$$\begin{aligned}
\vec{x}^{(1)} &= \vec{x}^{(0)} - h\nabla F(\vec{x}^{(i)}) \\
&= \begin{pmatrix} 1 \\ \frac{3}{4} \end{pmatrix} - \frac{1}{4} \times \nabla F(1, \frac{3}{4}) \\
&= \begin{pmatrix} 1 \\ \frac{3}{4} \end{pmatrix} - \frac{1}{4} \times \begin{pmatrix} 4 \times (1)^3 - 2 \times (1) + 2 \\ 8 \times (\frac{3}{4}) \end{pmatrix} \\
&= \begin{pmatrix} 1 \\ \frac{3}{4} \end{pmatrix} - \frac{1}{4} \times \begin{pmatrix} 4 \\ 6 \end{pmatrix} \\
&= \begin{pmatrix} 0 \\ -\frac{3}{4} \end{pmatrix}
\end{aligned}$$

$$\begin{aligned}
\vec{x}^{(2)} &= \vec{x}^{(1)} - h\nabla F(\vec{x}^{(i)}) \\
&= \begin{pmatrix} 0 \\ -\frac{3}{4} \end{pmatrix} - \frac{1}{4} \times \nabla F(0, -\frac{3}{4}) \\
&= \begin{pmatrix} 0 \\ -\frac{3}{4} \end{pmatrix} - \frac{1}{4} \times \begin{pmatrix} 4 \times (0)^3 - 2 \times (0) + 2 \\ 8 \times (-\frac{3}{4}) \end{pmatrix} \\
&= \begin{pmatrix} 0 \\ -\frac{3}{4} \end{pmatrix} - \frac{1}{4} \times \begin{pmatrix} 2 \\ -6 \end{pmatrix} \\
&= \begin{pmatrix} -\frac{1}{2} \\ \frac{3}{4} \end{pmatrix}
\end{aligned}$$

Exercise 8.1. Take two steps of gradient descent for $F\colon \mathbb{R}^2 \to \mathbb{R}$ given by

$$F(x_1, x_2) = 2x_1^2 + 2x_2^2 - 2x_1x_2.$$

Remark 8.2. We always need an initial value for the gradient descent algorithm. If F is convex, the choice of starting point does not affect whether we converge or not towards the global minimiser, although it does affect how many iterations are needed to reach it. In practice, a better initial guess usually leads to faster convergence.

However, in the non-convex case, different starting values converge to different local minima. In fact, this is a way to try to find a global minimum through using lots of different local minima and finding which one minimises F.

8.1 Choice of the step size h

The choice of step size has a major effect on the performance of gradient descent. If h is too small, we do not make enough progress towards the minimiser, but if h is too large the

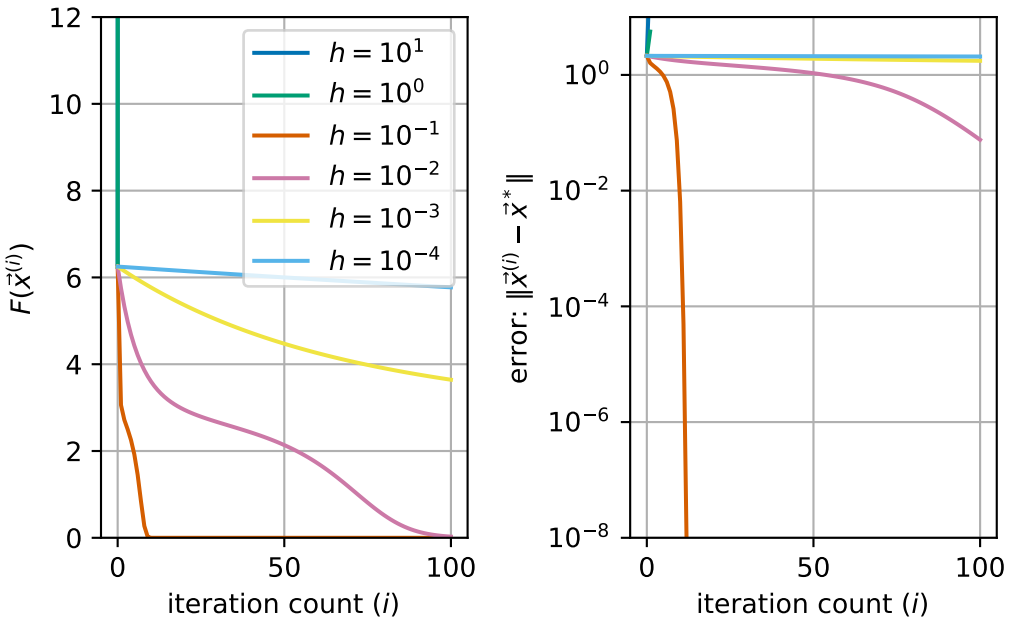
iteration may overshoot the minimum and function value may increase (e.g., if the function wiggles up again!).

Example 8.3. We consider again the function $F\colon \mathbb{R}^2 \to \mathbb{R}$ given by

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2.$$

From the plots in Figure 7.1, we can see we have a minimiser at $\vec{x}^* = (-1, 0)^T$ where $F^* = 0$.

We implement the iteration in code and plot the function values for different values of h



We see that:

- for $h = 10^1$ or 10^0, the time step is too large and the iterations quickly diverge;
- for $h = 10^{-3}$ or 10^{-4}, the time step is too small and the function does not decrease quickly enough;
- for $h = 10^{-1}$ or 10^{-2}, we see a nice decrease in the function values and quick convergence to the minimum!

Exercise 8.2. What would the *best* step size be? What if we know the second derivative of F as well? (*Hint*: think of 1d first, then think how this might work in higher dimensions). More on this later!

8.2 Stopping criteria

There are several possible stopping criteria available, similar to those used for other iterative schemes:

1. **Stop after a fixed number of iterations**: This is the simplest option. This does not guarantee that we are close to the correct solution but does stop us from computing if the solution will never converge (e.g. if the step size is too large). But how do we know we aren't just converging slowly?
2. **Stop when things stop changing**: We can monitor (i) the change in $\vec{x}^{(i)}$, (ii) the change in $F(\vec{x}^{(i)})$ or (iii) the magnitude of $\nabla F(\vec{x}^{(i)})$. All three can be used relatively straightforwardly to test if we have converged by comparing these values against a given tolerance TOL.
3. **Stop if the function values are too large**: This again avoids us continuing to compute when the iteration will not converge. But how large is too large as part of an iteration?

8.3 Relation to differential equations

There is another important way to derive the gradient descent algorithm: via a differential equation, often called the **gradient flow differential equation**:

Find $\vec{x}(t)$ such that

$$\vec{x}'(t) = -\nabla F(\vec{x}(t)) \quad \text{subject to the initial condition} \quad \vec{x}(0) = \vec{x}^{(0)}.$$

If we take the scalar product of this equation with $\vec{x}'(t)$, we see

$$\vec{x}'(t) \cdot \vec{x}'(t) = -\nabla F(\vec{x}(t))\vec{x}'(t).$$

We can identify the left hand side here as $\|\vec{x}'(t)\|^2$ which is always non-negative, and using the chain rule, we see that

$$-\nabla F(\vec{x}(t))\vec{x}'(t) = -\frac{\mathrm{d}}{\mathrm{d}t}(F(\vec{x}(t))).$$

Thus, we can see that

$$\frac{\mathrm{d}}{\mathrm{d}t}(F(\vec{x}(t))) = -\|\vec{x}'(t)\|^2 \leq 0,$$

or we can integrate additionally to see:

$$F(\vec{x}(t)) = F(\vec{x}^{(0)}) - \int_0^t \|\vec{x}'(s)\|^2 \,\mathrm{d}s \leq F(\vec{x}^{(0)}).$$

In words, these equations show that along solutions of the gradient flow equation, the quantity $F(\vec{x}(t))$ is non-increasing.

Now, if we take apply Euler's method to solve the gradient descent differential equation with step size h, we arrive at the scheme:

$$\vec{x}^{(i+1)} = \vec{x}^{(i)} - h\nabla F(\vec{x}^{(i)}).$$

So Euler's method applied to the gradient flow equation gives exactly the gradient descent algorithm!

Note there is no step size in the gradient flow differential equation: effectively, this is the case for $h \to 0$, and we see that h is small enough for the function value to *always* decrease.

Remark. The idea to think of gradient descent as a numerical method to solve the gradient flow differential equation allows us to think of other ways to generalise gradient flow.

For example, one approach called the *heavy ball* method (or momentum approach), uses a numerical method to solve the differential equation:

$$\vec{x}''(t) + \mu\vec{x}'(t) = -\nabla F(\vec{x}(t)) \quad \text{subject to appropriate initial conditions.}$$

8.4 Newton's method

In one dimension, we applied Newton's method to find a root of F' which we hoped would be a minimiser. We can relate the gradient descent algorithm to Newton's method for F':

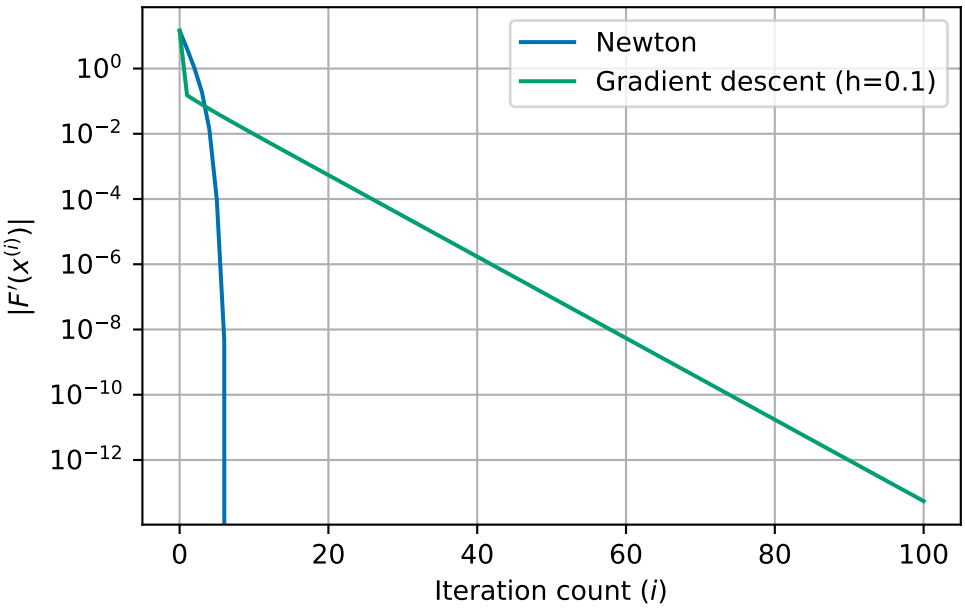
$$x^{(i+1)} = x^{(i)} - \frac{F'(x^{(i)})}{F''(x^{(i)})} \qquad \text{(Newton's method)}$$
$$x^{(i+1)} = x^{(i)} - hF'(x^{(i)}) \qquad \text{(gradient descent).}$$

We saw that Newton's method seem to converge very quickly, even faster than gradient descent!

Example 8.4. Consider the function from before:

$$F(x) = \frac{1}{4}x^4 - \frac{1}{12}x^3 - \frac{1}{2}x^2 + \frac{1}{4}x + \frac{1}{2}.$$

We can plot the value of $|F'(x^{(i)}|$ against the iteration count for the two methods we have written down here:



This can also be made to work for problems in $\mathbb{R}^n$ too. The idea is to replace division by the second derivative in one dimension with the solution of a linear system involving the Hessian in higher dimensions. More precisely, rather than finding the inverse of the Hessian, we solve a system of linear equations! At each step, we solve the system of linear equations:

$$(\nabla^2 F(\vec{x}^{(i)}))\vec{d} = -\nabla F(\vec{x}), \qquad \vec{x}^{(i+1)} = \vec{x}^{(i)} + \vec{d}.$$

Example 8.5. Consider the function $F\colon \mathbb{R}^2 \to \mathbb{R}$ given by

$$F(x_1, x_2) = x_1^4 - x_1^2 + 2x_1 + 4x_2^2 + 2.$$

We previously computed (Example 7.3) that

$$\nabla F(x_1, x_2) = (4x_1^3 - 2x_1 + 2, 8x_2),$$

and the Hessian is

$$\nabla^2 F(x_1, x_2) = \begin{pmatrix} 12x_1^2 - 2 & 0 \\ 0 & 8 \end{pmatrix}.$$

We start at $\vec{x}^{(0)} = \begin{pmatrix} 1 \\ \frac{3}{4} \end{pmatrix}$.

Iteration 1. We compute that

$$\nabla F(\vec{x}^{(0)}) = \begin{pmatrix} 4 \times (1)^3 - 2 \times (1) + 2 \\ 8 \times (3/4) \end{pmatrix} = \begin{pmatrix} 4 \\ 6 \end{pmatrix}$$

$$\nabla^2 F(\vec{x}^{(0)}) = \begin{pmatrix} 12 \times (1)^2 - 2 & 0 \\ 0 & 8 \end{pmatrix} = \begin{pmatrix} 10 & 0 \\ 0 & 8 \end{pmatrix}.$$

So we need to solve

$$\begin{pmatrix} 10 & 0 \\ 0 & 8 \end{pmatrix} \begin{pmatrix} d_1 \\ d_2 \end{pmatrix} = - \begin{pmatrix} 4 \\ 6 \end{pmatrix}.$$

In general, we could use Gaussian elimination (or another linearsolver) to solve this system, but we recognise that the matrix isdiagonal so we can immediately read off that the solution is:

$$\begin{pmatrix} d_1 \\ d_2 \end{pmatrix} = \begin{pmatrix} -\frac{2}{5} \\ -\frac{3}{4} \end{pmatrix} = \begin{pmatrix} -0.4 \\ -0.75 \end{pmatrix}.$$

Then we see that

$$\begin{aligned} \vec{x}^{(1)} &= \vec{x}^{(i)} + \begin{pmatrix} d_1 \\ d_2 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ \frac{3}{4} \end{pmatrix} + \begin{pmatrix} -\frac{2}{5} \\ -\frac{3}{4} \end{pmatrix} = \begin{pmatrix} \frac{3}{5} \\ 0 \end{pmatrix} = \begin{pmatrix} 0.6 \\ 0 \end{pmatrix} \end{aligned}$$

Iteration 2. We compute that

$$\begin{aligned} \nabla F(\vec{x}^{(1)}) &= \begin{pmatrix} 4 \times (3/5)^3 - 2 \times (3/5) + 2 \\ 8 \times (0) \end{pmatrix} = \begin{pmatrix} \frac{208}{125} \\ 0 \end{pmatrix} \\ \nabla^2 F(\vec{x}^{(1)}) &= \begin{pmatrix} 12 \times (3/5)^2 - 2 & 0 \\ 0 & 8 \end{pmatrix} = \begin{pmatrix} \frac{58}{25} & 0 \\ 0 & 8 \end{pmatrix}. \end{aligned}$$

So we need to solve

$$\begin{pmatrix} \frac{58}{25} & 0 \\ 0 & 8 \end{pmatrix} \begin{pmatrix} d_1 \\ d_2 \end{pmatrix} = - \begin{pmatrix} \frac{208}{125} \\ 0 \end{pmatrix}.$$

The matrix is again diagonal so we read off that the solution is:

$$\begin{pmatrix} d_1 \\ d_2 \end{pmatrix} = \begin{pmatrix} -\frac{104}{145} \\ 0 \end{pmatrix} = \begin{pmatrix} -0.717241379310345 \\ 0 \end{pmatrix}.$$

Then we see that

$$\begin{aligned} \vec{x}^{(2)} &= \vec{x}^{(i)} + \begin{pmatrix} d_1 \\ d_2 \end{pmatrix} \\ &= \begin{pmatrix} \frac{3}{5} \\ 0 \end{pmatrix} + \begin{pmatrix} -\frac{104}{145} \\ 0 \end{pmatrix} = \begin{pmatrix} -\frac{17}{145} \\ 0 \end{pmatrix} = \begin{pmatrix} -0.117241379310345 \\ 0 \end{pmatrix} \end{aligned}$$

9 An introduction to training neural networks

Important

These notes are to support the lecture which gives ideas of how to apply the gradient descent algorithm to train a simple neural network. This material is not directly examinable but understanding the material may help you with your exam preparations.

Module learning objective

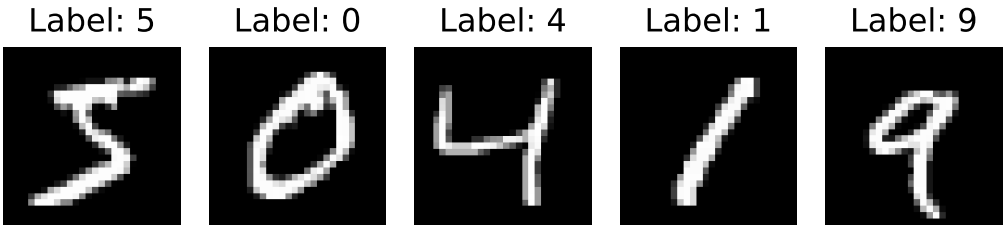
1. Explain at a high level how the gradient descent algorithm can be applied to train a simple neural network.

9.1 Problem

We want to train a neural network to recognise handwritten digits 0-9.

We will solve this using gradient descent to "train" the neural network. We will use the chain rule to find the gradient of a composite function.

We will use a dataset to help us called the MNIST handwritten digit dataset. It contains 70,000 images all labelled with the correct digit. Each image is a 28×28 greyscale image, so each image contains 784 pixel values. The task is to assign the image to one of the ten classes $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.



The input 28×28 images are "flattened" row-by-row by rearranging the pixel values into a single column vector $\vec{x} \in \mathbb{R}^{784}$.

For output, we want the network to give (approximately) a value 1 in the correct position of a 10 dimensional vector.

We express this mathematically by saying that we want a function:

$$\vec{\mathcal{F}}\colon \mathbb{R}^{784} \to \mathbb{R}^{10}$$

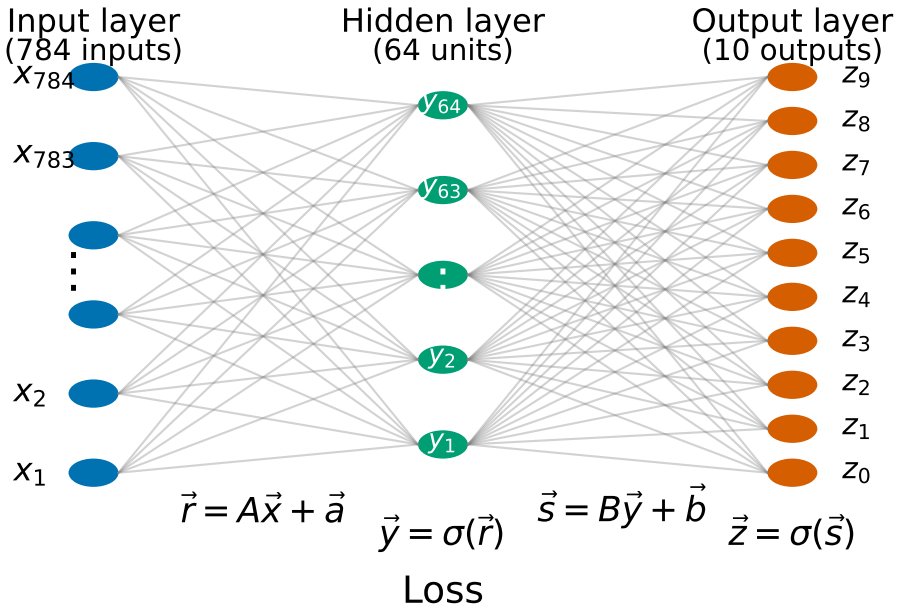
The function, we want to optimise should capture the given labels for a subset of all images (called the training set). This will leave further sample labels to test to see how well our method is performing. The optimisation will be over some parameters (which we define shortly) in the function $\vec{\mathcal{F}}$. We will call X the set of training images (of size N) and suppose we have a labelling function $L\colon X \to \mathbb{R}^{10}$ which takes an image in the training set and gives the correct label. We will use the notation $L = (L_1, \dots, L_1 0)$.

```
def indicator_matrix(targets, number_of_classes=10):
    # convert one label per image to a 10 dimensional output for comparison
    matrix = np.zeros((number_of_classes, targets.shape[0]), dtype=np.float32)
    matrix[targets, np.arange(targets.shape[0])] = 1.0
    return matrix
```

```
image 0 has L = 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0
image 1 has L = 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
image 2 has L = 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0
image 3 has L = 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
image 4 has L = 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0
```

9.2 The neural network

We will work with what is called a dense neural network with one input layer, one output layer and one hidden layer. The input layer represents the pixel values in the flattened image (784 values) and the output layer represents the output prediction of the correct label (10 values). The hidden layer is what brings everything together and we will use 64 values in this layer.



The function $\vec{\mathcal{F}}$ is composed of the following functions.

1. We first form the affine map $\vec{r} = \vec{r}(\vec{x}) = A\vec{x} + \vec{a}$ for $A \in \mathbb{R}^{64\times 784}$ and $\vec{a} \in \mathbb{R}^{64}$
2. Then, we apply a sigmoid function to each component of the output $\vec{r}$: $\vec{y} = \vec{y}(\vec{r})$ with $y_k = \sigma(r_k)$. The output $\vec{y}$ is called the "hidden layer". In this example, we will use the function σ
$$\sigma(t) = \frac{1}{1 + \exp(-t)}.$$
3. Then, we form a second affine map $\vec{s} = B\vec{y} + \vec{b}$ for $B \in \mathbb{R}^{10\times 64}$ and $\vec{b} \in \mathbb{R}^{10}$
4. Finally, we apply the sigmoid to each component of the output $\vec{s}$: $\vec{z} = \vec{z}(\vec{s})$ with $z_i = \sigma(s_i)$.

The full map $\vec{\mathcal{F}}$ is the composition:

$$\vec{\mathcal{F}}(\vec{x}) = \vec{z}(\vec{s}(\vec{y}(\vec{r}(\vec{x}))))$$

This gives function $\vec{\mathcal{F}}\colon \mathbb{R}^{784} \to \mathbb{R}^{10}$.

In the mathematical discussion below, we first describe the network for a single image $\vec{x} \in \mathbb{R}^{784}$. In the implementation, however, we process many images at once using matrix notation. We store all N flattened training images as the columns of a matrix $\mathbb{R}^{784\times N}$ and store the labels as $\mathbb{R}^{10\times N}$. The numpy code is therefore a *vectorised* version of the same formulas, applied to all the training samples simultaneously.

```python
def sigma(t):
    # sigmoid activation
    return 1.0 / (1.0 + np.exp(-t))


def forward_map(x, A, a, B, b):
    # broadcast multiplication in all values at the same time using [:, None]
    r = A @ x + a[:, None]
    y = sigma(r)

    s = B @ y + b[:, None]
    z = sigma(s)

    return (r, y, s, z)
```

9.3 The objective function

To train the network, we need a quantity to minimise. Since we want the neural network output to match the labels, the simplest choice uses the square of the Euclidean norm. For a single (flattened) image $\vec{x}$ this means

$$J(\vec{x}) = \frac{1}{2}\|\vec{L}(\vec{x}) - \vec{\mathcal{F}}(\vec{x})\|^2$$

For the full training set, we simply average this quantity over all N (flattened) images in X.

$$F = \frac{1}{N}\sum_{\vec{x}\in X} J(\vec{x}) = \frac{1}{2N}\sum_{\vec{x}\in X} \|\vec{L}(\vec{x}) - \vec{\mathcal{F}}(\vec{x})\|^2.$$

As we mentioned previously, we want to optimise for parameters in the function $\vec{\mathcal{F}}$. Precisely, these are the values A (size 64×784), a (size 64), B (size 10×64) and $\vec{b}$ (size 10). In total this corresponds to $50,890$ parameters so we can say that $F\colon \mathbb{R}^{50\,890} \to \mathbb{R}$.

```python
def objective(z, targets):
    # objective function F
    L = indicator_matrix(targets, number_of_classes=z.shape[0])
    return 0.5 * np.mean(np.sum((L - z) ** 2, axis=0))


def classification_accuracy(z, targets):
    # measures how many targets we have correct
```

```
    predicted_targets = np.argmax(z, axis=0)
    return np.mean(predicted_targets == targets)


def evaluate(x, targets, A, a, B, b):
    _, _, _, z = forward_map(x, A, a, B, b)
    current_objective = objective(z, targets)
    current_accuracy = classification_accuracy(z, targets)
    return current_objective, current_accuracy
```

9.4 Gradient descent

The gradient descent algorithm will update the parameters $A, \vec{a}, B$ and $\vec{b}$ using the rules:

$$A_{ij} \leftarrow A_{ij} - \alpha \frac{\partial F}{\partial A_{ij}}, \qquad a_i \leftarrow a_i - \alpha \frac{\partial F}{\partial a_i},$$
$$B_{ij} \leftarrow B_{ij} - \alpha \frac{\partial F}{\partial B_{ij}}, \qquad b_i \leftarrow b_i - \alpha \frac{\partial F}{\partial b_i}.$$

Note, here we are not tracking the iteration numbers in this notation.

The values of the parameters are well hidden in our definition of both the neural network operator and our objective function! But we can see that there are lots of compositions going on that need resolving!

We compute the gradients component by component in turn, working from outer-most to inner-most functions by applying the chain rule. The code then implements the same calculations in vectorised matrix form so that all training examples are processed simultaneously. To help writing these functions, we will make use of the Kronecker delta (identity matrix) δ which has entries $\delta_{ij} = 1$ if $i = j$ and 0 otherwise.

1. We have
$$F(\vec{z}) = \frac{1}{2N} \sum_{\vec{x} \in X} \|\vec{L}(\vec{x}) - \vec{z}\|^2,$$
so,
$$\frac{\partial F(\vec{z})}{\partial z_i} = \frac{1}{N} \sum_{\vec{x} \in X} z_i - L_i(\vec{x}).$$

2. Next, we have $\vec{z}_i = \sigma(s_i)$ so we have
$$\frac{\partial z_i}{\partial s_j} = \delta_{ij} \sigma'(s_j).$$

So we can compute that

$$\frac{\partial F(\vec{s})}{\partial s_j} = \sum_{i=0}^{9} \frac{\partial J(\vec{z})}{\partial z_i}\frac{\partial z_i}{\partial s_j} = \sum_{i=0}^{9} \frac{\partial J(\vec{z})}{\partial z_i}\delta_{ij}\sigma'(s_j) = \frac{\partial J(\vec{z})}{\partial z_j}\sigma'(s_j)$$
$$= \frac{1}{N}\sum_{\vec{x}\in X}(z_j - L_j(\vec{x}))\sigma'(s_j).$$

3. We can compute partial derivatives with respect to the second matrix and offset vector B and $\vec{b}$. We have $\vec{s} = B\vec{y} + \vec{b}$, so

$$\frac{\partial s_j}{\partial B_{kl}} = \delta_{jk}y_l, \quad \text{and} \quad \frac{\partial s_j}{\partial b_k} = \delta_{jk},$$

and we can compute that

$$\frac{\partial F(B,\vec{b})}{\partial B_{kl}} = \sum_{j=0}^{9} \frac{\partial F(\vec{s})}{\partial s_j}\frac{\partial s_j}{\partial B_{kl}} = \sum_{j=0}^{9} \frac{\partial F(\vec{s})}{\partial s_j}\delta_{jk}y_l = \frac{\partial F(\vec{s})}{\partial s_k}y_l$$
$$= \frac{1}{N}\sum_{\vec{x}\in X}(z_k - L_k(\vec{x}))\sigma'(s_k)y_l,$$
$$\frac{\partial F(B,\vec{b})}{\partial b_k} = \sum_{j=0}^{9} \frac{\partial F(\vec{s})}{\partial s_j}\frac{\partial s_j}{\partial b_k} = \sum_{j=0}^{9} \frac{\partial F(\vec{s})}{\partial s_j}\delta_{jk} = \frac{\partial F(\vec{s})}{\partial s_k}$$
$$= \frac{1}{N}\sum_{\vec{x}\in X}(z_k - L_k(\vec{x}))\sigma'(s_k).$$

4. Then we continue with derivatives with respect to $\vec{y}$. Recall, we have $\vec{s} = B\vec{y} + \vec{b}$ so

$$\frac{\partial s_j}{\partial y_k} = B_{jk},$$

and we can compute that

$$\frac{\partial F}{\partial y_k} = \sum_{j=0}^{9} \frac{\partial F}{\partial s_j}\frac{\partial s_j}{\partial y_k} = \sum_{j=0}^{9} \frac{\partial F}{\partial s_j}B_{jk} = \frac{1}{N}\sum_{\vec{x}\in X}\sum_{j=0}^{9}(z_j - L_j(\vec{x}))\sigma'(s_j)B_{jk}.$$

5. Then we compute derivatives with respect to $\vec{r}$. We have $y_k = \sigma(r_k)$ so we can compute that

$$\frac{\partial y_k}{\partial r_l} = \delta_{kl}\sigma'(r_l),$$

and

$$\frac{\partial F}{\partial r_l} = \sum_{k=1}^{64} \frac{\partial F}{\partial y_k}\frac{\partial y_k}{\partial r_l} = \sum_{k=1}^{64} \frac{\partial F}{\partial y_k}\delta_{kl}\sigma'(r_l) = \frac{\partial F}{\partial y_l}\sigma'(r_l)$$
$$= \frac{1}{N}\sum_{\vec{x}\in X}\sum_{j=0}^{9}(z_j - L_j(\vec{x}))\sigma'(s_j)B_{jl}\sigma'(r_l).$$

6. Finally, we can compute the derivatives with respect the first parametric matrix and offset vector From $\vec{r} = A\vec{x} + a$, we have

$$\frac{\partial r_l}{\partial A_{mn}} = \delta_{lm} x_n, \quad \text{and} \quad \frac{\partial r_l}{\partial a_m} = \delta_{lm},$$

so

$$\begin{aligned}
\frac{\partial F}{\partial A_{mn}} &= \sum_{l=1}^{64} \frac{\partial F}{\partial r_l} \frac{\partial r_l}{\partial A_{mn}} = \sum_{l=1}^{64} \frac{\partial F}{\partial r_l} \delta_{lm} x_n = \frac{\partial F}{\partial r_m} x_n \\
&= \frac{1}{N} \sum_{\vec{x} \in X} \sum_{j=0}^{9} (z_j - L_j(\vec{x})) \sigma'(s_j) B_{jm} \sigma'(r_m) x_n, \\
\frac{\partial F}{\partial a_m} &= \sum_{l=1}^{64} \frac{\partial F}{\partial r_l} \frac{\partial r_l}{\partial a_m} = \sum_{l=1}^{64} \frac{\partial F}{\partial r_l} \delta_{lm} = \frac{\partial F}{\partial r_m} \\
&= \frac{1}{N} \sum_{\vec{x} \in X} \sum_{j=0}^{9} (z_j - L_j(\vec{x})) \sigma'(s_j) B_{jm} \sigma'(r_m).
\end{aligned}$$

This can be implemented in code.

We use one final helper function. The sigmoid function is often chosen to help simplify the calculation of the derivative of the σ function. We can use this same trick here:

```
def sigma_prime_from_value(sigma_value):
    # derivative of sigmoid when the sigmoid value is already known
    return sigma_value * (1.0 - sigma_value)
```

```
# ----------------------------
# Parameter initialisation
# ----------------------------


def initialise_parameters(random_number_generator):
    A = random_number_generator.normal(
        0.0,
        np.sqrt(1.0 / input_dimension),
        size=(hidden_dimension, input_dimension),
    ).astype(np.float32)

    a = np.zeros((hidden_dimension,), dtype=np.float32)

    B = random_number_generator.normal(
```

```
        0.0,
        np.sqrt(1.0 / hidden_dimension),
        size=(output_dimension, hidden_dimension),
    ).astype(np.float32)

    b = np.zeros((output_dimension,), dtype=np.float32)

    return A, a, B, b


# ----------------------------
# Gradient descent
# ----------------------------
def train_network(x, targets, test_inputs, test_targets):
    random_number_generator = np.random.default_rng(seed)

    A, a, B, b = initialise_parameters(random_number_generator)

    L = indicator_matrix(targets, output_dimension)
    N = x.shape[1]

    accuracy_measures = {
        "objective": [],
        "test objective": [],
        "test accuracy": [],
    }

    for _ in range(1, iterations + 1):
        # Forward map
        (_, y, _, z) = forward_map(x, A, a, B, b)

        # ------------------------------------------------------
        # Gradient via chain rule (backpropagation)
        # ------------------------------------------------------
        sigma_p_z = sigma_prime_from_value(z)
        sigma_p_y = sigma_prime_from_value(y)

        # gradient of objective wrt to s
        dF_ds = ((z - L) * sigma_p_z) / N  # componentwise multiply

        # Gradients for the second parameter matrix and offset vector
        # dF / dB
```

```
        dF_dB = dF_ds @ y.T
        # dF / db
        dF_db = np.sum(dF_ds, axis=1)

        # Move the gradient back to the hidden layer
        # dF / dy
        dF_dy = B.T @ dF_ds

        # Gradient of objective wrt to r
        # dF / dr
        dF_dr = dF_dy * sigma_p_y  # component wise multiply

        # Gradients for the first parameter matrix and offset vector
        # dF / dA
        dF_dA = dF_dr @ x.T
        # dF / da
        dF_da = np.sum(dF_dr, axis=1)

        # ------------------------------------------------------
        # Gradient descent step
        # ------------------------------------------------------
        A = A - alpha * dF_dA
        a = a - alpha * dF_da
        B = B - alpha * dF_dB
        b = b - alpha * dF_db

        # test on training data
        (_, _, _, z) = forward_map(x, A, a, B, b)
        current_objective = objective(z, targets)

        # test on test data
        test_objective, test_accuracy = evaluate(
            test_inputs, test_targets, A, a, B, b
        )

        accuracy_measures["objective"].append(current_objective)
        accuracy_measures["test objective"].append(test_objective)
        accuracy_measures["test accuracy"].append(100.0 * test_accuracy)

    return (A, a, B, b, accuracy_measures)
```

9.5 Testing it out

We set some important parameters for the algorithm

We also use an extra function to pick out the output $\vec{z}$ with the highest value to make our prediction of the final output value:

```
def predict(x, A, a, B, b):
    _, _, _, z = forward_map(x, A, a, B, b)
    return np.argmax(z, axis=0)
```

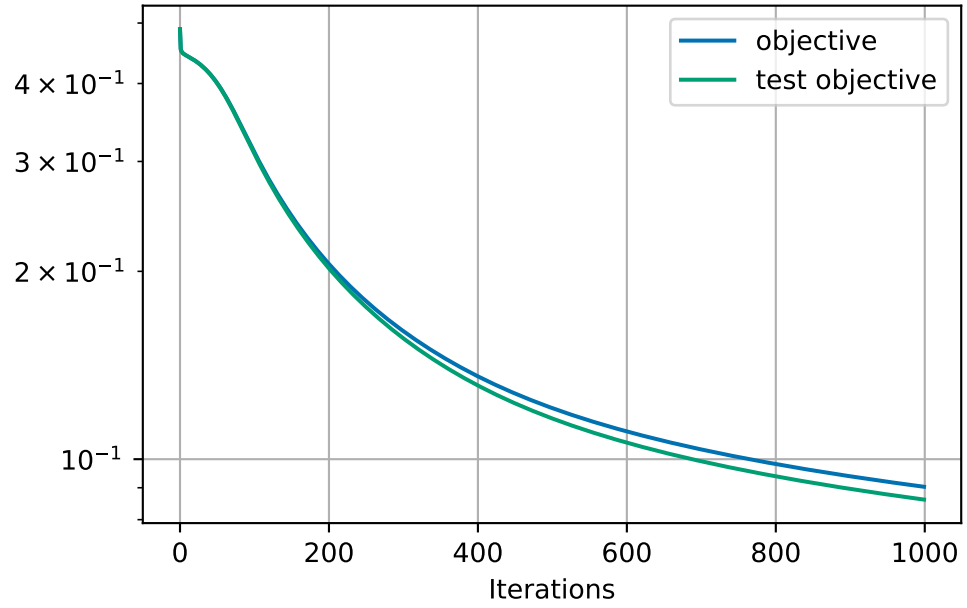
We can now do the training:

```
training_inputs, training_targets, test_inputs, test_targets = load_mnist()

# take transpose so that each column is a flattened image
training_inputs = training_inputs.T
test_inputs = test_inputs.T

print("training inputs shape:", training_inputs.shape)
print("training targets shape:", training_targets.shape)
print("test inputs shape:    ", test_inputs.shape)
print("test targets shape:   ", test_targets.shape)
print()

start = time.perf_counter()

A, a, B, b, accuracy_measures = train_network(
    training_inputs,
    training_targets,
    test_inputs,
    test_targets,
)

end = time.perf_counter()

print(f"completed {iterations} iterations in time {end - start:.4f}s.")
```

```
training inputs shape: (784, 60000)
training targets shape: (60000,)
test inputs shape:     (784, 10000)
test targets shape:    (10000,)
```

```
completed 1000 iterations in time 190.7765s.
```

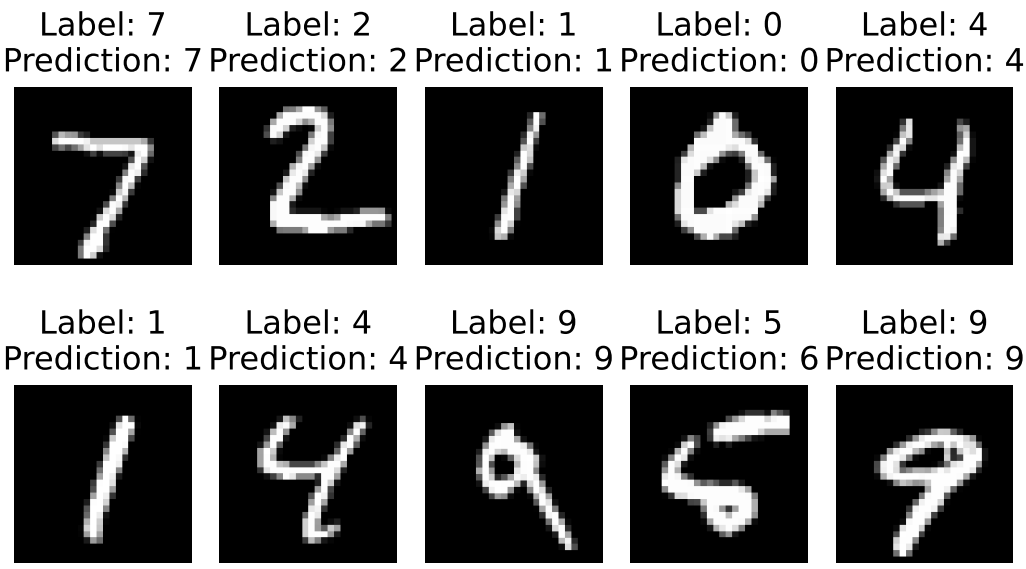
Let's see how well we've done:




```
Final train objective: 0.090323
Final test objective:  0.086149
Final test accuracy:   90.97%
```

We see that we are achieving more than 90% accuracy within 1000 iterations and we have not converged to the best solution yet.

Let's see some examples of the classes and our predictions



I don't think this is too bad!

For one incorrect prediction (predicted 6 instead of 5), we can also check all $\vec{z}$ values that have:

```
z values:
0: 0.032788
1: 0.002198
2: 0.085792
3: 0.000049
4: 0.108685
5: 0.025088
6: 0.593296
7: 0.001090
8: 0.008090
9: 0.012378
```

We see that we were quite confident in our prediction and weren't close to predicting 5!

9.6 Summary and outlook

We have demonstrated the simplest possible approach to training a neural network for the task for recognising hand-written numbers.

There are a few changes recommended to improve the performance of our neural network.

1. Replace the second sigmoid function by a *softmax*. This would mean $\vec{z} = \vec{z}(\vec{s})$ is given by:

$$z_i(\vec{s}) = \frac{\exp(s_i)}{\sum_{j=0}^{9} \exp(s_j)},$$

and replace the L2 loss function by a *cross-entropy loss*. Then J would be given by:

$$J(\vec{x}) = -\sum_{i=0}^{9} L_i(\vec{x}) \log z_i.$$

2. Replace the first sigmoid with a different activation function such as $ReLU$:

$$y_k = \max(0, r_k).$$

3. Add one or more network layers…

Slides

- Introduction to dynamical problems
- Analysis of errors
- Systems of differential equations